

数据结构

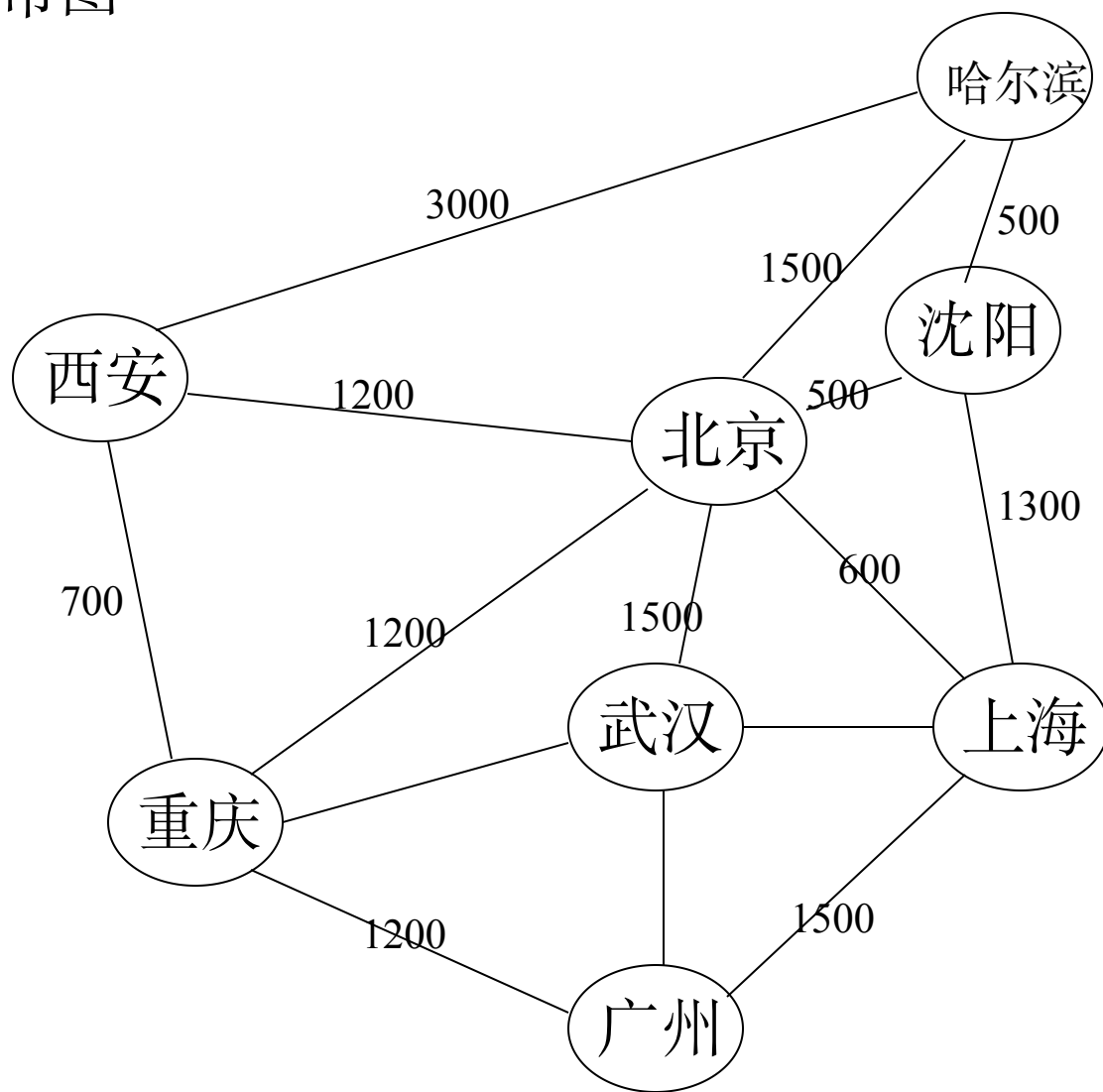
Ch7 图结构

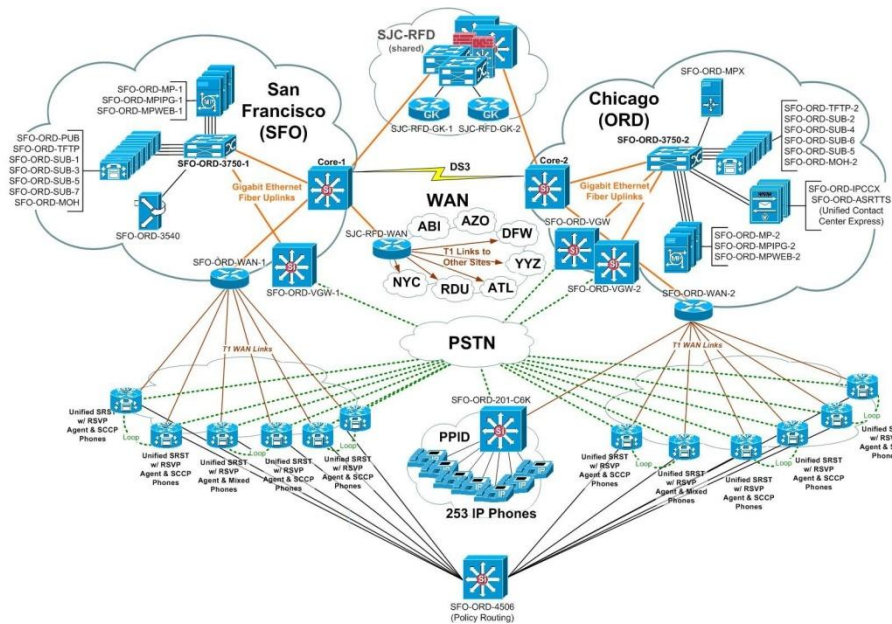
北京邮电大学 计算机学院

第7章 图结构

- 7.1 图的定义和术语
- 7.2 图的存储结构
- 7.3 图的遍历
- 7.4 图的连通性问题（连通分量与最小生成树）
- 7.5 有向无环图及其应用（拓扑排序、关键路径）
- 7.6 最短路径

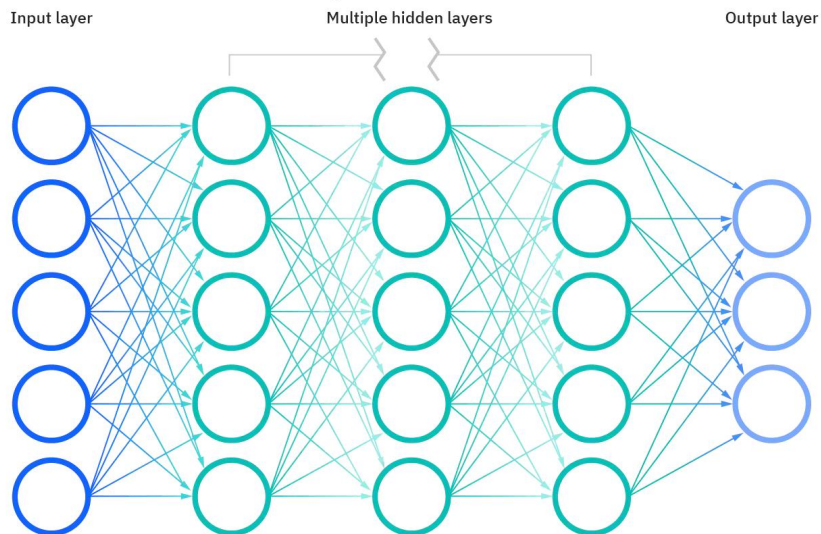
航线分布图



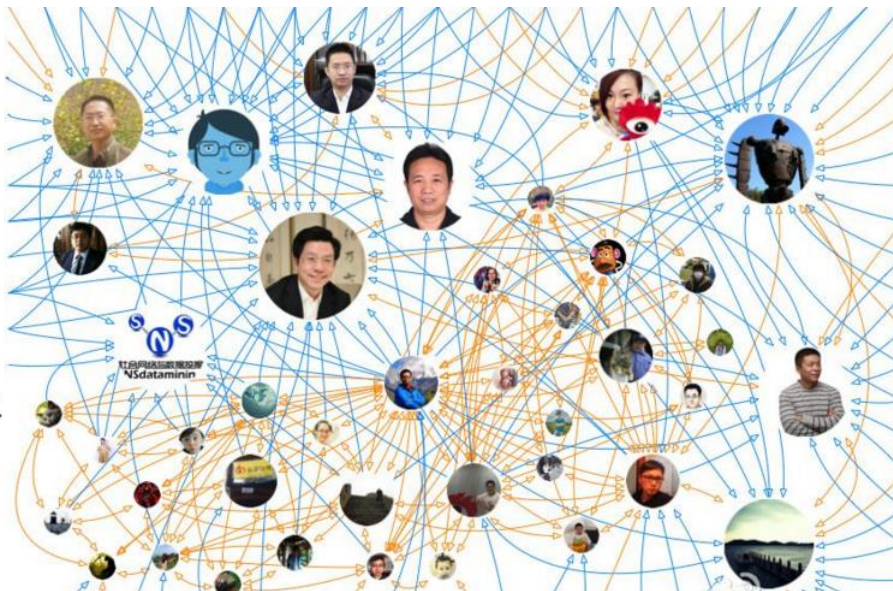


计算机网络

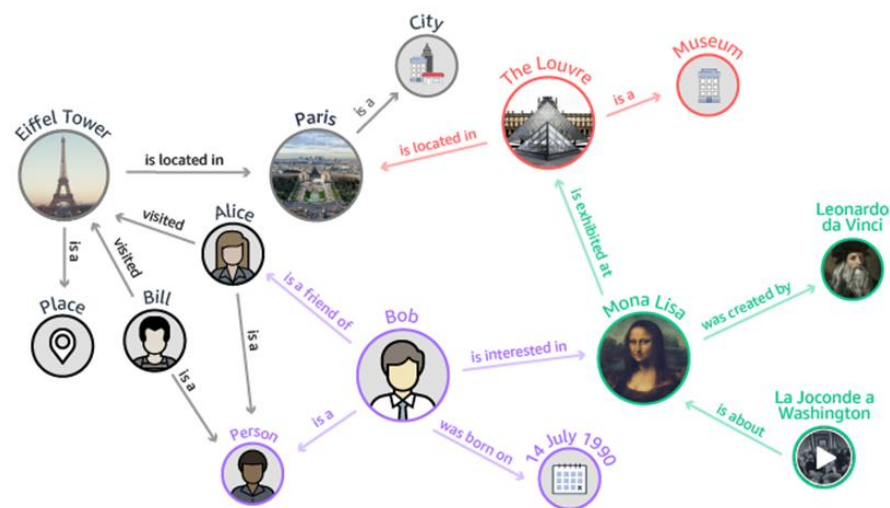
Deep neural network



神经网络



社交网络



知识图谱

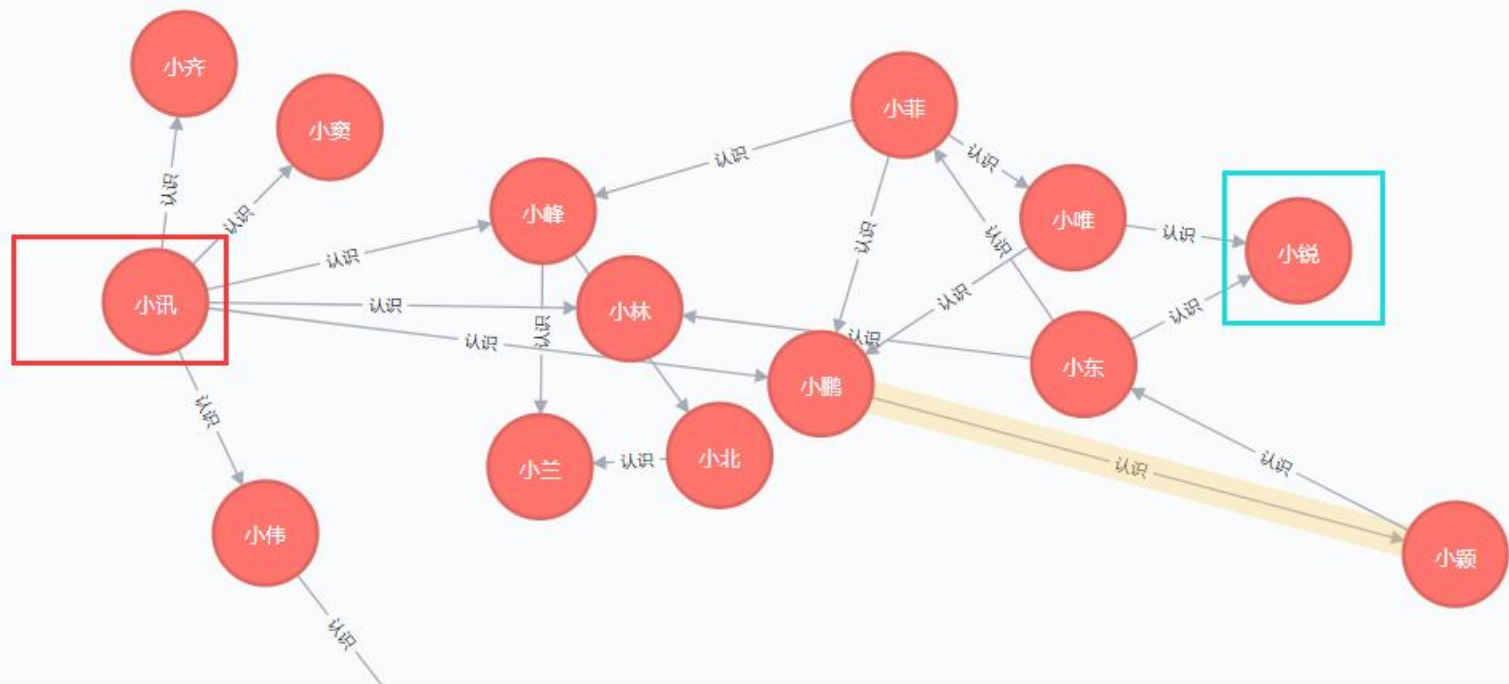
DB-Engines Ranking of Graph DBMS

<https://db-engines.com/en/ranking/graph+dbms>

☐ include secondary database models

43 systems in ranking, December 2024

Rank			DBMS	Database Model	Score		
Dec 2024	Nov 2024	Dec 2023			Dec 2024	Nov 2024	Dec 2023
1.	1.	1.	Neo4j 	Graph	43.07	+0.37	-6.92
2.	2.	2.	Microsoft Azure Cosmos DB 	Multi-model 	23.06	-0.89	-11.48
3.	3.	3.	Aerospike	Multi-model 	5.34	+0.01	-1.84
4.	4.	4.	Virtuoso	Multi-model 	3.58	-0.29	-1.69
5.	5.	5.	ArangoDB	Multi-model 	2.96	-0.13	-1.47
6.	6.	6.	OrientDB	Multi-model 	2.87	-0.10	-1.23
7.	7.	 8.	GraphDB 	Multi-model 	2.78	-0.11	+0.01
8.	8.	 7.	Memgraph 	Graph	2.71	-0.02	-0.47
9.	9.	9.	Amazon Neptune	Multi-model 	2.17	+0.00	-0.59
10.	 12.	 11.	JanusGraph	Graph	1.76	-0.02	-0.53



查询出所有小讯到小锐的关系最短路径，语句如下：

```
MATCH n=shortestPath((a:朋友圈{姓名:"小讯"})-[*]-(b:朋友圈{姓名:"小锐"}))  
return n
```

7.1 图的定义和术语

非线性结构，数据元素之间呈多对多的关系。

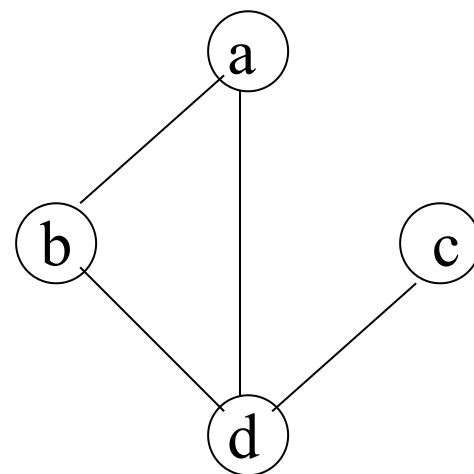
7.1.1 图的定义

图：是由**顶点集合**和一个描述顶点之间关系----**边的集合**组成。

$\text{Graph}=(V,E)$

V：顶点(数据元素)的有穷非空集合。

E：边的有穷集合。



7.1.1 图的定义

ADT Graph{

数据对象V: V是具有相同特性的数据元素的集合, 称为顶点集。

数据关系R:

$$R=\{VR\}$$

$VR=\{<v, w> | v, w \in V \text{ 且 } P(v, w) \}$, $<v, w>$ 表示从v到w的弧, 谓词 $P(v, w)$ 定义了弧 $<v, w>$ 的意义或信息}

基本操作P:

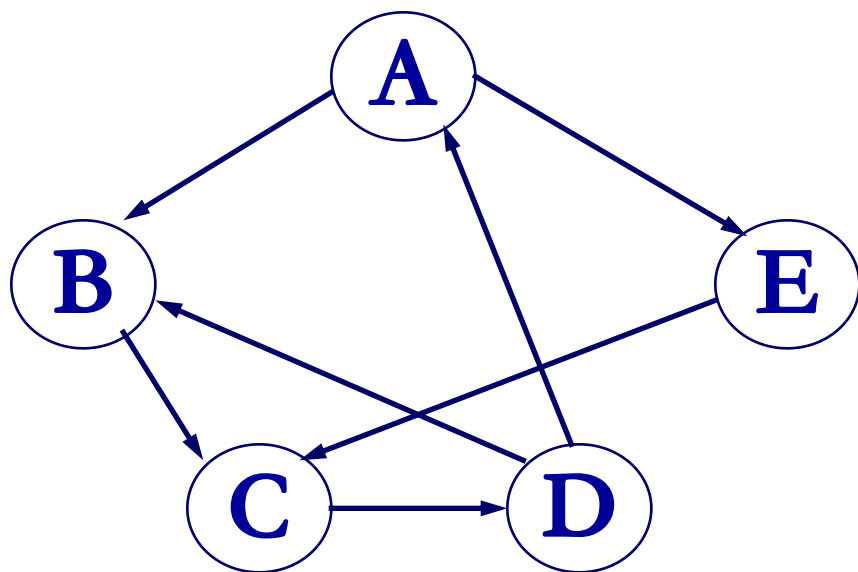
见: 7.1.3 图的基本操作

}

例1: $G_1 = (V_1, VR_1)$

$V_1 = \{A, B, C, D, E\}$

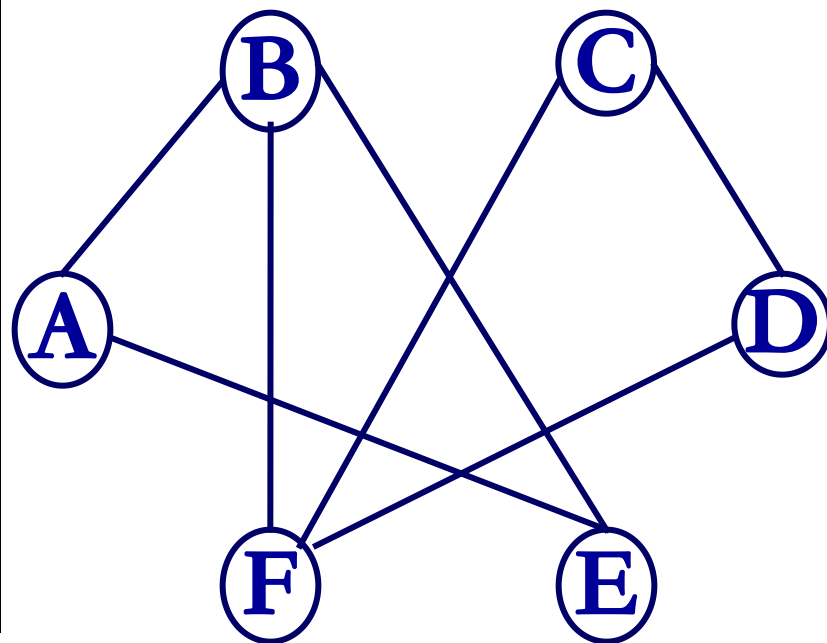
$VR_1 = \{ \langle A, B \rangle, \langle A, E \rangle, \langle B, C \rangle, \langle C, D \rangle, \langle D, B \rangle, \langle D, A \rangle, \langle E, C \rangle \}$



例2: $G_2 = (V_2, VR_2)$

$V_2 = \{A, B, C, D, E, F\}$

$VR_2 = \{ (A, B), (A, E), (B, E), (C, D), (D, F), (B, F), (C, F) \}$



7.1.2 图的相关术语

顶点(Vertex): 数据元素所构成的结点。

弧(Arc): 顶点关系集合中的一个元素 $\langle i, j \rangle$, 表示从顶点 i 到顶点 j 的一条弧。

有向图(Digraph): 弧(Arc)的顶点偶对是有序的。对弧 $\langle v_i, v_j \rangle$ 而言,

v_i 是**弧尾(Tail)**/初始点; v_j 是**弧头(Head)**/终端点。

无向图(Undigraph) 弧的顶点偶对是无序的。

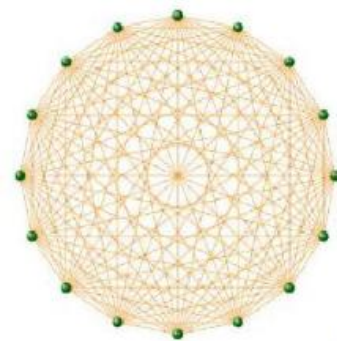
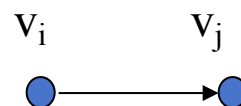
(v_i, v_j) 和 (v_j, v_i) 代表同一条**边(Edge)**($i \neq j$)。

(无向)完全图 每个顶点与其余顶点都有边的无向图。

顶点数为 n 时, 边数 $e = n(n-1)/2$

有向完全图 每个顶点与其余顶点都有弧的有向图。

顶点数为 n 时, 弧数 $e = n(n-1)$



7.1.2 图的相关术语

稀疏图(Sparse graph) 有很少边或弧的图。 ($e < n \log n$)

稠密图(Dense graph) 有较多边或弧的图。

实际的图大多数是**稀疏**的

WWW (Stanford-Berkeley):	$N=319,717$	$\langle k \rangle=9.65$
Social networks (LinkedIn):	$N=6,946,668$	$\langle k \rangle=8.87$
Communication (MSN IM):	$N=242,720,596$	$\langle k \rangle=11.1$
Coauthorships (DBLP):	$N=317,080$	$\langle k \rangle=6.62$
Internet (AS-Skitter):	$N=1,719,037$	$\langle k \rangle=14.91$
Roads (California):	$N=1,957,027$	$\langle k \rangle=2.82$
Protein (S. Cerevisiae):	$N=1,870$	$\langle k \rangle=2.39$

(Source: Leskovec et al., *Internet Mathematics*, 2009)

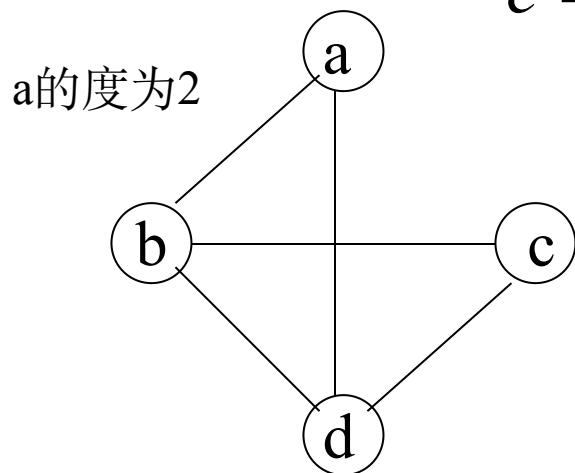
顶点的度(Degree) 与该顶点相关联的边的数目，记为 $D(v)$ 。

入度ID(v): 有向图中，以该顶点为弧头的弧数目。

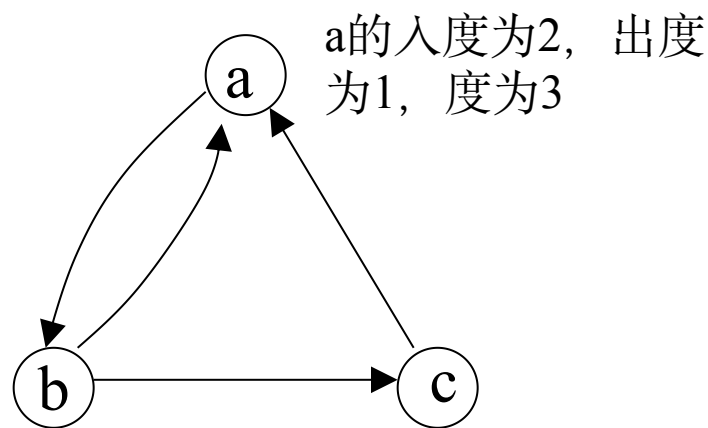
出度OD(v): 有向图中，以该顶点为弧尾的弧数目。

顶点数 n 、边数 e 和度数之间的关系：

$$e = \frac{1}{2} \sum_{i=1}^n D(v_i)$$



无向图

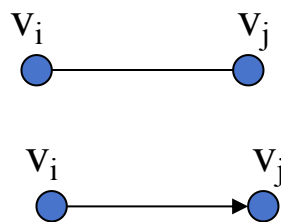


有向图

邻接 有边/弧相连的**两个顶点**之间的关系。

存在 (v_i, v_j) , 则称 v_i 和 v_j 互为**邻接点**(Adjacent);

存在 $\langle v_i, v_j \rangle$, 则称 v_i **邻接到** v_j , v_j **邻接于** v_i

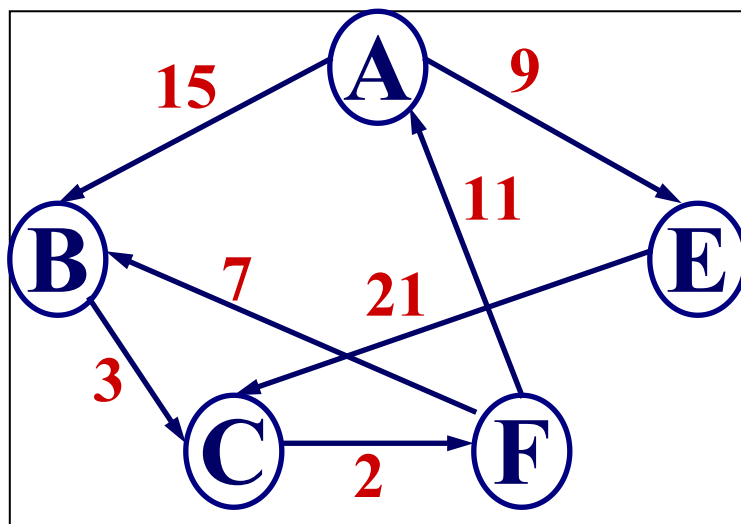
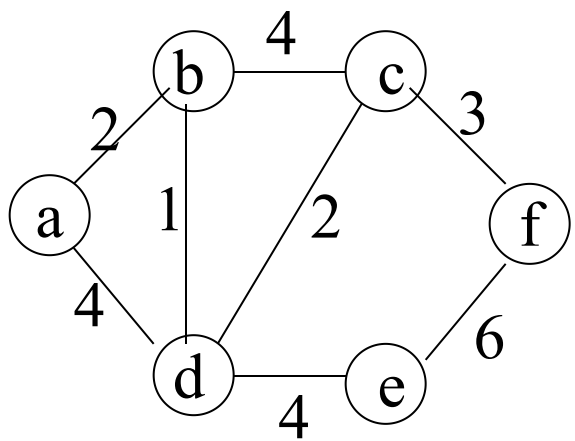


关联(依附) **边/弧与顶点**之间的关系。

存在 (v_i, v_j) / $\langle v_i, v_j \rangle$, 则称该边/弧关联于 v_i 和 v_j

权(Weight) 图中的边或弧具有一定的大小的概念。

网(Network) 边/弧带权的图。



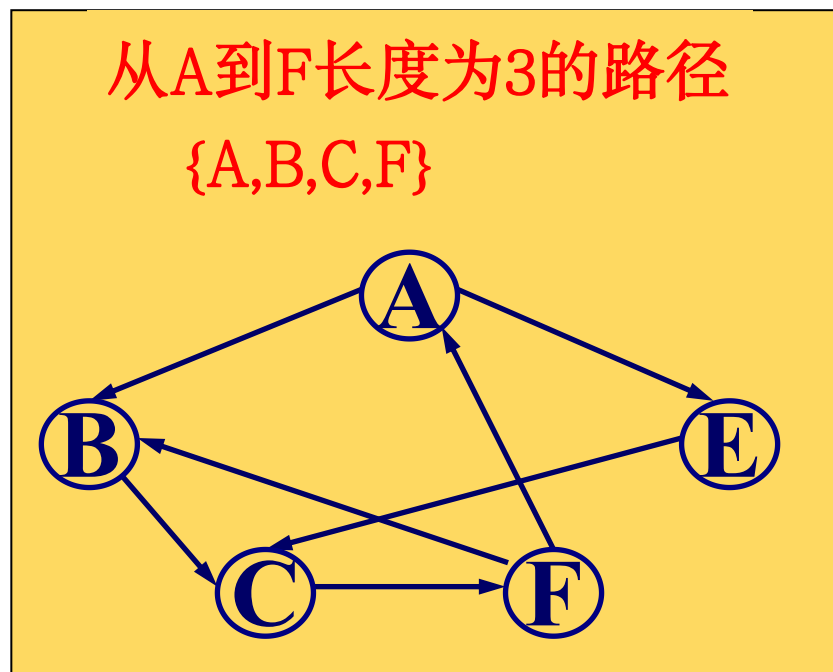
路径： 接续的边构成的顶点序列。

路径长度： 路径上边或弧的数目/权值之和。

回路(环)： 第一个顶点和最后一个顶点相同的路径。

简单路径： 序列中顶点均不相同的路径。

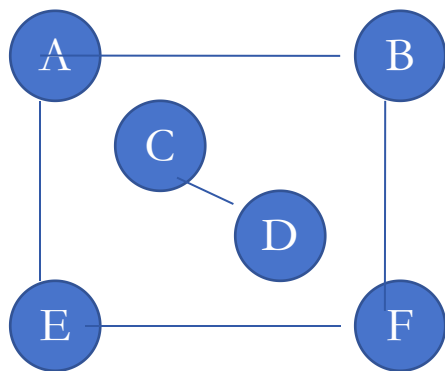
简单回路(简单环)： 除路径起点和终点相同外，其余顶点均不相同的路径。



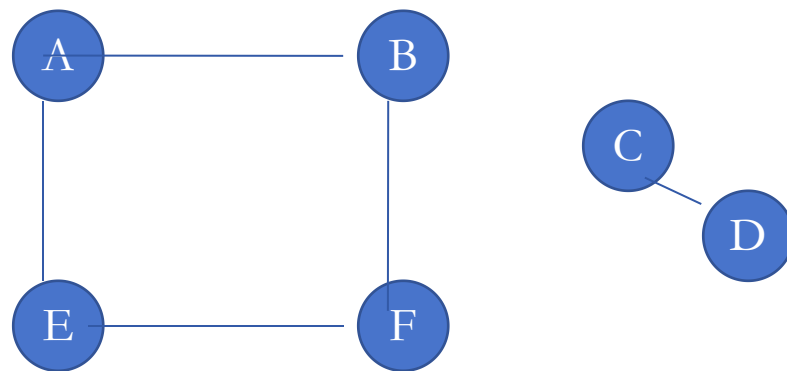
连通： 无向图中若从顶点 V 到顶点 V' 存在路径，则称 V 和 V' 是连通的。

连通图： 无向图中，任何一对顶点间都存在路径。

连通分量： 无向图中的极大连通子图。



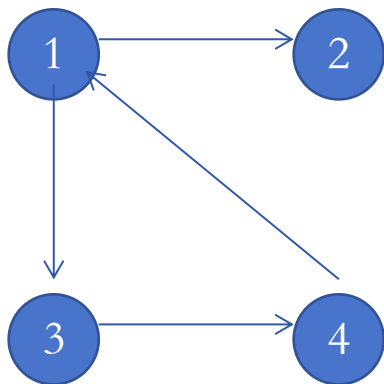
非连通图



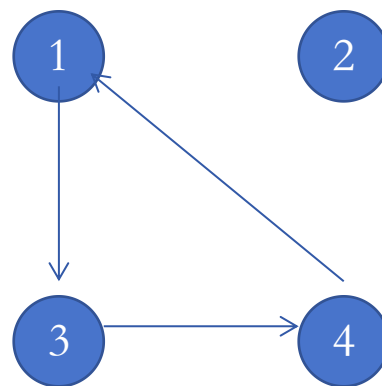
两个连通分量

强连通图 有向图中，任何一对顶点间都存在路径。

强连通分量 有向图中的极大连通子图。

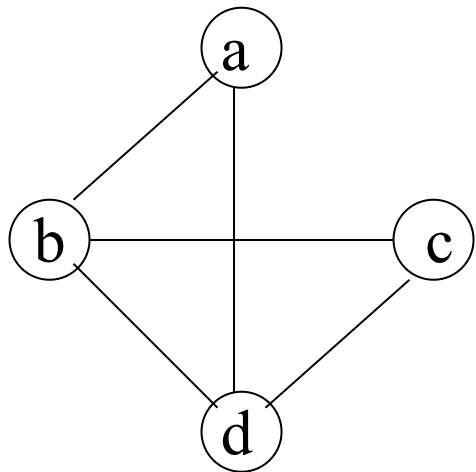


非强连通有向图

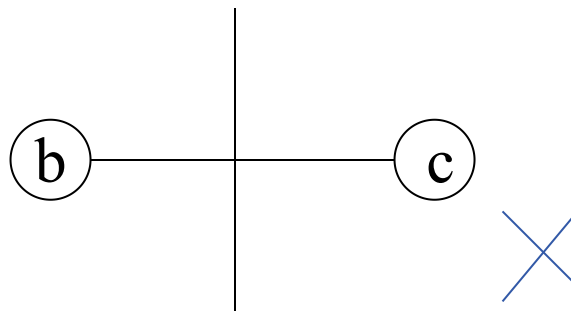


两个强连通分量

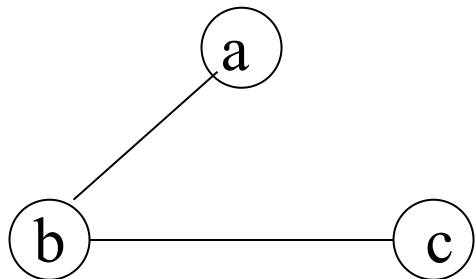
子图： 对于图 $G=(V,E)$ 和 $G'=(V',E')$ ，如果 $V'\subseteq V$ ， $E'\subseteq E$ ，且 E' 关联的顶点都在 V' 中，则称 G' 是 G 的子图。



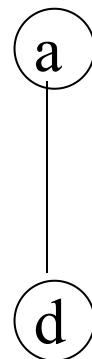
(1)



(4)



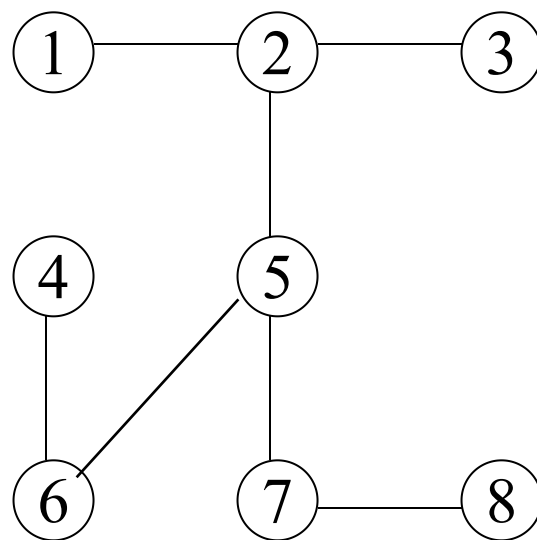
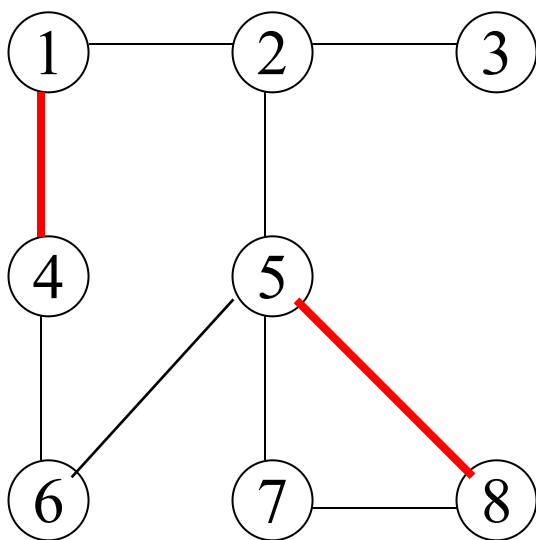
(2)



(3)

生成子图： 由图的全部顶点和部分边组成的子图称为原图的生成子图。

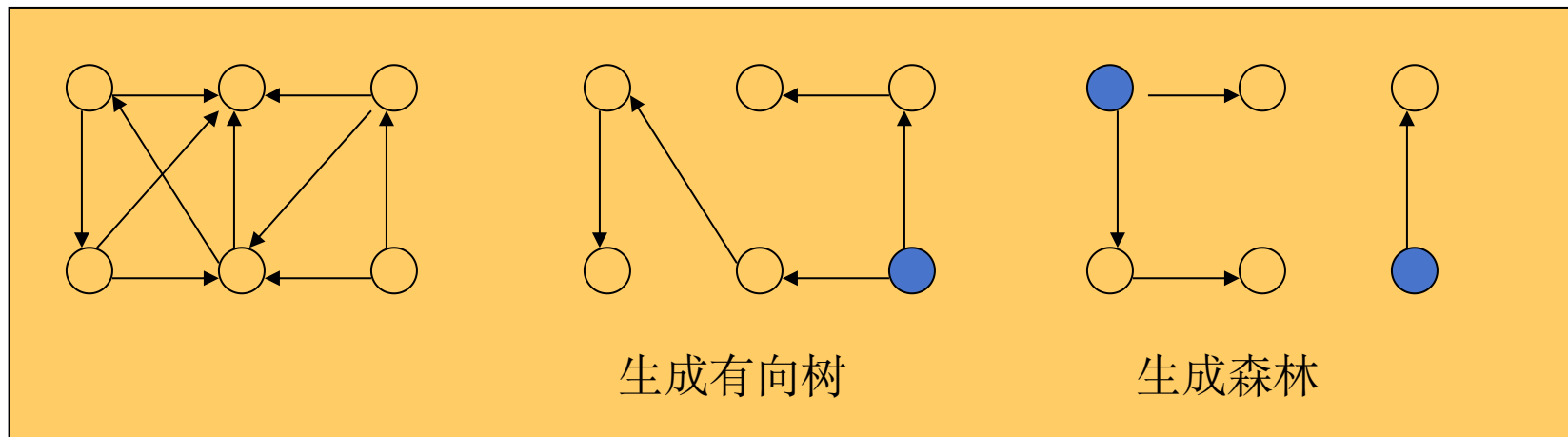
生成树： 包含图中**全部顶点**的**极小连通**子图。



一棵有 n 个顶点的生成树**有且仅有 $n-1$ 条边**，如果一个图有 n 个顶点和小于 $n-1$ 条边，则是非连通图。如果它**多于 $n-1$ 条边**，则**一定有环**。但是，**有 $n-1$ 条边的图不一定是生成树**。

有向树: 如果一个有向图中恰有一个顶点入度为0，其余顶点入度均为1。

生成森林: 有向图中，包含所有顶点的若干棵有向树构成的子图。



7.1.3 图的基本操作

参数中 v 、 w 为顶点编号

(1)图的生成 **CreatGraph**(&G, V,VR)

(2)销毁图 **DestroyGraph**(&G)

(3)顶点定位 **LocateVex**(G,value)

(4)取顶点数据 **Getvex**(G,v)

(5)对顶点赋值 **Putvex**(&G, v, value)

(6)求第一个邻接顶点 **FirstAdjVex**(G,v)

(7)求下一个邻接顶点 **NextAdjVex**(G,v,w)

(8)插入顶点 **InsertVex**(&G,v)

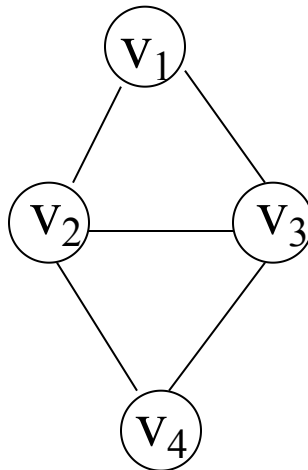
(9)删除顶点 **DeleteVex**(&G,v)

(10)插入边/弧 **InsertArc**(&G,v,w)

(11)删除边/弧 **DeleteArc**(&G,v,w)

(12)图的遍历 **Traverse**(G)

为指定图中数据元素方便起见，逻辑上通常首先将顶点编号



7.2 图的存储结构

7.2.1 数组/邻接矩阵 表示法(Adjacency Matrix) 顺序存储方式

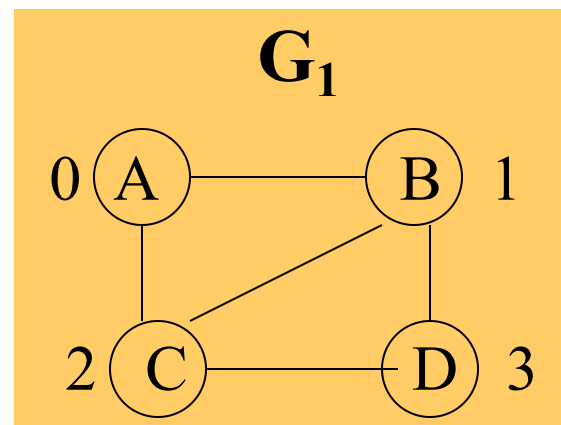
[例1] 无向图

顶点数组 **vexs**

0	A
1	B
2	C
3	D

邻接矩阵(边表) **arcs**

0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0
	0	1	2	3



无向图的邻接矩阵具有对称性

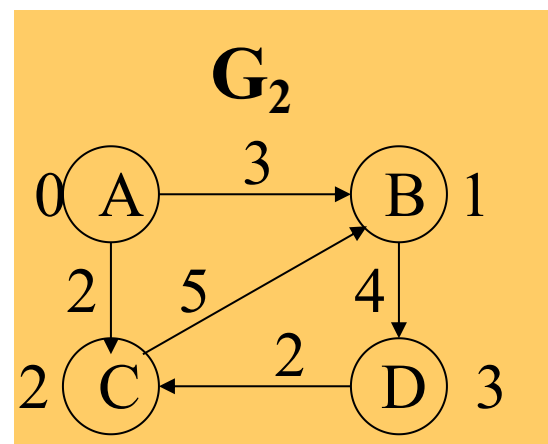
[例2] 有向网

顶点数组 **vexs**

0	A
1	B
2	C
3	D

邻接矩阵(边表) **arcs**

0	0	3	2	∞
1	∞	0	∞	4
2	∞	5	0	∞
3	∞	∞	2	0
	0	1	2	3



[存储结构定义]

```
#define INFINITY      INT_MAX    // 最大值 $\infty$ 

#define MAX_VERTEX_NUM 20      // 最大顶点个数

typedef enum{DG, DN, UDG, UDN} GraphKind;    // 四种图类型

typedef struct ArcCell {
    VRType  adj;           // 顶点关系类型。如带权图，则为权值类型
    InfoType * info;       // 该弧的相关信息指针
ArcCell, AdjMatrix[MAX_VERTEX_NUM][MAX_VERTEX_NUM];

typedef struct {
    VertexType vexs[MAX_VERTEX_NUM];    // 顶点向量
    AdjMatrix arcs;                       // 邻接矩阵
    int  vexnum, arcnum;                   // 图的当前顶点数和弧数
    GraphKind  kind;                       // 图的种类标志
MGraph;
```


[算法示例] 建立无向图的数组型存储结构

步骤:

1) scanf(顶点数 n , 边数 e) $O(1)$

2) 依次读入每个顶点数据, 填入顶点表 $vexs$ $O(n)$

3) 邻接矩阵 $arcs$ 初始化:

全部置 ∞ ; 主对角线置0; $O(n^2)$

4) 建立邻接矩阵: $O(e)$

4.1) 读入一条边的两个顶点编号 i 、 j 和权值 w

4.2) 置 $arcs[i][j]=w$, $arcs[j][i]=w$

若尚有未读入的边, 转4.1)

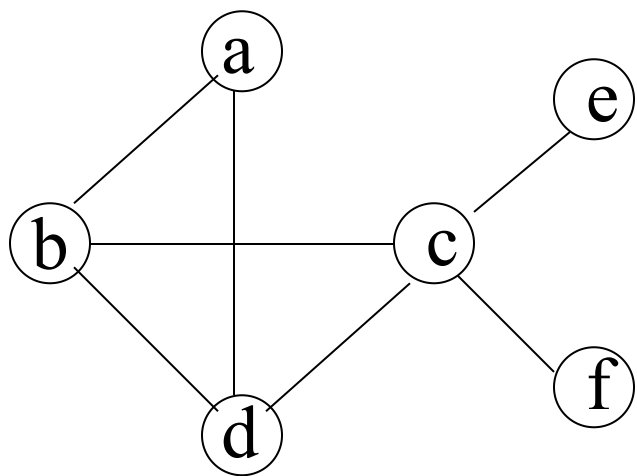
$O(n^2)$

邻接矩阵存储方法的特点

- ①无向图的邻接矩阵一定是一个对称矩阵。
- ②对于无向图，邻接矩阵的第 i 行（或第 i 列）非零元素（或非 ∞ 元素）的个数正好是第 i 个顶点的度 $TD(v_i)$ 。
- ③对于有向图，邻接矩阵的第 i 行（或第 i 列）非零元素（或非 ∞ 元素）的个数正好是第 i 个顶点的出度 $OD(v_i)$ （或入度 $ID(v_i)$ ）。
- ④用邻接矩阵方法存储图，很容易确定图中任意两个顶点之间是否有边相连；但是，要确定图中有多少条边，则必须按行、按列对每个元素进行检测，所花费的时间代价很大。这是用邻接矩阵存储图的局限性。

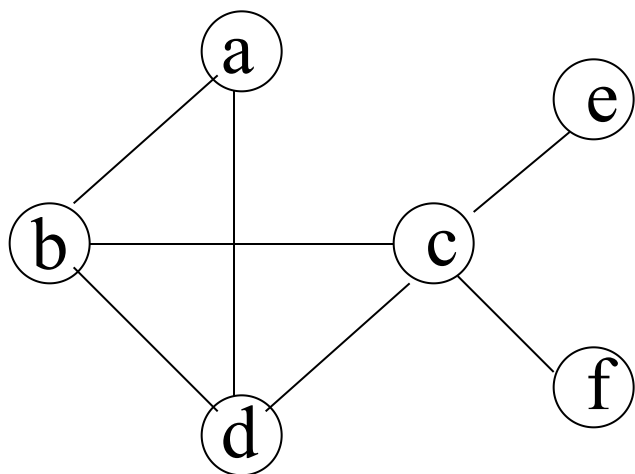
7.2.2 邻接表(Adjacency List) 顺序存储+链式存储

邻接表(Adjacency List)是图的一种顺序存储与链式存储结合的存储方法。

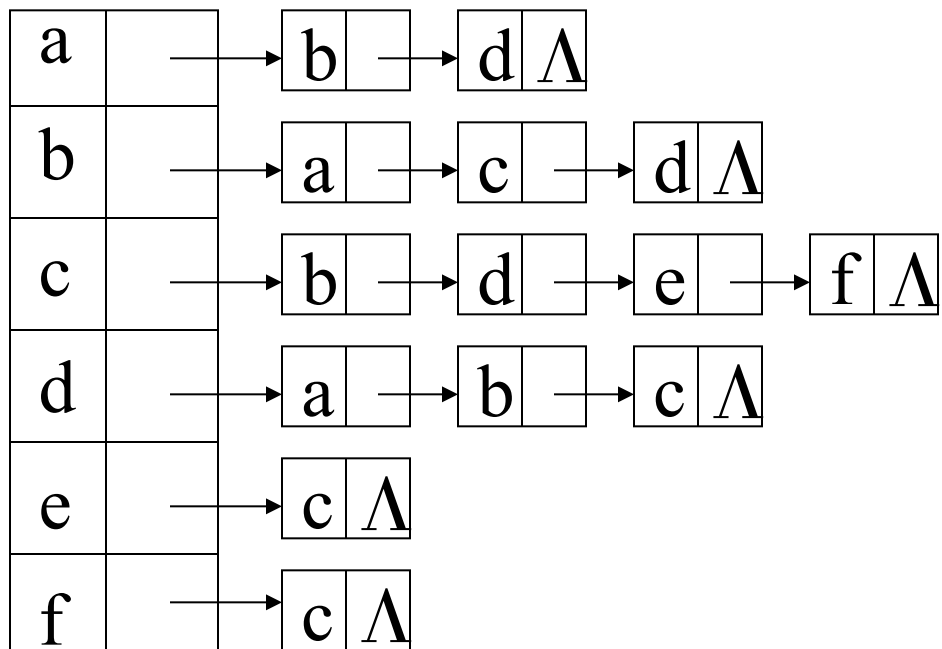


a	b, d
b	a, c, d
c	b, d, e, f
d	a, b, c
e	c
f	c

邻接表

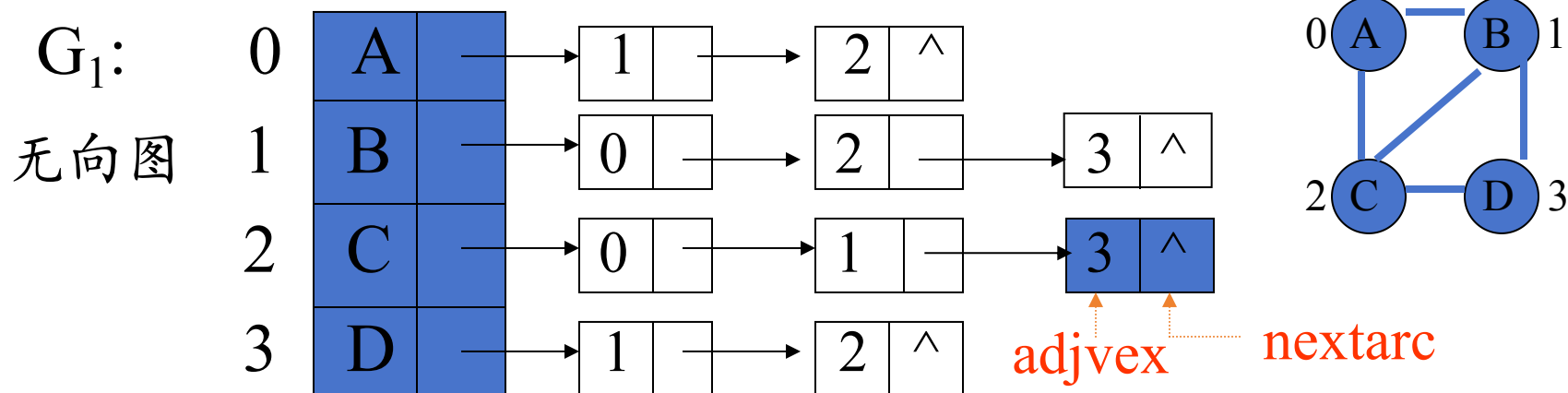


a	b, d
b	a, c, d
c	b, d, e, f
d	a, b, c
e	c
f	c



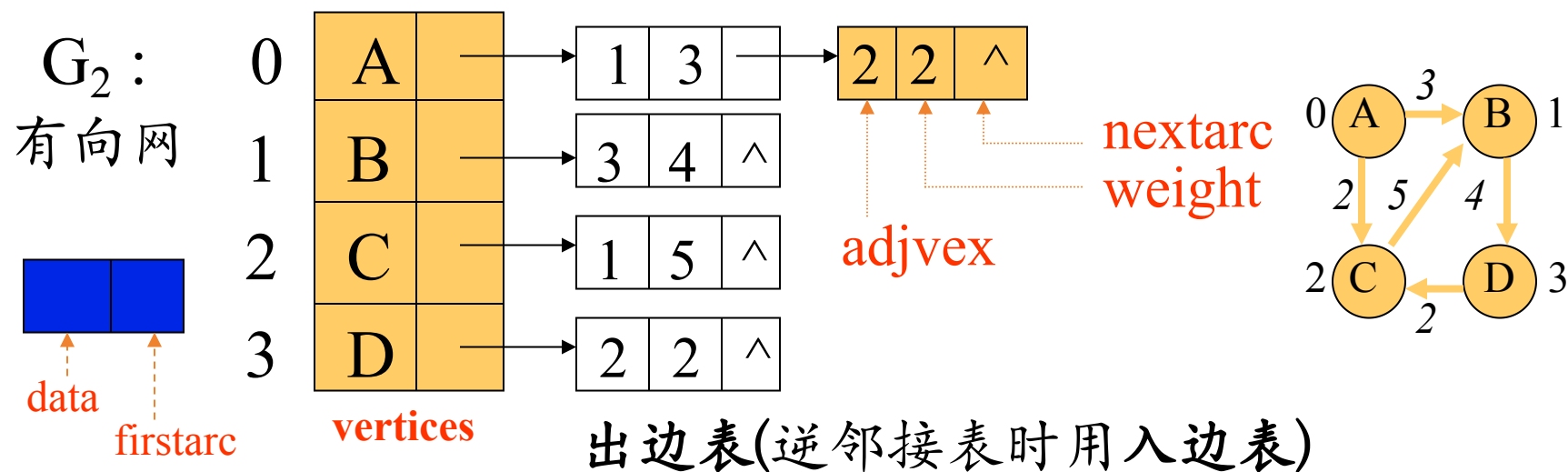
7.2.2 邻接表(Adjacency List) 顺序存储+链式存储

顶点顺序表 邻接顶点的单链表(边表)



7.2.2 邻接表(Adjacency List) 顺序存储+链式存储

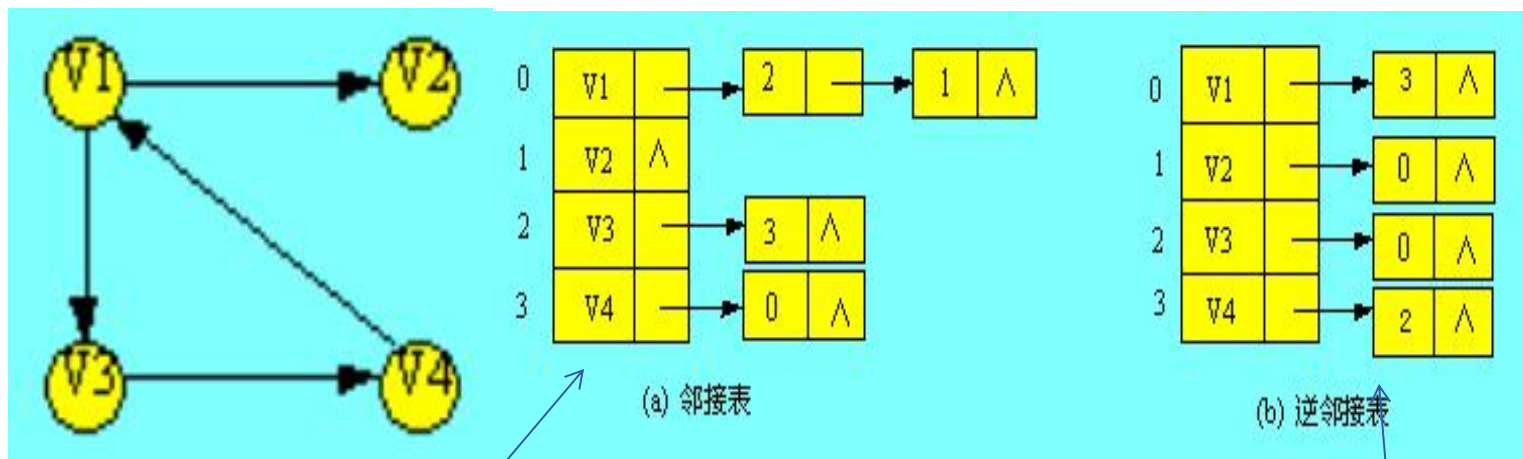
顶点顺序表 邻接顶点的单链表(边表)



有向图的逆邻接表

若邻接表的边结点链表建立的是以 v_i 为头的弧链表，这样的邻接表称为逆邻接表。逆邻接表便于确定顶点的入度或以顶点 v_i 为头的弧。

下图所示为有向图 G_2 的邻接表和逆邻接表。



邻接表：记录了结点都指向谁

逆邻接表：谁指向这个结点

[存储结构定义]

```
#define MAX_VERTEX_NUM 20           // 最大顶点个数
typedef enum {DG, DN, UDG, UDN} GraphKind; // 四种图类型
typedef struct ArcNode {
    int          adjvex;           // 该弧所指向的顶点的位置
    struct ArcNode * nextarc;      // 指向下一条弧的指针
    int          weight;           // 权值
    InfoType     * info;           // 该弧的其它相关信息的指针
} ArcNode;

typedef struct VNode {
    VertexType data;               // 顶点信息
    ArcNode     * firstarc;        // 指向第一条依附该顶点的弧的指针
} VNode, AdjList[MAX_VERTEX_NUM];
```


[存储结构定义]

```
typedef struct {  
    AdjList vertices;           // 顶点向量  
    int vexnum, arcnum;        // 图的当前顶点数和弧数  
    GraphKind kind;            // 图的种类标志  
} ALGraph;
```

[算法示例] 建立无向图的邻接表表示

步骤:

- 1) scanf(顶点数 n , 边数 e) $O(1)$
- 2) 依次读入每个顶点数据,
填入顶点表vertices的相应位置 $O(n)$
- 3) 建立边表: $O(e)$
 - 3.1) 读入一条边的两个顶点编号 i 、 j 和权值 w
 - 3.2) 申请边表结点, $adjvex$ 域= j , $weight$ 域= w ,
将此结点插入顶点 v_i 的边表头部
 - 3.3) 申请边表结点, $adjvex$ 域= i , $weight$ 域= w ,
将此结点插入顶点 v_j 的边表头部若尚有未读入的边, 转3.1)

$O(n+e)$

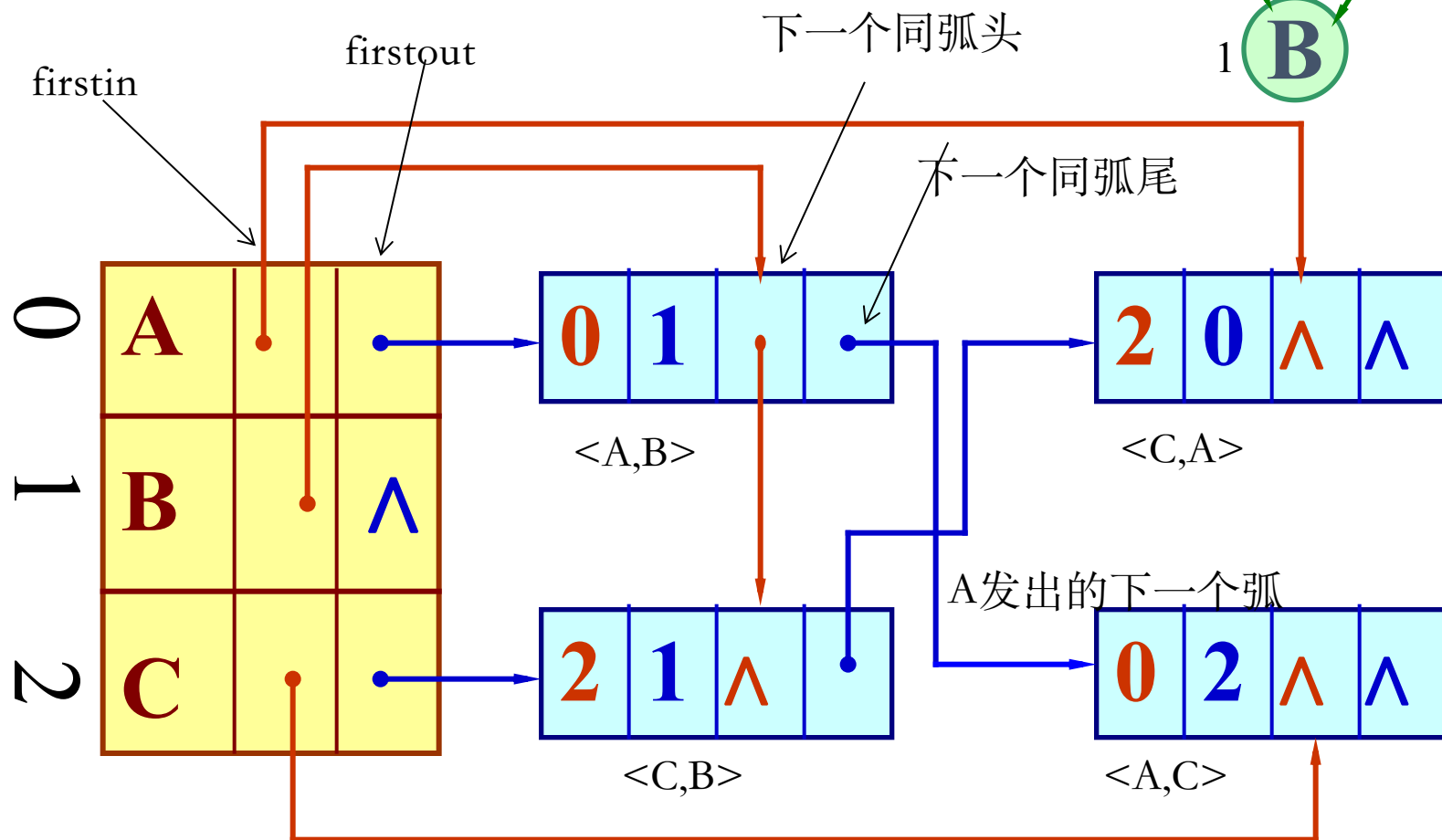
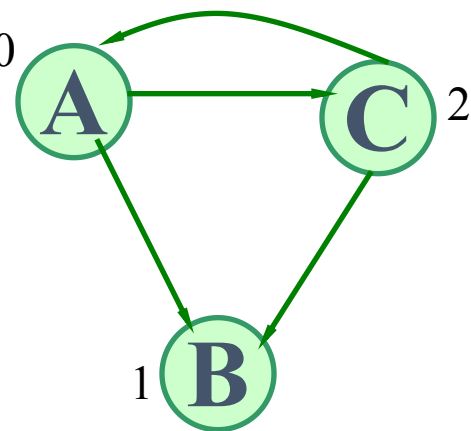
[两种常用存储结构的比较]

邻接矩阵			邻接表		
无向图 有向图			无向图 有向图		
空 间	顶点表结点	n n	顶点表结点	n n	
	邻接矩阵	$n^2/2$ n^2	边表结点	2e e	
适合稠密图			适合稀疏图		
时 间	求顶点的度	扫描矩阵一行 $O(n)$	扫描某个边表	求出度同 最坏 $O(n)$	无向图; 求入度 $O(n+e)$
	判定边是 否存在	$O(1)$	扫描某个边表	最坏 $O(n)$	
	求边的数目	检测矩阵 $O(n^2)$	检测每个边表	$O(n+2e)$	

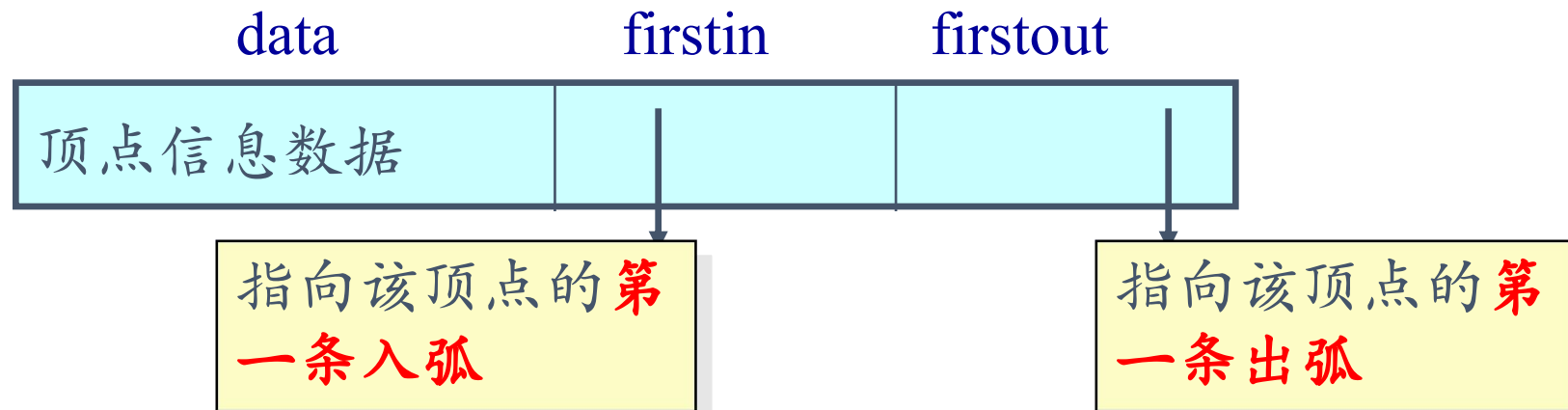
十字链表：适用于**有向图** - 邻接表和逆邻接表的结合。

核心思路：一条链表记录结点指向了谁

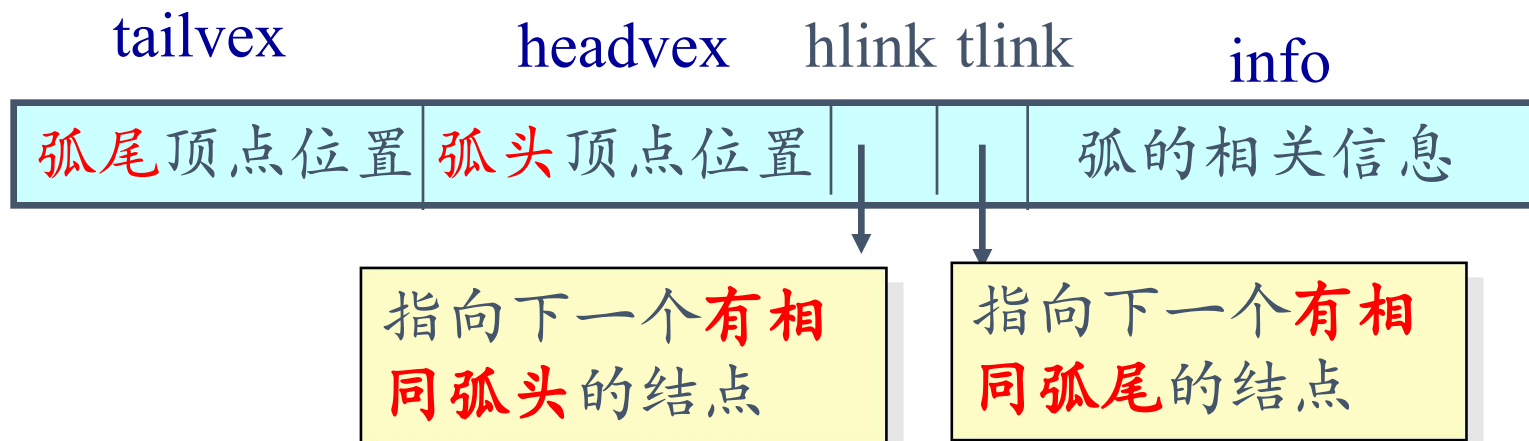
另一条链表记录谁指向了当前结点



顶点的结点结构:

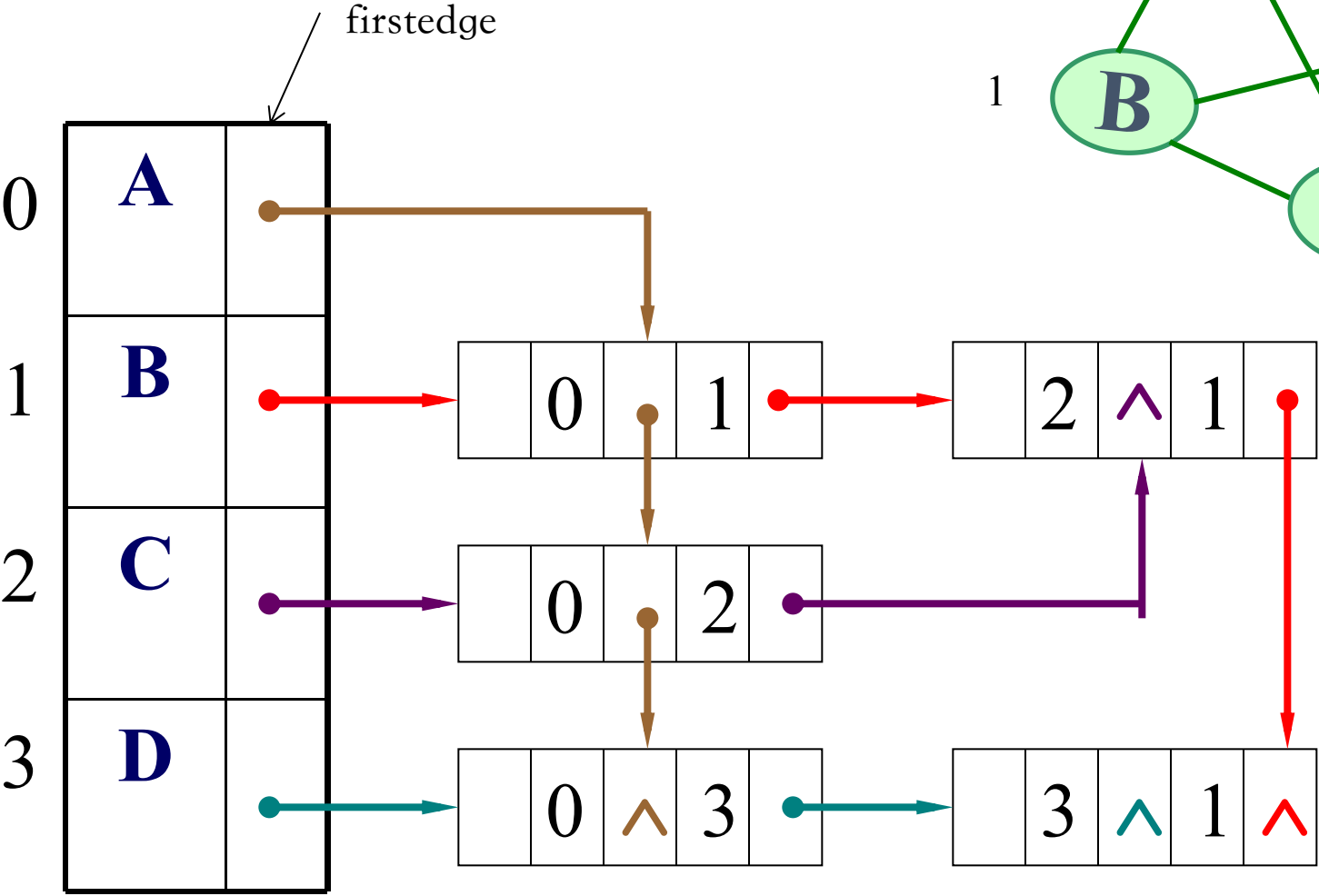


弧的结点结构:



[存储结构定义] {参教材第165页}

邻接多重表 适用对象无向图 - 邻接表的改进，每条边对应一个结点，方便无向图中对边的操作。

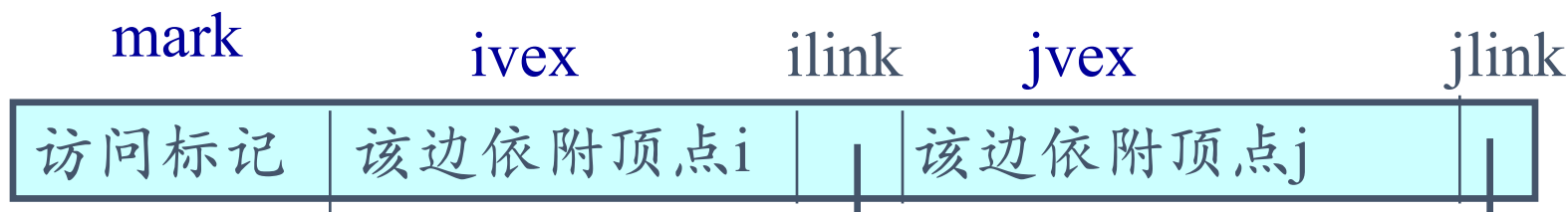


顶点的结点结构:



指向第一条依附该顶点的边

边的结点结构:



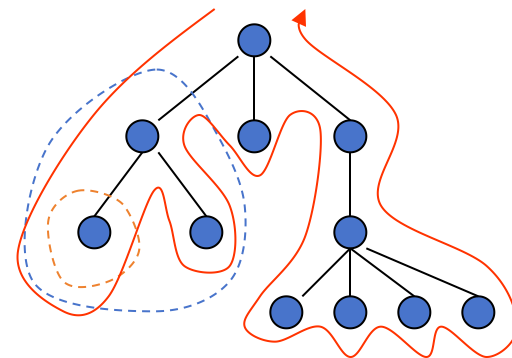
指向下一个有相同边的结点

指向下一个有相同边的结点

[存储结构定义] {参教材第167页}

7.3 图的遍历

- 深度优先遍历 (树的先根遍历的推广)
- 广度优先遍历 (树的按层次遍历的推广)

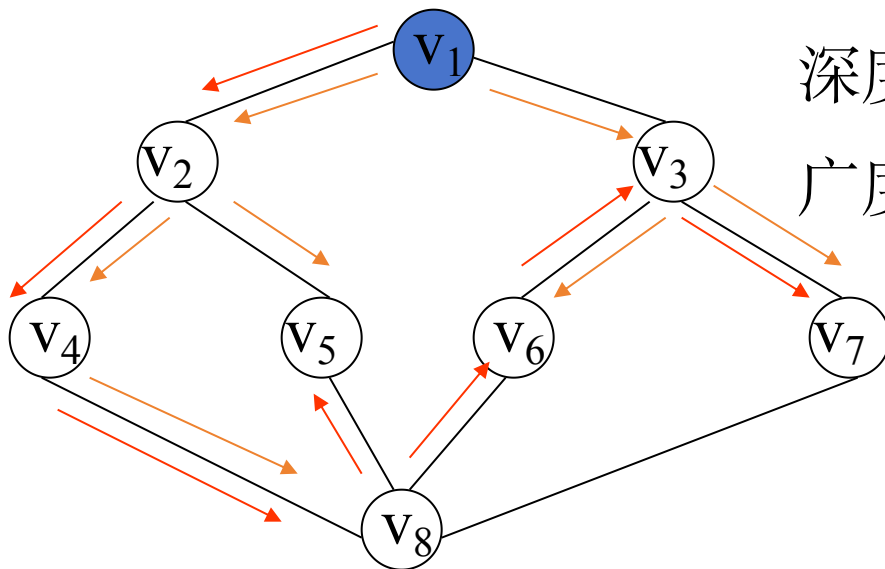


[例]

设从 v_1 出发遍历

深度: $v_1 v_2 v_4 v_8 v_5 v_6 v_3 v_7$

广度: $v_1 v_2 v_3 v_4 v_5 v_6 v_7 v_8$



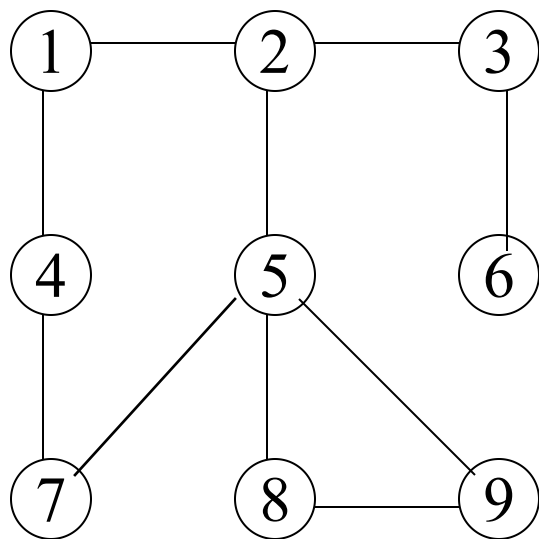
数据结构: 主——图的存储结构

辅——数组 $visited[0..n-1]$

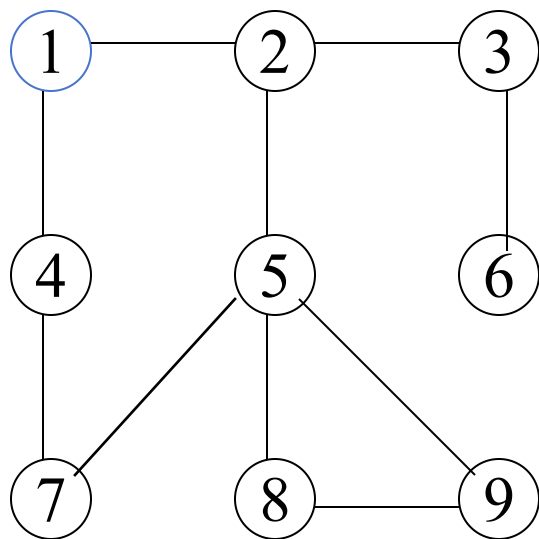
深度优先遍历

- ①首先访问图中某一指定的出发点V,
- ②以V的一个未访问过的邻接点W为新的出发点,
重复①、②
- ③若邻接点都已访问过, 则退回到前一顶点
- ④直到图中所有的顶点都被访问过。

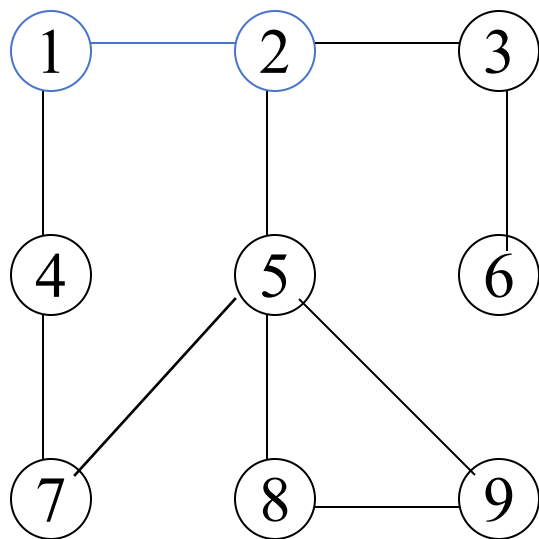
深度优先遍历



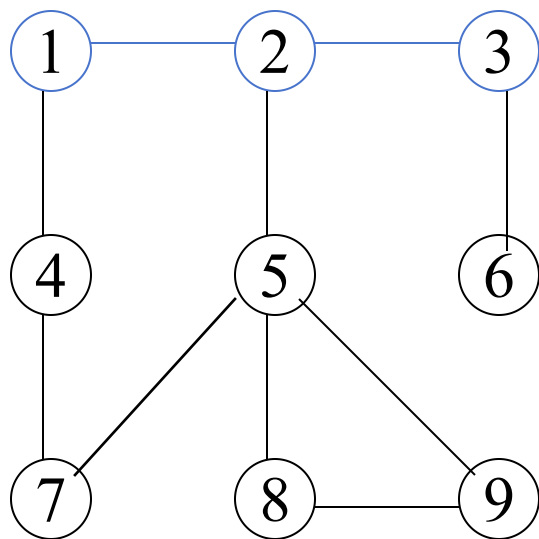
深度优先遍历



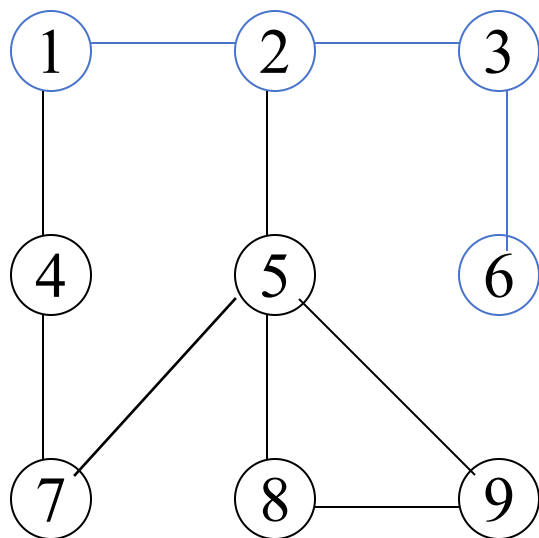
深度优先遍历



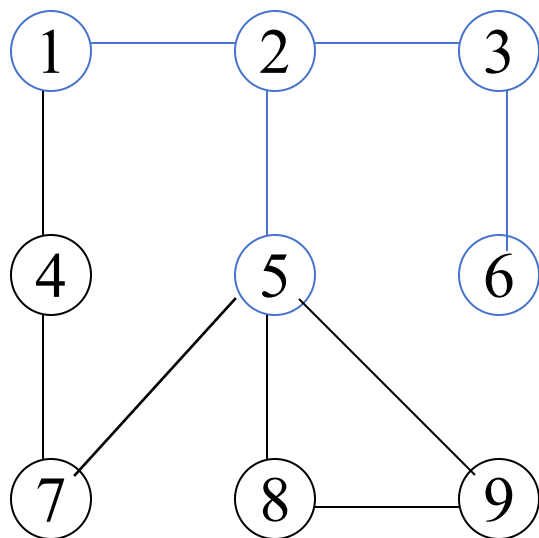
深度优先遍历



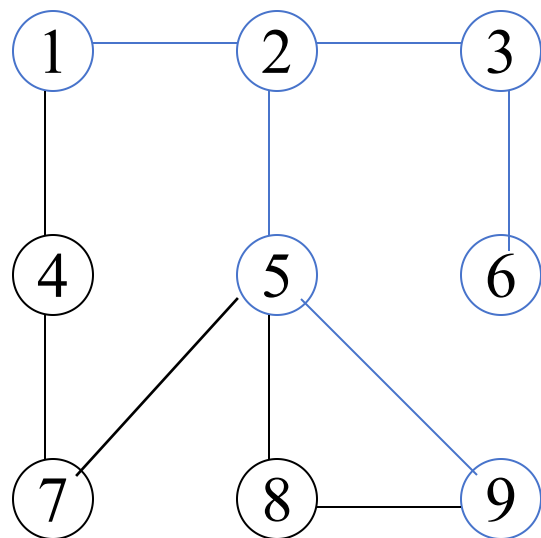
深度优先遍历



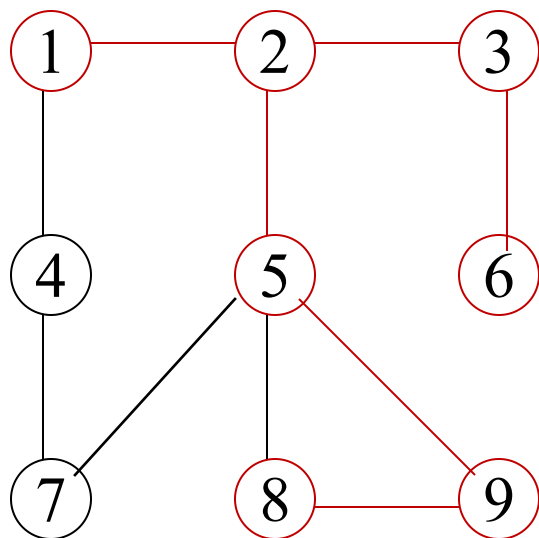
深度优先遍历



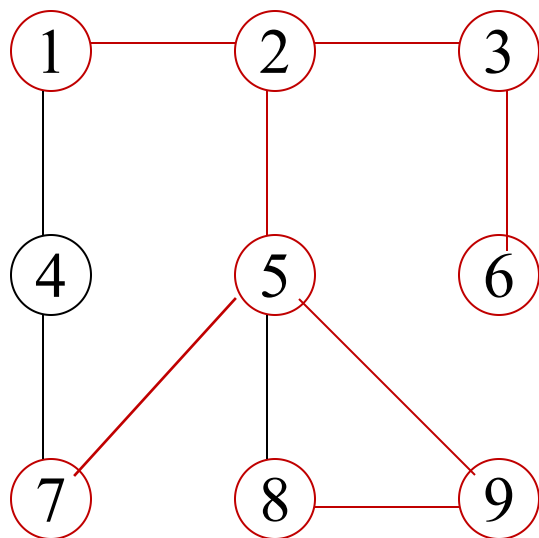
深度优先遍历



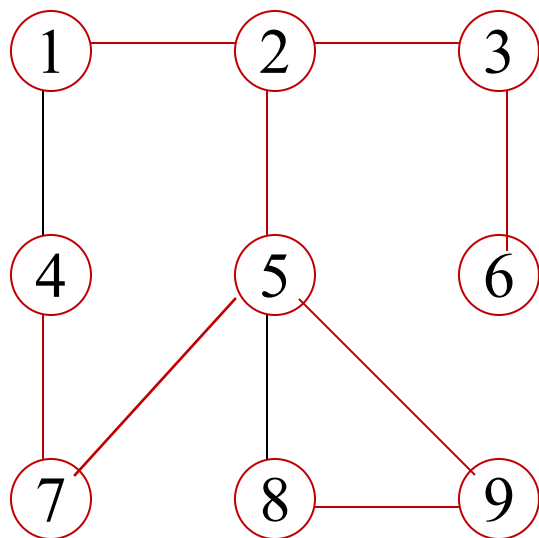
深度优先遍历



深度优先遍历

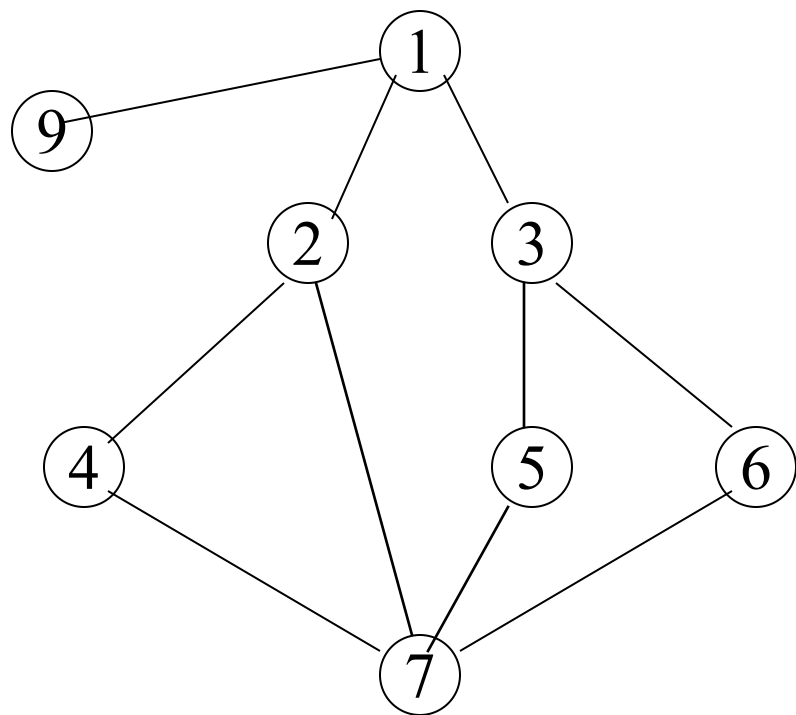


深度优先遍历



1 2 3 6 5 9 8 7 4

深度优先遍历



1 2 4 7 5 3 6 9

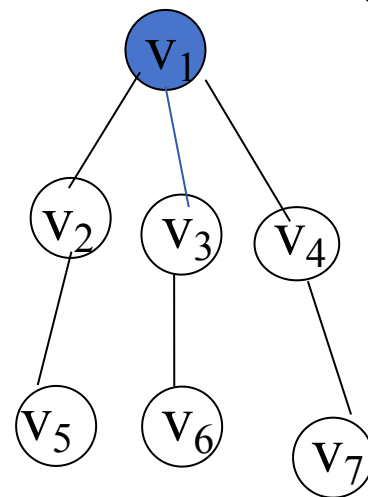
7.3.1 深度优先搜索

递归的算法思想

(1)访问顶点 v ，并记录 v 已被访问

(2)依次从 v 的未访问的邻接点出发，深度优先搜索图 G 。

通过图的特例--树
来加深对代码印象



Boolean **visited**[MAX_VERTEX_NUM]; //辅助访问标志向量

void **DFS**(Graph G, int v)

{ **visit**(v);

visited[v]=TRUE; //访问顶点 v ，并标记

for (w=**FirstAdjVex**(G,v); w>-1; w=**NextAdjVex**(G,v,w))

if (! visited[w]) **DFS**(G,w); //依次从未访问的邻接点出发

} // DFS

树的先序遍历的扩展

```
void DFS(Graph G, int v)
```

```
{ visit(v);
```

```
    visited[v]=TRUE; //访问顶点v, 并标记
```

```
    for ( w=FirstAdjVex(G,v); w>-1; w=NextAdjVex(G,v,w) )
```

```
        if ( ! visited[w] ) DFS(G,w);
```

```
    } // DFS
```

```
void DFS(MGraph G, int v)
```

```
//深度优先遍历 邻接矩阵 表示的图
```

```
{ visit(G.vexs[v]);
```

```
    visited[v]=TRUE;
```

```
    for ( j=0; j<G.vexnum; ++j )
```

```
        if ( G.arcs[v][j]!=0 && ! visited[j] ) DFS(G, j);
```

```
    } // DFS
```

0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0
	0	1	2	3

```
void DFS(Graph G, int v)
```

```
{ visit(v);
```

```
visited[v]=TRUE; //访问顶点v, 并标记
```

```
for ( w=FirstAdjVex(G,v); w>-1; w=NextAdjVex(G,v,w) )
```

```
    if ( ! visited[w] ) DFS(G,w);
```

```
} // DFS
```

```
void DFS(ALGraph G, int v)
```

```
//深度优先遍历 邻接表 表示的图
```

```
{ visit(G.vertices[v].data);
```

```
visited[v]=TRUE;
```

```
p=G.vertices[v].firstarc; //取v边表的头指针
```

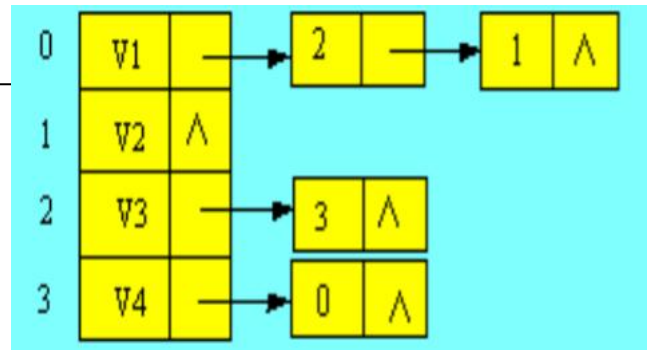
```
while (p) {
```

```
    if ( ! visited[p->adjvex] ) DFS(G, p->adjvex);
```

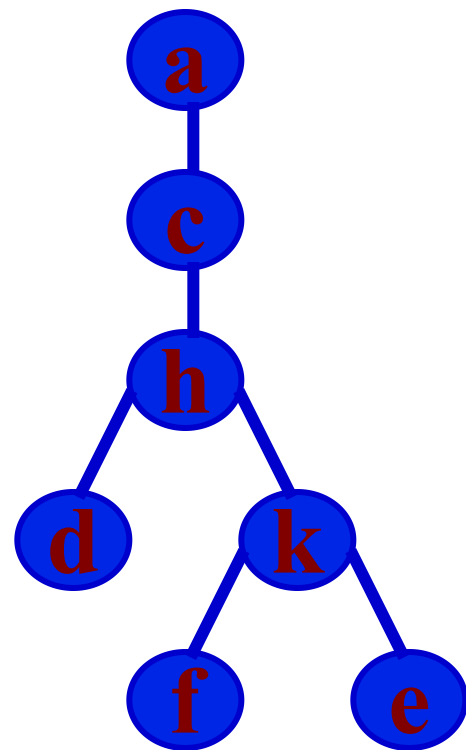
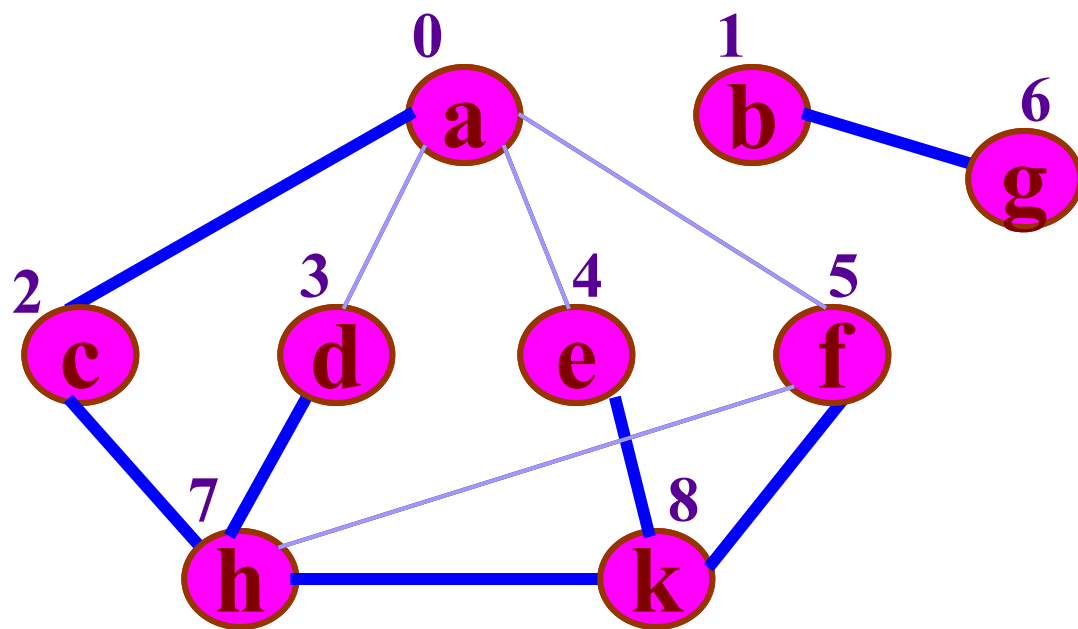
```
    p=p->nextarc;
```

```
}
```

```
} // DFS
```



例



访问标志:

0	1	2	3	4	5	6	7	8
T	T	T	T	T	T	T	T	T

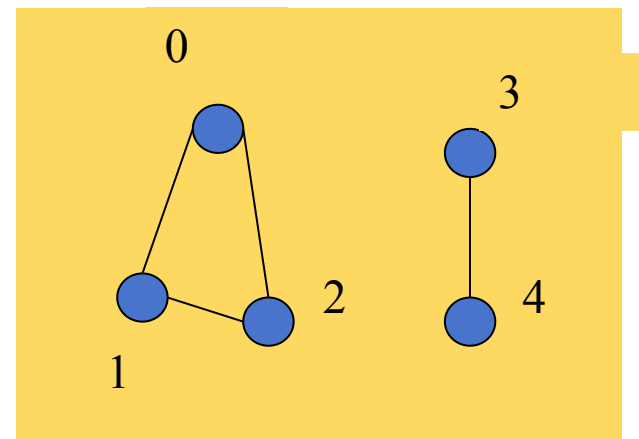
访问次序:

a	c	h	d	k	f	e	b	g
---	---	---	---	---	---	---	---	---

7.3.1 深度优先搜索

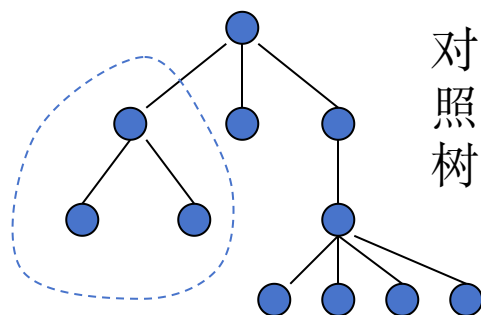
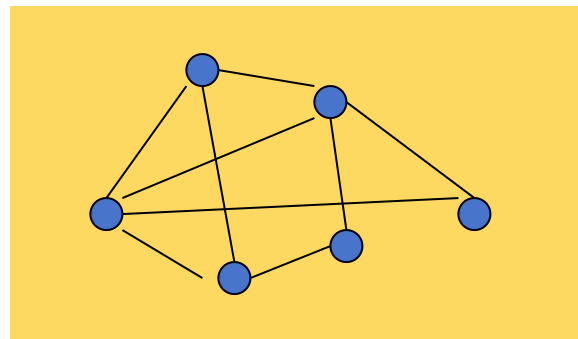
当图分连通时，需要多次启动DFS，每启动一次对应一个连通分量

```
void DFSTraverse(Graph G)
{
    for (v=0; v<G.vexnum; ++v)
        visited[v]=FALSE; //标志向量初始化
    for (v=0; v<G.vexnum; ++v)
        if ( ! visited[v] ) DFS(G, v);
} // DFSTraverse
```



[非递归的算法思想]

- (1)访问一个顶点，并记录它已被访问；
将它的**所有未访问的邻接顶点入栈**；
- (2)如果栈空，则退出；
否则，栈中一顶点出栈；
- (3)如果该顶点已被访问过，则转(2)
否则，转(1)



[算法时间复杂度分析]

主要操作：查找每个顶点的所有邻接点

邻接矩阵 $O(n^2)$

邻接表 $O(n+e)$ $\left\{ \begin{array}{l} \text{无向图结点数 } n+2e \\ \text{有向图结点数 } n+e \end{array} \right.$

7.3.2 广度(宽度)优先遍历

[算法思想]

(1)访问顶点 v ，并记录它已被访问；

顶点 v 入队列；

(2)如果队列空，则退出；

否则，从队中取出一顶点；

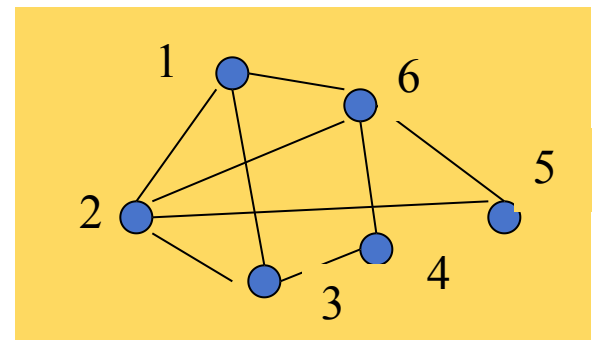
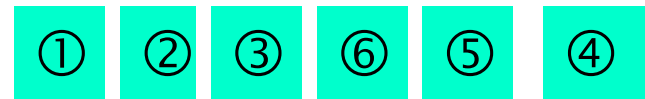
(3)求该顶点的一个邻接点；

如果此邻接点未被访问，

则访问它，并记录它已被访问，将其入队列；

(4)如果该顶点还有下一个邻接点，则转(3)；

否则，转(2)



算法描述

访问-->入队-->出队-->依次找未访问邻接点-->访问

```
void BFSTraverse(Graph G)
```

```
{   for (v=0; v<G.vexnum; ++v)
```

```
    visited[v]=FALSE; //标志向量初始化
```

```
    InitQueue(Q);      //辅助队列初始化
```

```
    for (v=0; v<G.vexnum; ++v)
```

```
        if ( ! visited[v] ) { //从一个未访问的顶点开始启动BFS, 每启动一次对应一个连通分量
```

```
            visit(v); visited[v]=TRUE; EnQueue(Q, v);
```

```
            while (!QueueEmpty(Q)) {
```

```
                DeQueue(Q, u);
```

```
                for ( w=FirstAdjVex(G,u); w>-1; w=NextAdjVex(G,u,w) )
```

```
                    if (!visited[w]) {
```

```
                        visited[w]=TRUE; visit(w); EnQueue(Q,w);
```

```
                    }
```

```
            }
```

```
    }
```

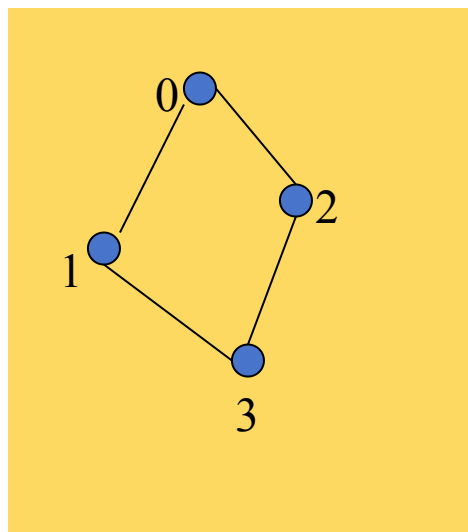
```
} // BFSTraverse
```

7.3.3 图的遍历小结

深度优先遍历算法借助于栈结构实现；广度优先遍历算法借助于队列结构实现

图的遍历序列与算法和存储方式有关

对于非连通图，通过遍历可求得各连通分量



	0	1	2	3
0	<u>0</u>	<u>1</u>	1	0
1	1	0	0	<u>1</u>
2	1	0	0	1
3	0	1	<u>1</u>	0

DFS 0 1 3 2

0		→	2, 1
1		→	3, 0
2		→	3, 0
3		→	2, 1

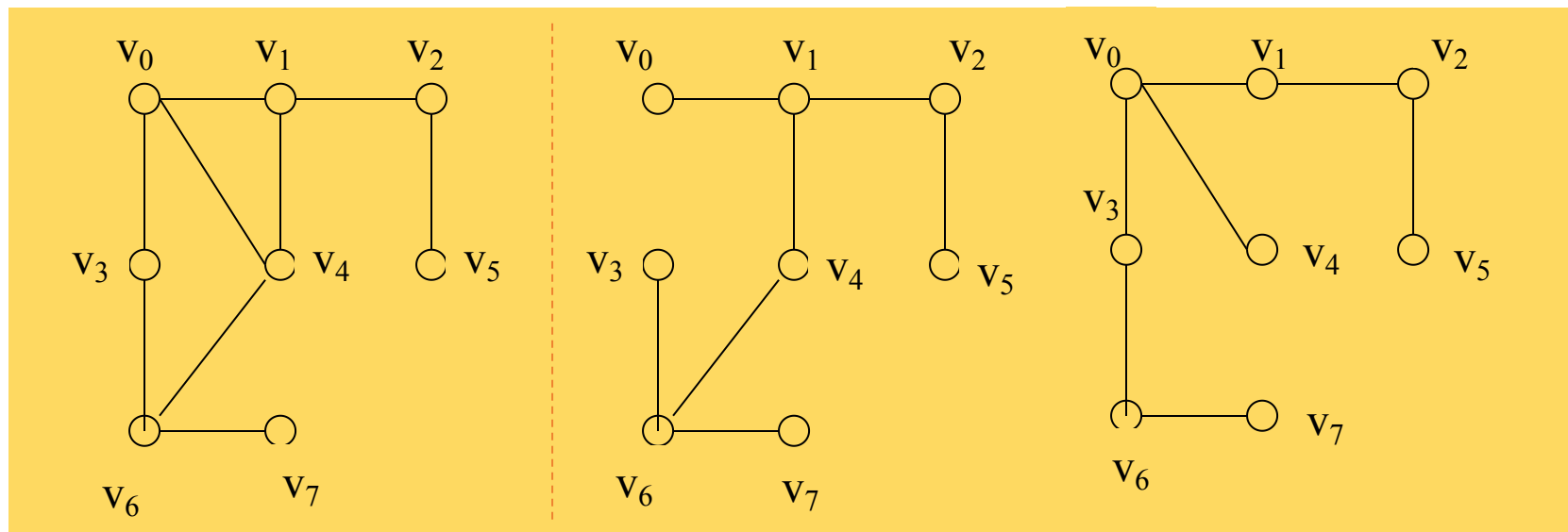
0 2 3 1

7.4 图的连通性问题

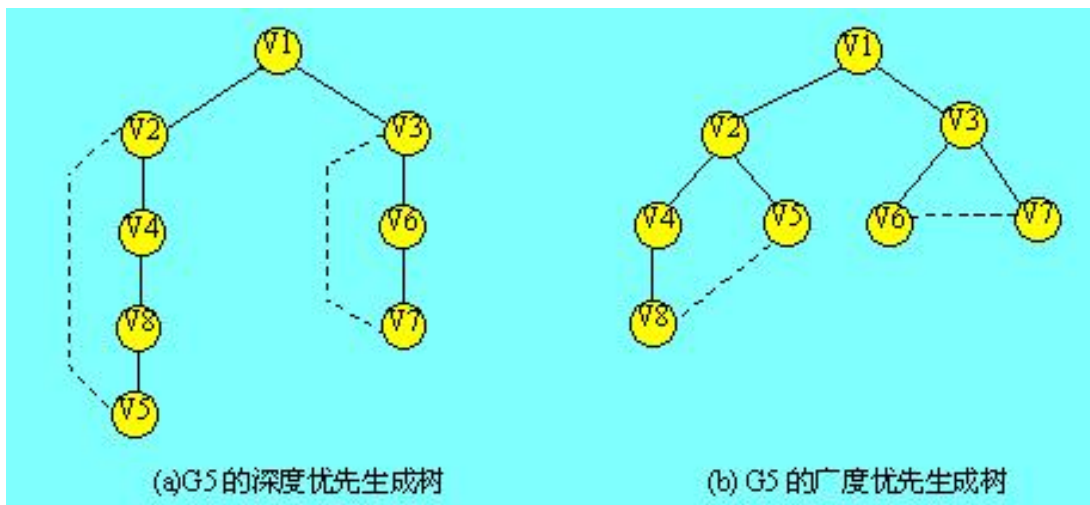
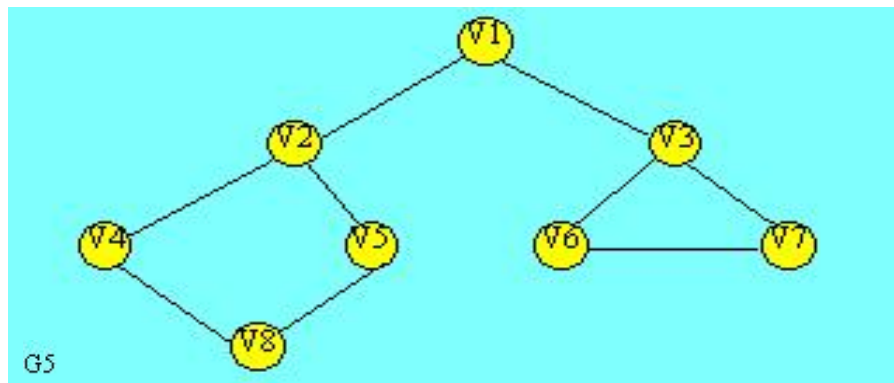
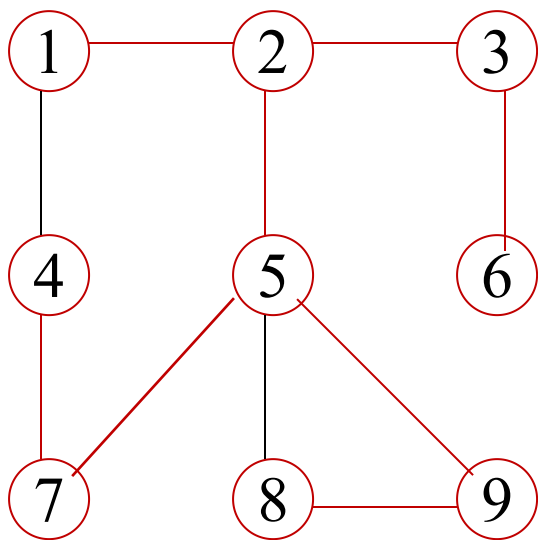
7.4.1 无向图的连通分量和生成树

(1) 连通图的生成树

生成树是图 G 的一个子图，该子图是**包含 G 的所有顶点的树**(或者是自由树—未确定根结点)。



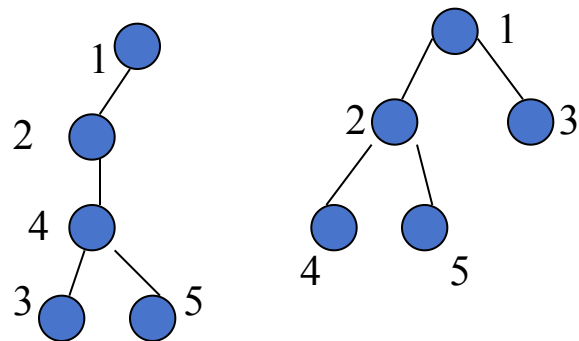
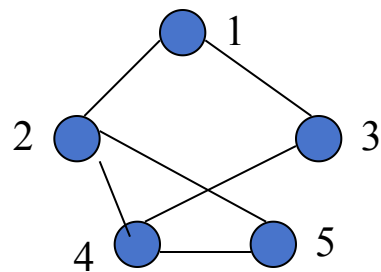
设 $G=(V_n, E_n)$ ，则从图中任一顶点进行遍历图时，必定将 E_n 分成两个集合 $T(G)$ 和 $B(G)$ ，其中 $T(G)$ 是遍历时经过的边的集合， $B(G)$ 是剩余边的集合。显然 $T(G)$ 和所有顶点一起构成连通图 G 的**极小连通子图**，也就是连通图的一棵生成树。



- 若图G有n个顶点，则其生成树必具有n-1条边。
- 生成树是包含图中所有顶点的极小连通子图。
- 生成树可以通过对图的深度/广度优先遍历而得到，称之为深度/广度优先生成树。

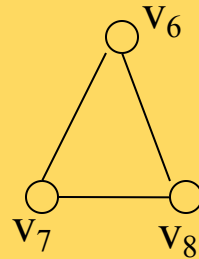
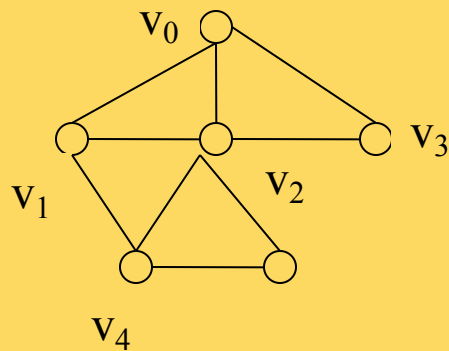
在算法中，访问一个结点时 v_j ，同时记录它的父结点 v_i ， (v_i, v_j) 是生成树的一条边。

- 一般情况，BFS(广度优先搜索)生成树的树高小于DFS(深度优先搜索)生成树的高度。
- 一个图的生成树是不唯一的
 - 从不同的顶点出发
 - 采用不同的存储结构和存储顺序

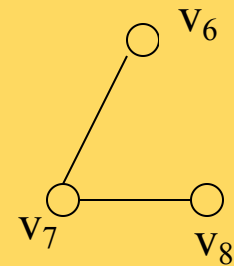
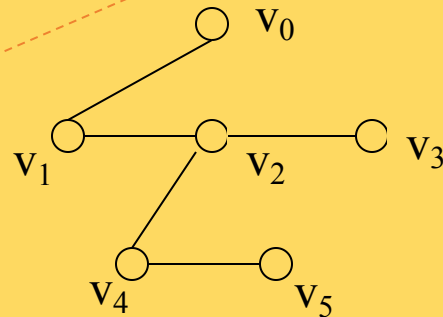


(2) 非连通图的生成森林

- 一个连通分量及其遍历时走过的边构成一棵生成树。
- 非连通图的各连通分量的生成树组成**生成森林**。



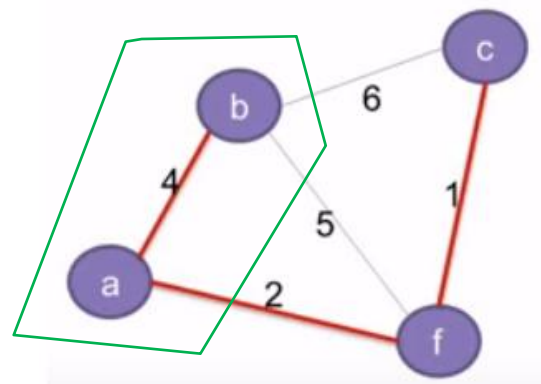
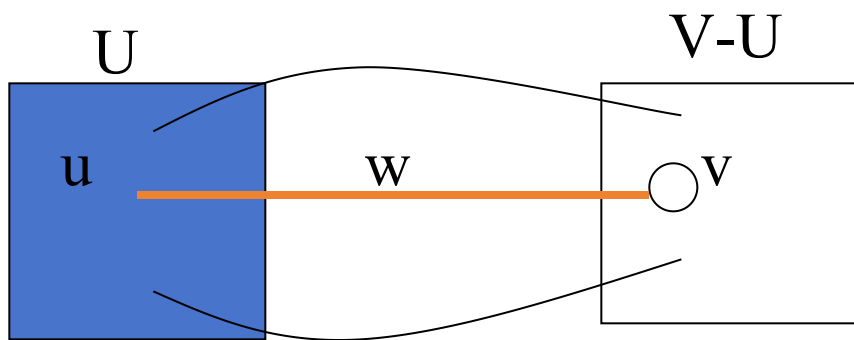
v_5



7.4.2 最小生成树

最小生成树 由一个网络生成的各边的**权数总和最小**的生成树，记为**MST**(Minimum Cost Spanning Tree)。

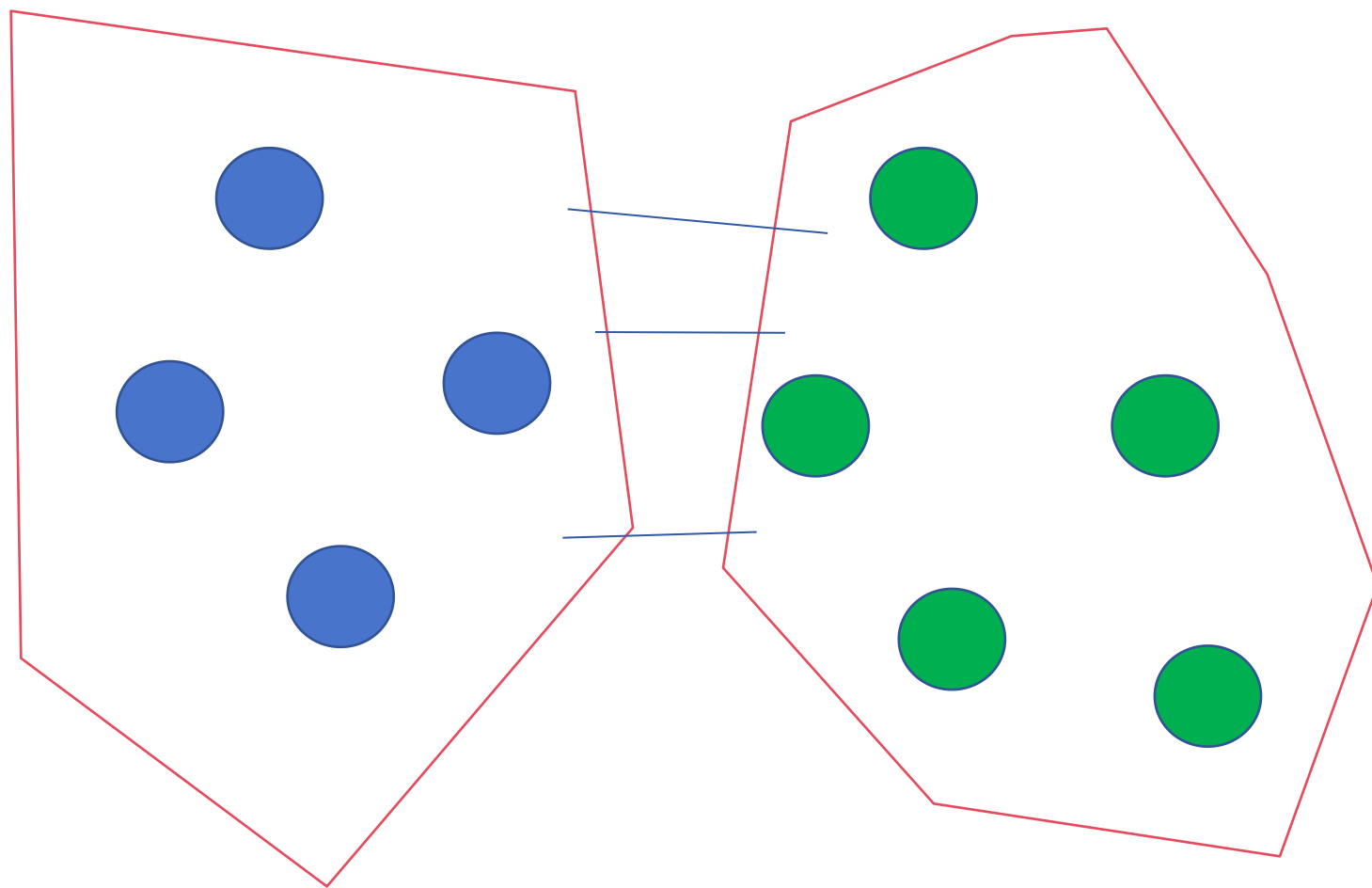
MST性质 设 $N=(V, \{E\})$ 是一个连通的网络， U 是 V 的真子集，若边 $(u, v)[u \in U, v \in V-U]$ 是 E 中所有一个端点在 U 内，一个端点不在 U 内的边中权值最小的一条边(轻边)，则**一定存在 G 的一棵生成树包括此边**。



例如：U包含顶点a, b

V-U包含顶点c, f

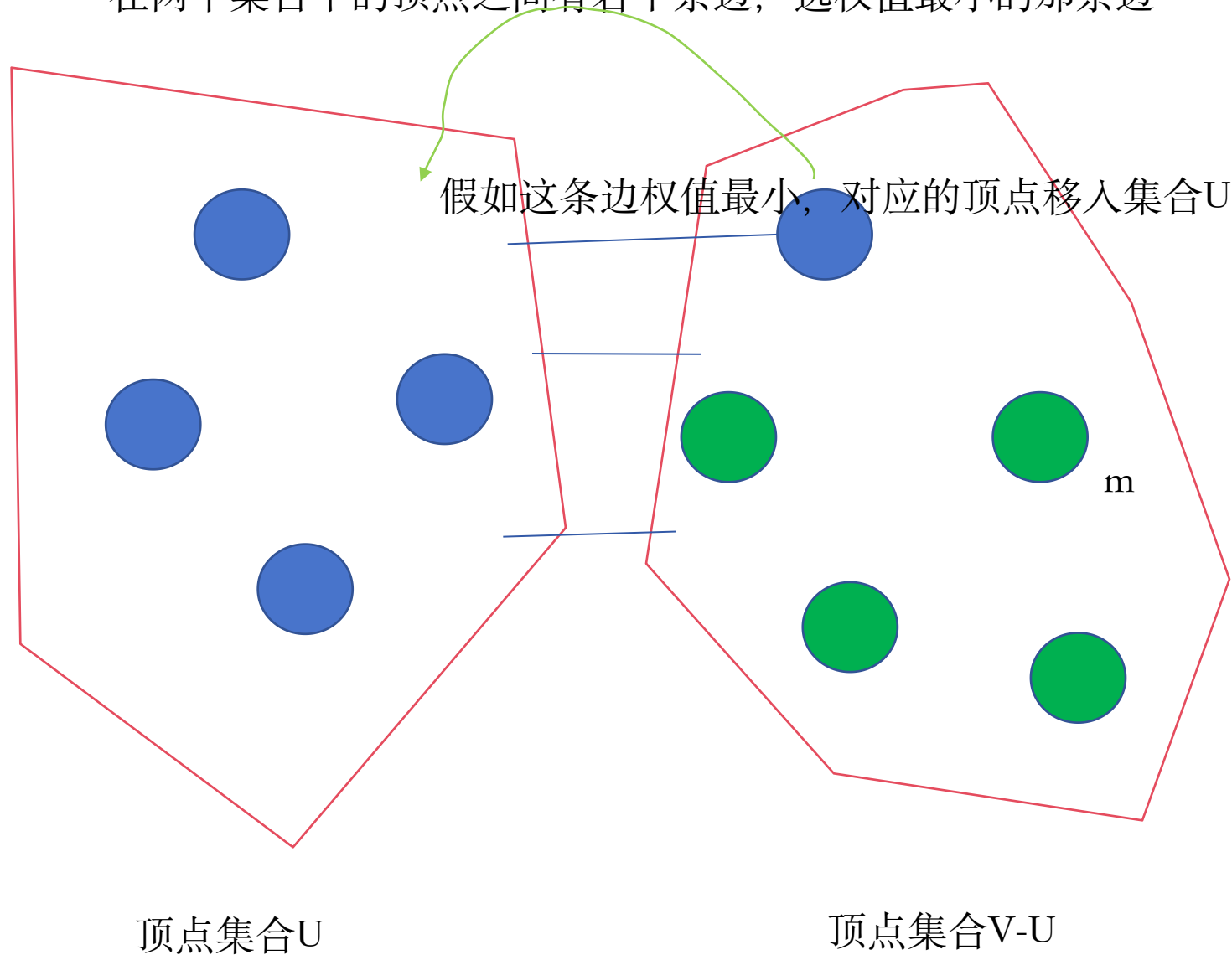
在两个集合中的顶点之间有若干条边，选权值最小的那条边



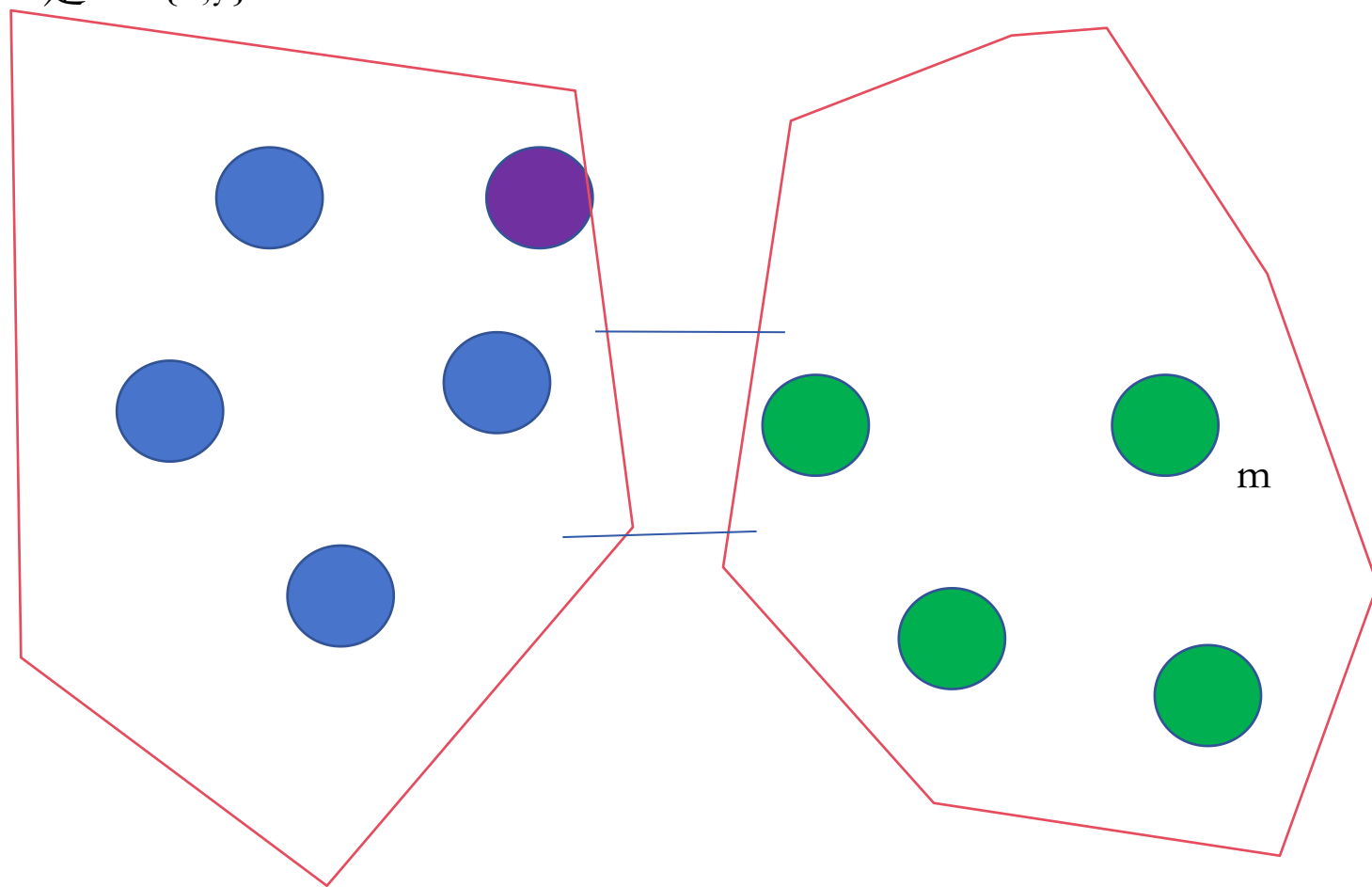
顶点集合U

顶点集合V-U

在两个集合中的顶点之间有若干条边，选权值最小的那条边



假如在上一步时顶点 m 到集合 U 中顶点的最小cost是 x ，那么只需要用顶点 m 和紫色顶点之间的cost（假设是 y ）和 x 比较一下，就能得到顶点 m 现在到集合 U 中顶点的最小cost是 $\min\{x,y\}$



顶点集合U

顶点集合V-U

• Prim算法可用下述过程描述，其中用 W_{uv} 表示顶点 u 与顶点 v 边上的权值。

(1) $U = \{u_1\}, T = \{\}$;

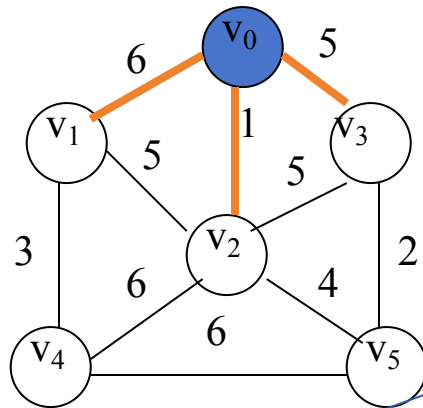
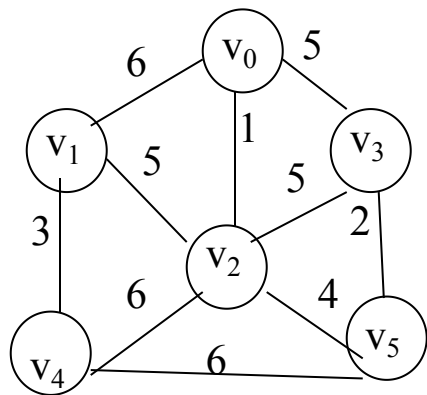
(2) while ($U \neq V$)do

$(u, v) = \min\{W_{uv} \mid u \in U, v \in V - U\}$ // (u, v) 是最轻边

$T = T + \{(u, v)\}$ // 将 (u, v) 加入边集合

$U = U + \{v\}$ // 将 v 加入顶点集合

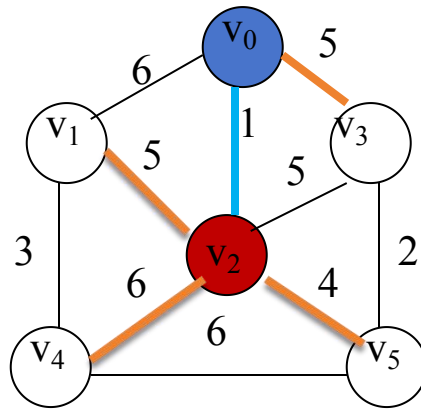
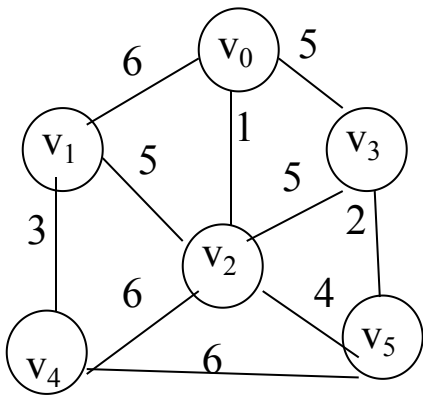
(3) 结束。



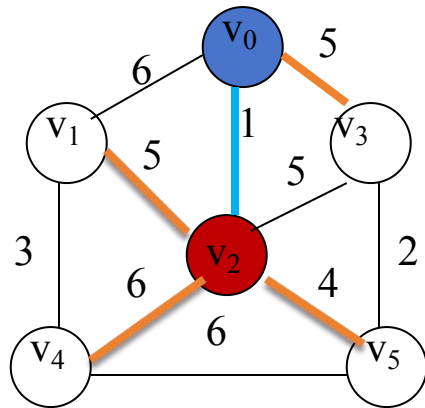
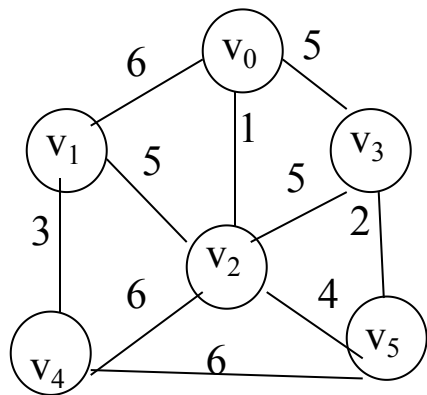
各个顶点和集合U
中的顶点连通的最
小cost

将本轮最小
cost的顶点加入
U

closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{}	2
lowcost	6	1	5					
adjvex								
lowcost								
adjvex								
lowcost								
adjvex								
lowcost								



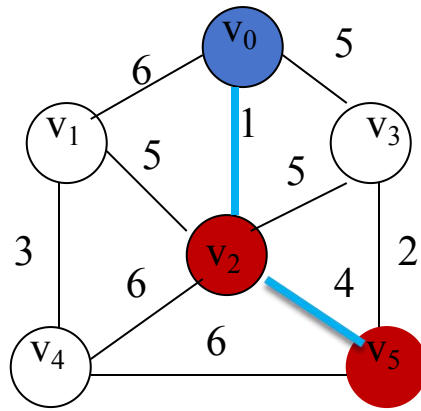
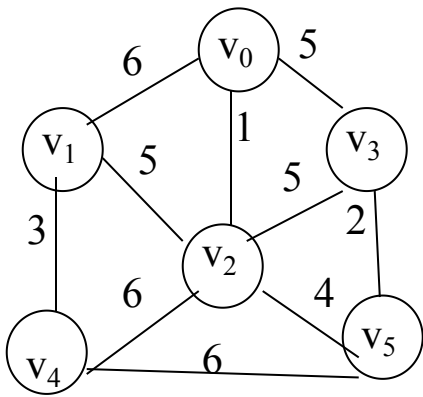
closededge i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex						{v0, v2}	{v1, v3, v4, v5}	
lowcost								
adjvex								
lowcost								
adjvex								
lowcost								



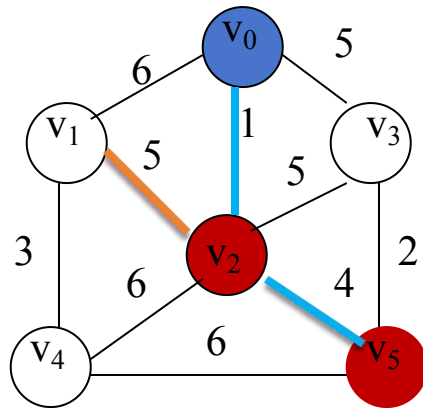
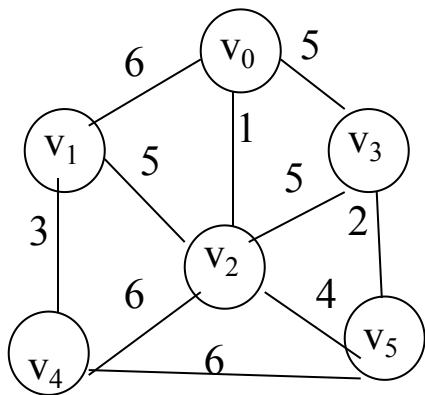
各个顶点和集合U
中的顶点连通的最
小cost

将本轮最小
cost的顶点加入
U

closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	5
lowcost	5	0	5	6	4			
adjvex								
lowcost								
adjvex								
lowcost								
adjvex								
lowcost								



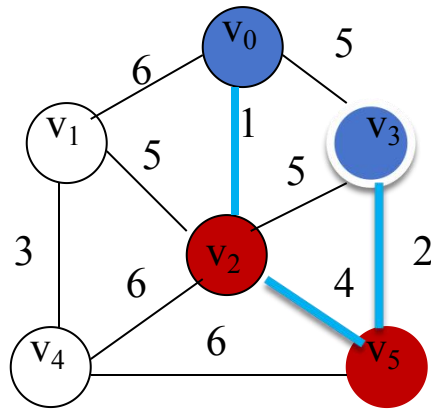
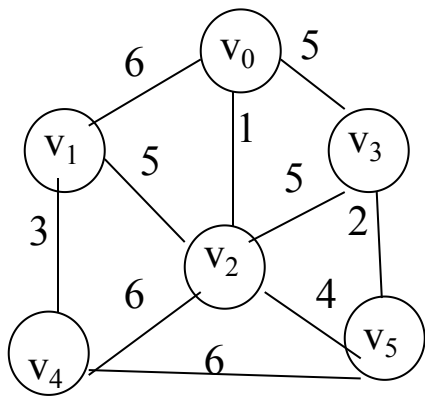
closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	
lowcost	5	0	5	6	4			5
adjvex						{v0, v2, v5}	{v1, v3, v4}	
lowcost								
adjvex								
lowcost								



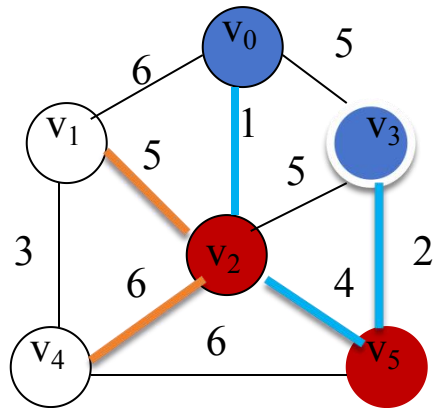
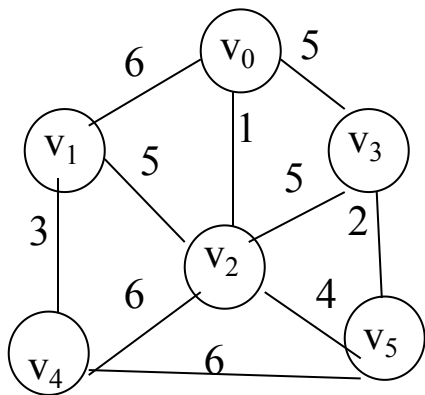
各个顶点和集合U
中的顶点连通的最
小cost

将本轮最小
cost的顶点加入
U

closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	5
lowcost	5	0	5	6	4			
adjvex	v2		v5	v2		{v0, v2, v5}	{v1, v3, v4}	3
lowcost	5	0	2	6	0			
adjvex								
lowcost								
adjvex								
lowcost								



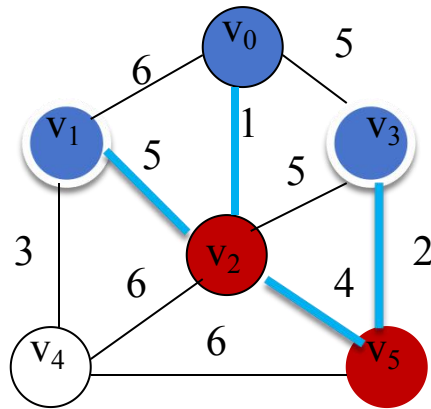
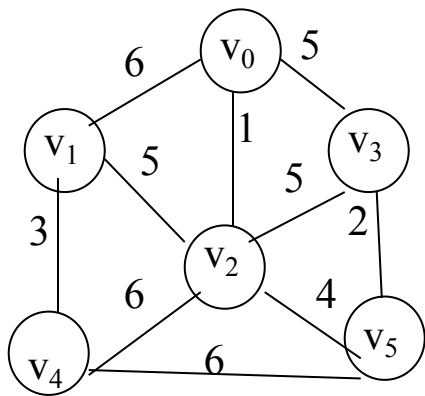
closededge i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	
lowcost	5	0	5	6	4			5
adjvex	v2		v5	v2		{v0, v2, v5}	{v1, v3, v4}	3
lowcost	5	0	2	6	0			
adjvex						{v0, v2, v5, v3}	{v1, v4}	
lowcost								
adjvex								
lowcost								



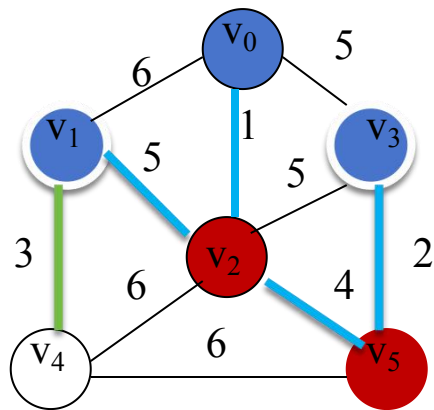
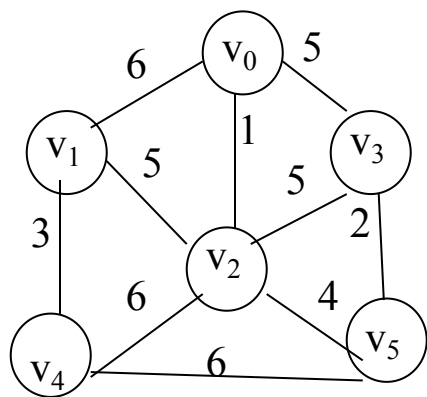
各个顶点和集合U
中的顶点连通的最
小cost

将本轮最小
cost的顶点加入
U

closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	5
lowcost	5	0	5	6	4			
adjvex	v2		v5	v2		{v0, v2, v5}	{v1, v3, v4}	3
lowcost	5	0	2	6	0			
adjvex	v2			v2		{v0, v2, v5, v3}	{v1, v4}	1
lowcost	5	0	0	6	0			
adjvex								
lowcost								



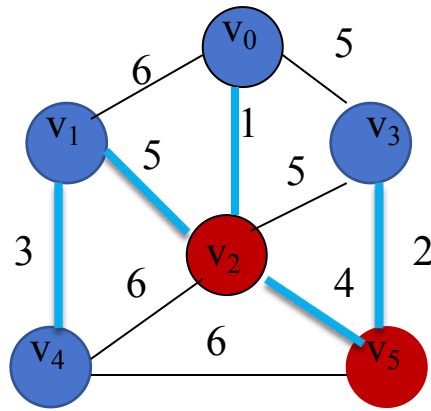
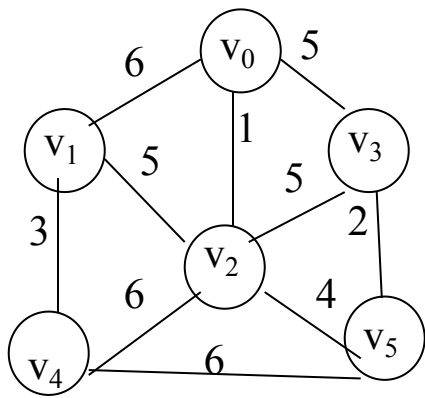
closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	5
lowcost	5	0	5	6	4			
adjvex	v2		v5	v2		{v0, v2, v5}	{v1, v3, v4}	3
lowcost	5	0	2	6	0			
adjvex	v2			v2		{v0, v2, v5, v3}	{v1, v4}	1
lowcost	5	0	0	6	0			
adjvex						{v0, v2, v5, v3, v1}	{v4}	
lowcost								



各个顶点和集合U
中的顶点连通的最
小cost

将本轮最小
cost的顶点加入
U

closededge \ i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	5
lowcost	5	0	5	6	4			
adjvex	v2		v5	v2		{v0, v2, v5}	{v1, v3, v4}	3
lowcost	5	0	2	6	0			
adjvex	v2			v2		{v0, v2, v5, v3}	{v1, v4}	1
lowcost	5	0	0	6	0			
adjvex				v1		{v0, v2, v5, v3, v1}	{v4}	4
lowcost	0	0	0	3	0			



closededge i	1	2	3	4	5	U	V-U	k
adjvex	v0	v0	v0			{v0}	{v1, v2, v3, v4, v5}	2
lowcost	6	1	5					
adjvex	v2		v0	v2	v2	{v0, v2}	{v1, v3, v4, v5}	
lowcost	5	0	5	6	4			5
adjvex	v2		v5	v2		{v0, v2, v5}	{v1, v3, v4}	3
lowcost	5	0	2	6	0			
adjvex	v2			v2		{v0, v2, v5, v3}	{v1, v4}	1
lowcost	5	0	0	6	0			
adjvex				v1		{v0, v2, v5, v3, v1}	{v4}	4
lowcost	0	0	0	3	0			
	0	0	0	0	0	{v0, v2, v5, v3, v1, v4}	{}	

[存储结构]

- 主：带权邻接矩阵 $G.arcs[n][n]$ —记录图
- 辅：数组 $closedge[i](i=0..n-1)$ —记录候选边

- **lowcost**

当前关联 v_i 的轻边权值

- **adjvex**

该边依附的U集合中的顶点下标

示例初值:

lowcost

adjvex

0	1	2	3	4	5
0	6	1	5	∞	∞
0	0	0	0	0	0



k=2

0 1 2 3 4 5

0	1	2	3	4	5
0	5	0	5	6	4
0	2	0	0	2	2

k=5

	0	1	2	3	4	5
0	0	6	1	5	∞	∞
1	6	0	5	∞	3	∞
2	1	5	0	5	6	4
3	5	∞	5	0	∞	2
4	∞	3	6	∞	0	6
5	∞	∞	4	2	6	0

顶点2加入了集合U，然后看看顶点2的加入能不能缩短集合V-U中顶点到U的cost

[算法步骤]

1)初始化closedge[j](j=0..n-1)

$O(n)$

2)重复n-1次以下操作:

2.1)在closedge[j](j=0..n-1)中选择最小且非0的lowcost, 记录其j 值
(设为k)和相应的adjvex;

$O(n^2)$

2.2)输出该边(adjvex,k);

$O(n)$

2.3)顶点k并入U集: closedge[k].lowcost=0;

$O(n)$

2.4)调整候选边集closedge[j](j=0..n-1):

$O(n^2)$

若 $G.arcs[k][j] < closedge[j].lowcost$,

则更改closedge[j]: adjvex=k, lowcost= $G.arcs[k][j]$

$O(n^2)$

[算法描述]

```
struct {  
    int      adjvex;  
    VRType   lowcost;  
}closedge[MAX_VERTEX_NUM];
```

```
void Prim(MGraph G, VertexType u )  
{ k=LocateVex(G,u);  
  for (j=0; j<G.vexnum; ++j)    //初始化辅助数组  
    closedge[j]={k, G.arcs[k][j].adj};  
  for (i=1; i<G.vexnum; ++i) { //进行G.vexnum-1次  
    k=minium(closedge);        //选择顶点  
    printf(closedge[k].adjvex, k);  
    closedge[k].lowcost=0;      //顶点k并入U集  
    for (j=0; j<G.vexnum; ++j) //调整候选边集  
      if (G.arcs[k][j].adj<closedge[j].lowcost)  
        closedge[j]={k, G.arcs[k][j].adj}; //cost可以减小  
  } //Prim
```

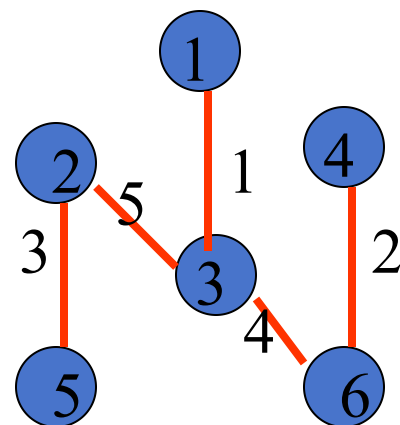
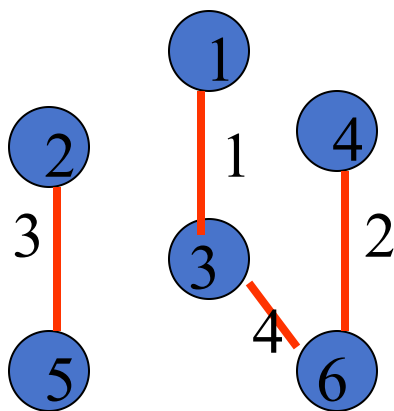
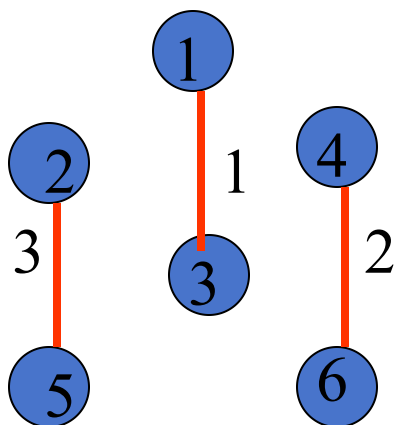
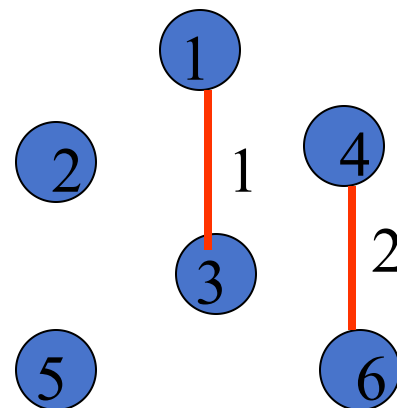
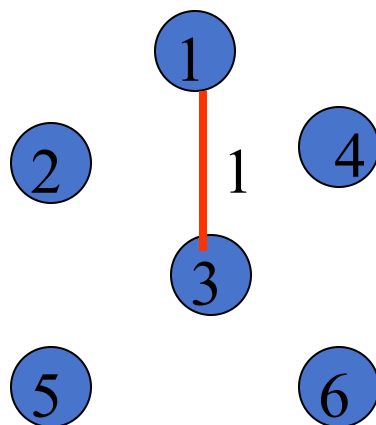
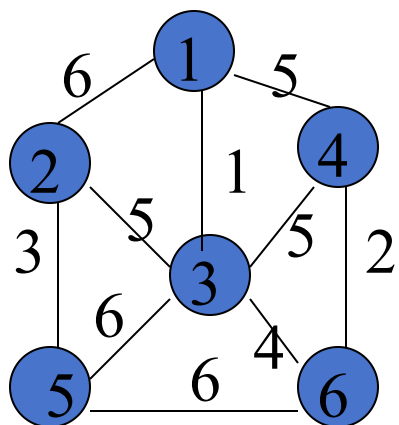
7.4.2.2 Kruskal算法

边表

v1
v2
w

1	4	2	3	3	2	1	5	3	
3	6	5	6	4	3	2	6	5	
1	2	3	4	5	5	6	6	6	

[示例] $n=6$



[算法步骤]

1)初始化T: 顶点集=所有顶点, 每个独立的顶点作为一棵树, 边集= $\emptyset$;

$O(n)$

2) 依**权值递增序**对图G的边排序, 结果为 $E[0..e-1]$ $O(e \log e)$

3) 依次检测E中的各边(u,v): $O(e \log e)$

3.1) 若u和v分属于T中两棵不同的树, 则将该边加入T, 并合并u和v分属的两棵树

3.2) 若T中所有顶点尚未属于一棵树, 转3)

$O(e \log e)$

其中3) 可参阅教材139页“树与等价问题”

[算法特点] 适用于稀疏图

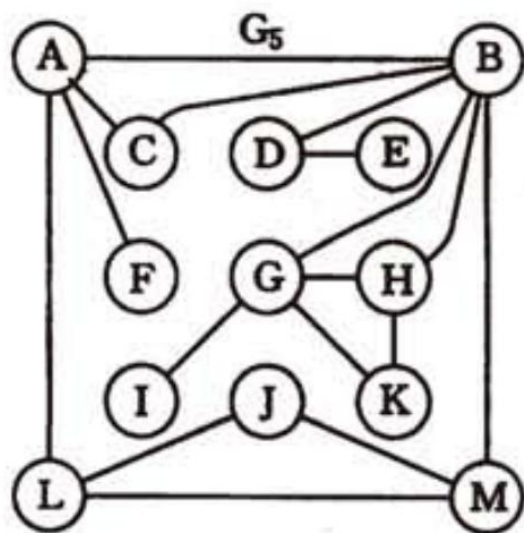
比较两种算法

算法名	普里姆算法	克鲁斯卡尔算法
方法	基于MST性质	加边
时间复杂度	$O(n^2)$	$O(eloge)$
适应范围	稠密图	稀疏图

关节点和重连通分量

如果删去顶点 v 及和 v 相关联的各边之后，将图的一个连通分量分割成两个或两个以上的连通分量，则称顶点 v 为该图的一个**关节点**。

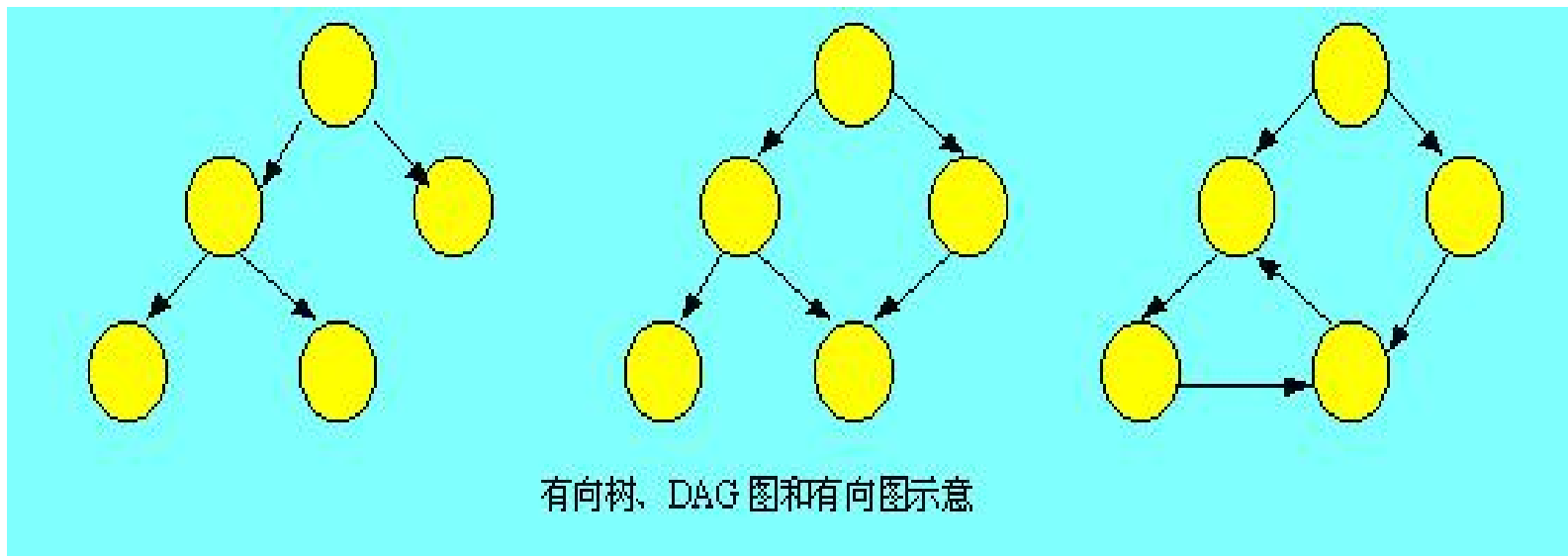
一个没有关节的点连通图称为**重连通图**。在重连通图上，任意一对顶点之间至少有两条路径。若在连通图上至少**删去 k 个**顶点才能破坏图的连通性，则称此图的**连通度为 k** 。



左图中有4个关节点，A、B、D、G

7.5 有向无环图及其应用

(DAG, Directed Acyclic Graph)



有向树是DAG的特例

假设以有向图表示一个工程的施工图或程序的数据流程图，则图中不允许出现回路。

AOV网与拓扑排序

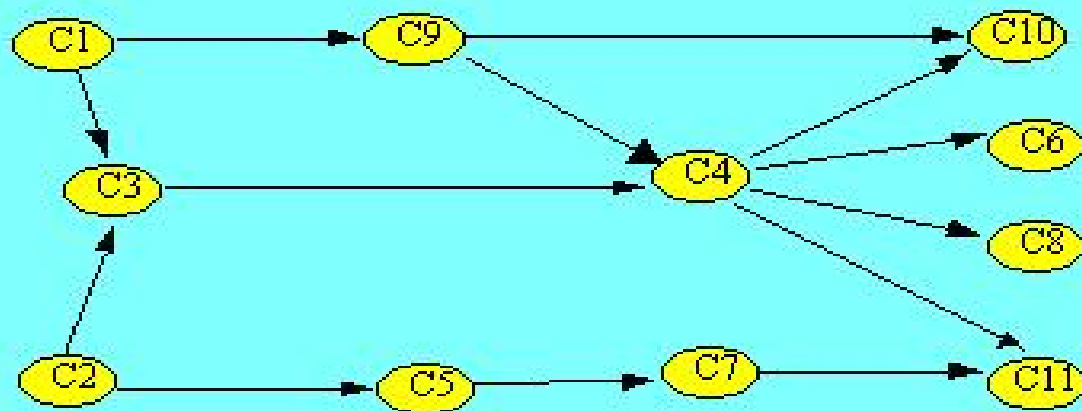
1. AOV网 (Activity On Vertex Network, 顶点表示活动的网)

(1) AOV概念: 顶点表示活动, 弧表示活动之间存在的制约关系网称为AOV。

(2) 用AOV表示一个工程:

例如：计算机专业的学生必须完成一系列规定的基础课和专业课才能毕业。

课程代号	课程名	先行课程代号	课程代号	课程名	先行课程代号
C1	程序设计导论	无	C7	计算机原理	C5
C2	高等数学	无	C8	算法分析	C4
C3	离散数学	C1, C2	C9	高级语言	C1
C4	数据结构	C3, C9	C10	编译系统	C4, C9
C5	普通物理	C2	C11	操作系统	C4, C7
C6	人工智能	C4			



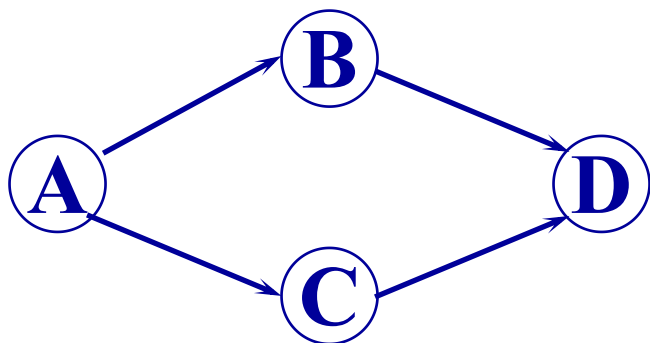
[问题目标]

当一个任务可以划分为若干个子任务/子活动/子事件，其中的任何子任务可能又以另外的一些子任务作为先决条件时，**如何排定子任务的执行顺序，达到整体任务的完成。**

[什么是拓扑排序]

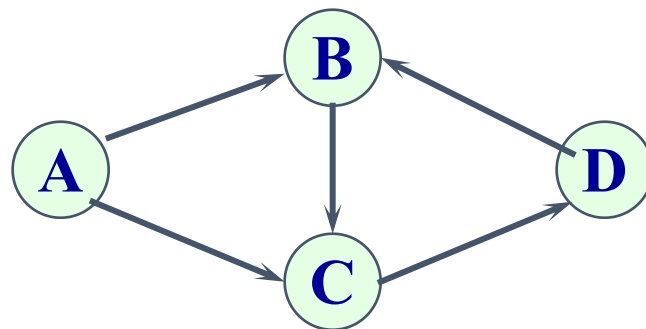
按照有向图给出的次序关系，将图中顶点排成一个**线性**序列。

检查有向图中是否存在回路的方法之一，是对有向图进行**拓扑排序**。



可求得**拓扑有序序列**:

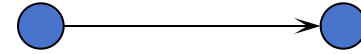
A B C D 或 **A C B D**



不能求得它的**拓扑有序序列**。

因为图中存在一个回路 {B, C, D}

[无前趋的顶点优先算法]



算法原理

在一个拓扑序列中，每个顶点必定出现在它的所有前趋顶点之后。

算法思想

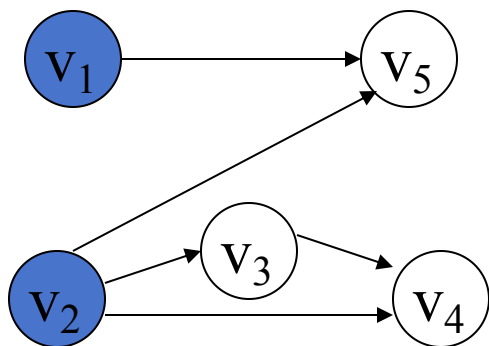
1. 选择一个入度为0的顶点(无前趋的顶点)，输出它
2. 删去该顶点及其关联的所有出边

重复上述两步，直至图中不再有入度为0的顶点为止。

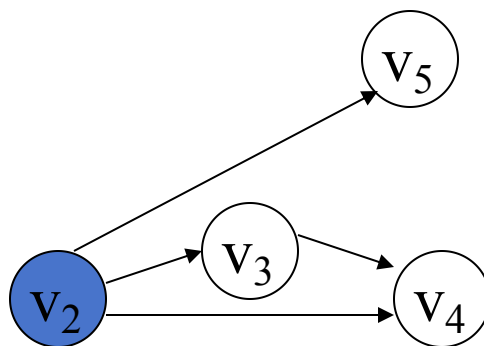
若所有顶点均被输出，则排序成功，

否则图中存在有向环。

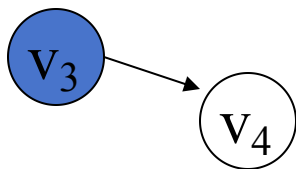
例



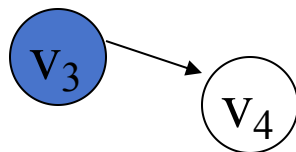
V_1



$V_1 V_2$



$V_1 V_2 V_5$



$V_1 V_2 V_5 V_3$



$V_1 V_2 V_5 V_3 V_4$

有向无环图的拓扑序列**不唯一**

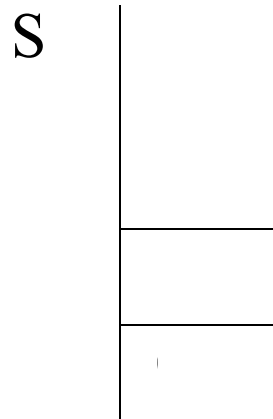
数据结构

- 主：邻接表G（或者邻接矩阵）
- 辅：
 - 一维数组indegree[0..n-1]：记录每个顶点的当前入度
 - 栈S（或者队列Q）：暂存当前所有待输出的入度为0的顶点。作用是避免每次扫描indegree，提高算法效率。

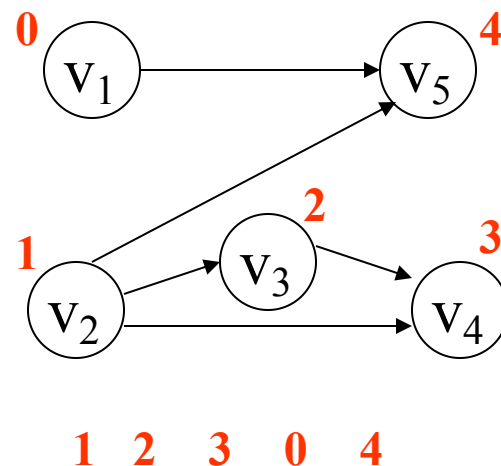
例中各辅助数据结构初值如下：

indegree	0	0
	1	0
	2	0
	3	0
	4	0

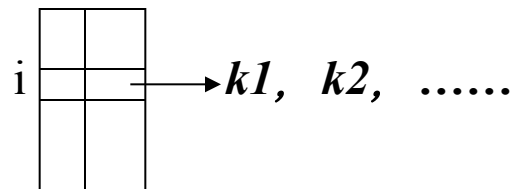
初始入度



入度为0的顶点序号入栈



算法步骤



1)初始化辅助数据结构:

1.1)扫描邻接表G的各出边表, 在indegree中计数; $O(n+e)$

1.2)检查indegree数组, 将入度为0的顶点入栈; $O(n)$

1.3)设置顶点计数器 $count = 0$ $O(1)$

2)若栈不空, 反复执行以下操作:

2.1)栈顶元素i出栈; $O(n)$

2.2)输出顶点i, count加 1; $O(n)$

2.3)扫描顶点i的出边表: $O(n+e)$

修改indegree, 将出边表上顶点的入度减 1;

若其入度为 0, 则令其入栈

3)若 $count < \text{顶点数} vexnum$, write (“排序失败!”) $O(1)$

$O(n+e)$

[算法描述]

Status TopologicalSort(ALGraph G)

```
{ FindInDegree(G, indegree); // 求各顶点入度 indegree[G.vexnum]
  InitStack(S);
  for (i=0; i<G.vexnum; ++i) if (! indegree[i]) push(S, i);
  count=0;
  while (! StackEmpty(S)) {
    pop(S, i); printf(i, G.vertices[i].data); ++count;
    for (p=G.vertices[i].firstarc; p; p=p->nextarc) { // 修改入度
      k=p->adjvex;
      if (! (--indegree[k])) push(S,k); // 若入度为0则入栈
    }
  }
  if (count<G.vexnum) return ERROR; else return OK;
} // TopologicalSort
```


[无后继的顶点优先]

算法原理

在一个拓扑序列中，每个顶点必定出现在它的所有后继顶点之前。

方法一

(按逆拓扑次序生成顶点序列)

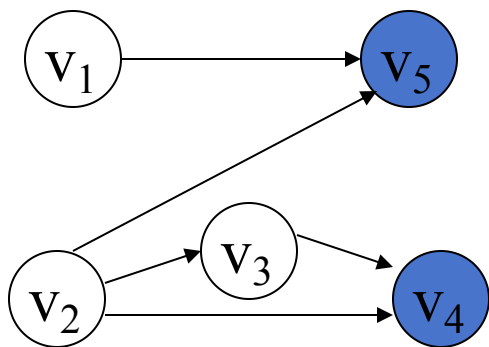
1. 选择一个出度为0的顶点(无后继的顶点)，输出它
2. 删去该顶点及其关联的所有入边

重复上述两步，直至图中不再有出度为0的顶点为止。

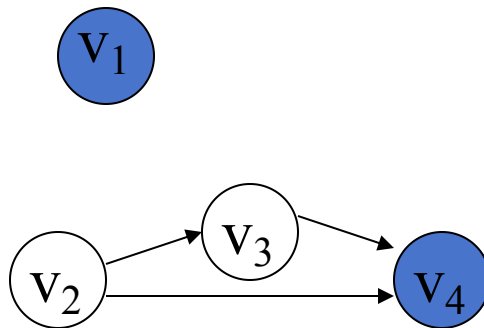
若所有顶点均被输出，则排序成功，

否则图中存在有向环。

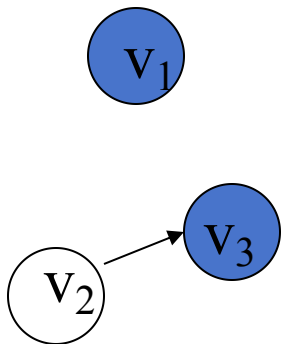
例



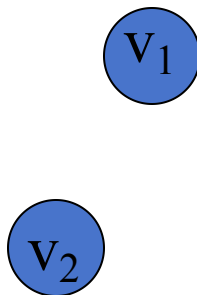
V_5



$V_5 V_4$



$V_5 V_4 V_3$



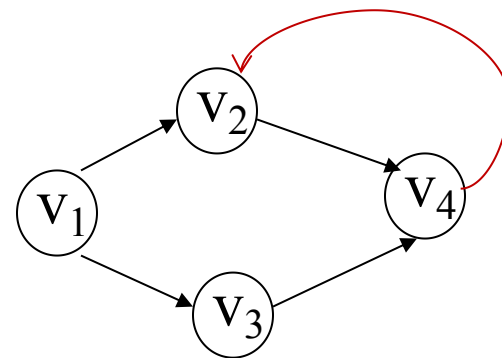
$V_5 V_4 V_3 V_2$



$V_5 V_4 V_3 V_2 V_1$



方法二 利用深度优先遍历 (从入度为0的顶点出发)

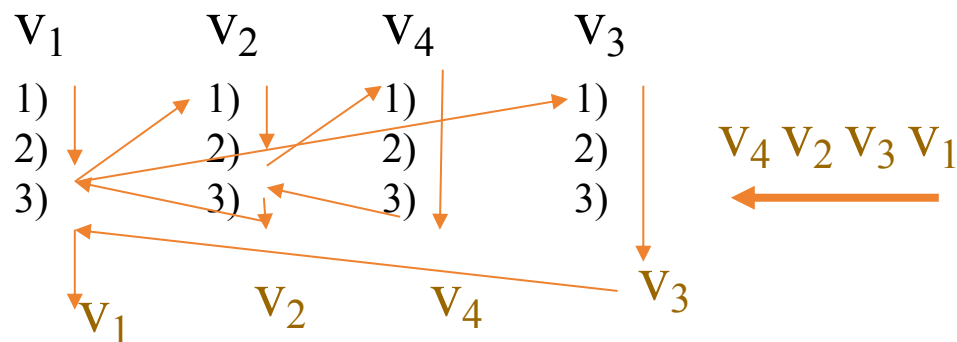


step1: 访问顶点 v ，并记录 v 已被访问

step2: 依次从 v 的未访问的邻接点出发，深度优先
拓扑排序图 G 。

step3: 输出顶点 v (此时 v 相当于无后继的顶点)

注意：此方法仅适用于有向无环图
(此算法不能检测出有向环，得到
虚假拓扑序列)



7.5.2 关键路径



课外作业：

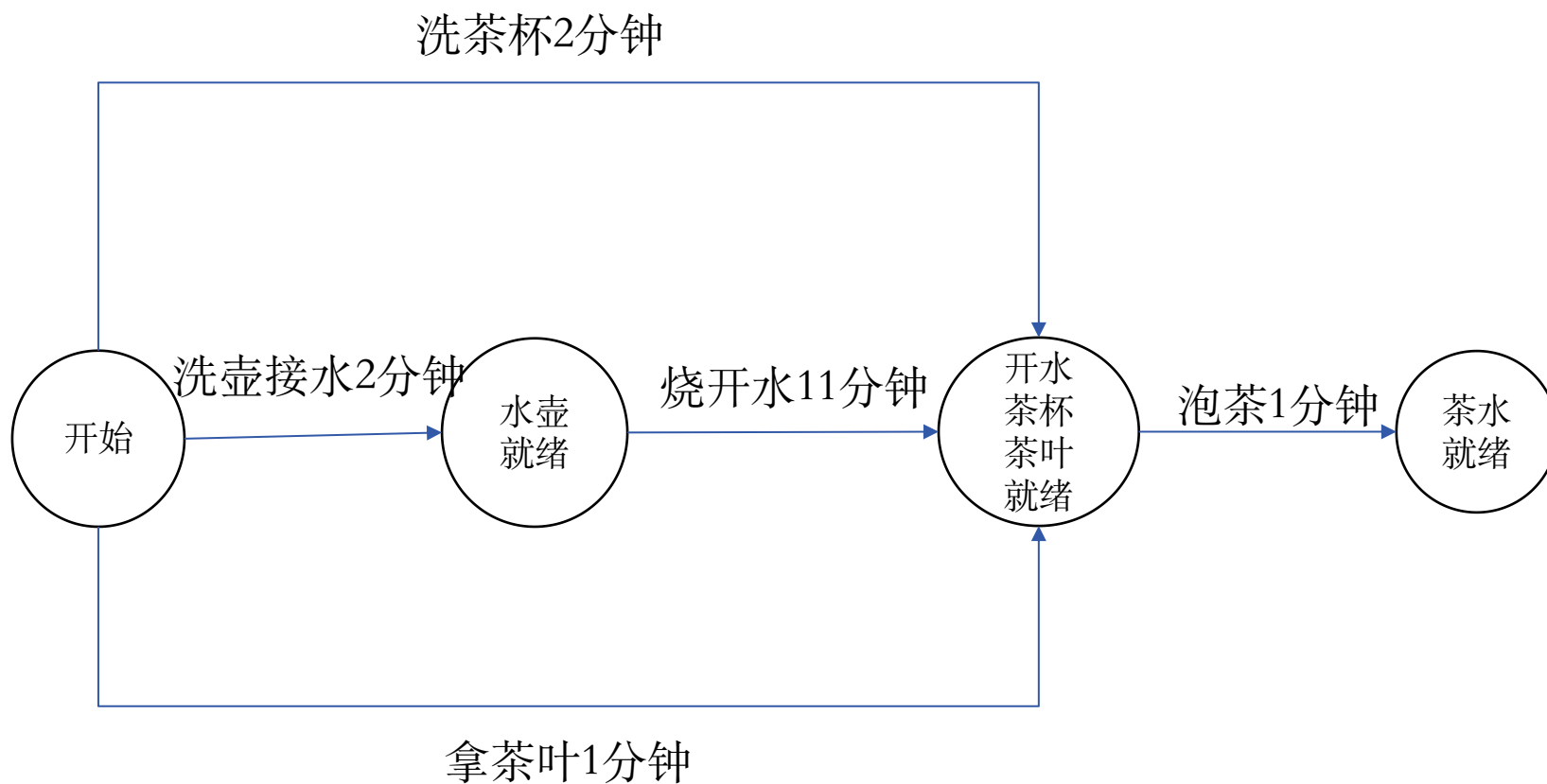
啊！妈妈回来了。妈妈今天可辛苦了，她想先喝杯茶，润润嗓子。

洗茶杯	2分钟
烧开水	11分钟
拿茶叶	1分钟
洗壶接水	2分钟
泡茶	1分钟

1. 你能让妈妈在最短的时间内喝到茶吗？
2. 小组内展示交流，介绍自己的流程图及设计原因。

小学数学：合理安排时间

7.5.2 关键路径



7.5.2 关键路径

1. AOE网 (Activity On Edge Network, 边表示活动的网)

(1) AOE网概念: 若在带权的有向图中, 以顶点表示事件, 以有向边表示活动, 边上的权值表示活动的开销 (如该活动持续的时间), 则此带权的有向图称为AOE网。

(2) AOE网表示一项工程能表示出:

- ①完成预定工程计划所需要进行的活动;
- ②每个活动计划完成的时间;
- ③要发生哪些事件以及这些事件与活动之间的关系;

(3) 通过AOE网可以求得：

①估算工程完成的时间

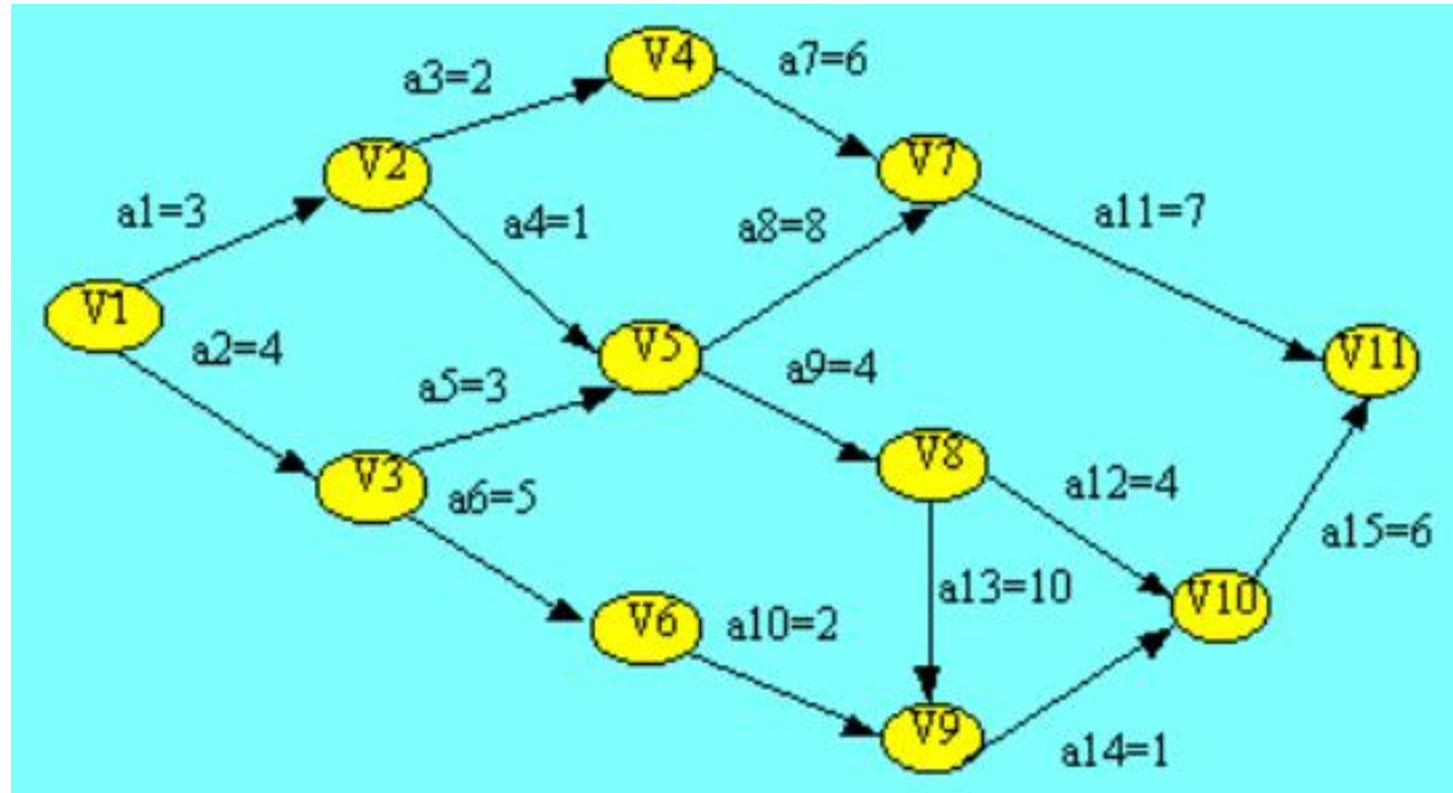
②确定哪些活动是影响工程进度的关键。

(4) AOE网的两个特点：

①只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始。

②只有在进入一某顶点的各有向边所代表的活动都已经结束，该顶点所代表的事件才能发生。

下图给出了一个具有15个活动、11个事件的假想工程的AOE网。



1. 总工期最少是多少？
2. 能否加快某些活动的进度，使整个工期缩短呢？

[相关概念和术语]

源点 入度为0的顶点，即工程的开始点

汇点 出度为0的顶点，即工程的完成点

关键路径 从源点到汇点的**最长路径**

关键路径长度=最短工期

关键活动 关键路径上的活动 $e(i)=l(i)$

关键活动的加快可以缩短工期

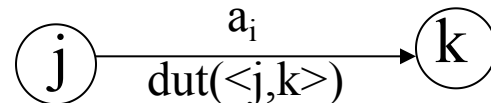
[确定关键路径时涉及的几个变量]

$e(i)$: 活动 a_i 的**最早**开始时间

$l(i)$: 活动 a_i 的**最迟**开始时间

$ve(j)$: 事件 v_j 的**最早**发生时间

$vl(j)$: 事件 v_j 的**最迟**发生时间



$$e(i)=ve(j)$$

$$l(i)=vl(k)-dut(<j,k>)$$

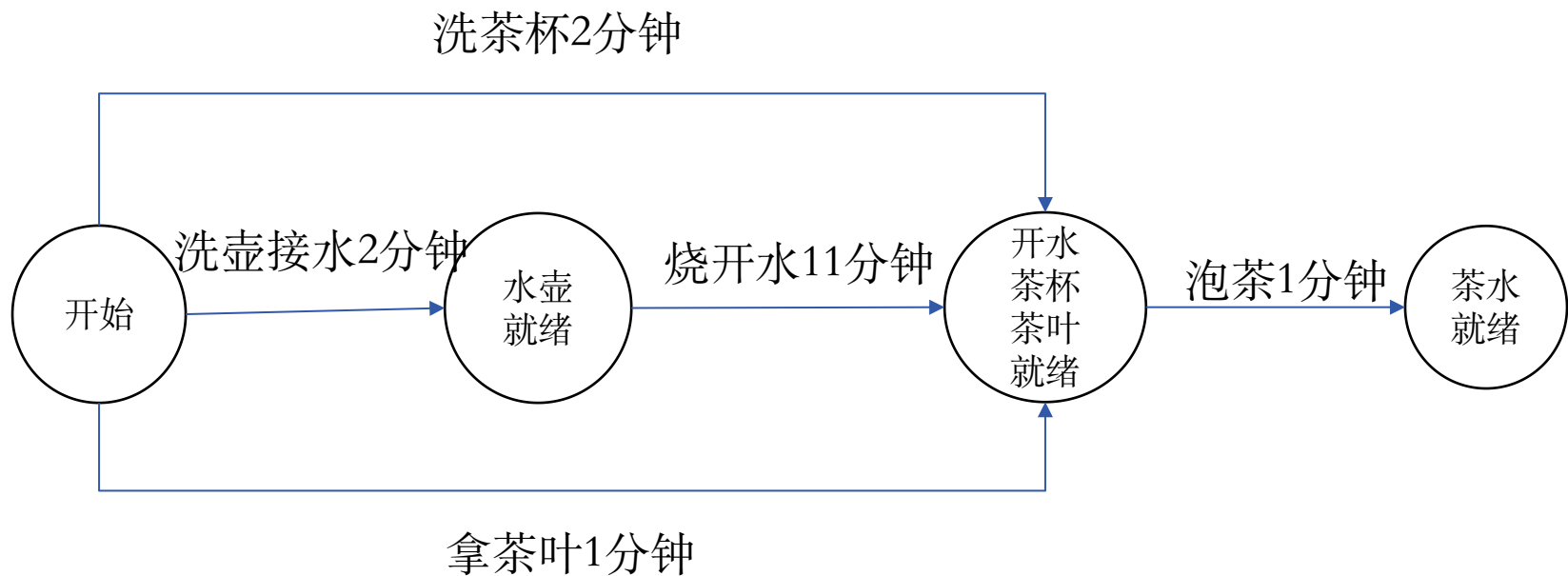
为了在AOE网中找出关键路径，需要定义几个参量：

(1) 事件的**最早发生时间** $ve[k]$ ：定义为从 V_1 到事件 V_k 的最长路径

(2) 事件的**最迟发生时间** $vl[k]$ ：不拖延整个工期的情况下，最迟发生时间

(3) 活动 a_i 的**最早开始时间** $e[i]$ ：该活动的弧尾事件的最早发生时间

(4) 活动 a_i 的**最晚开始时间** $l[i]$ ：不拖延整个工期的情况下最晚开始时间



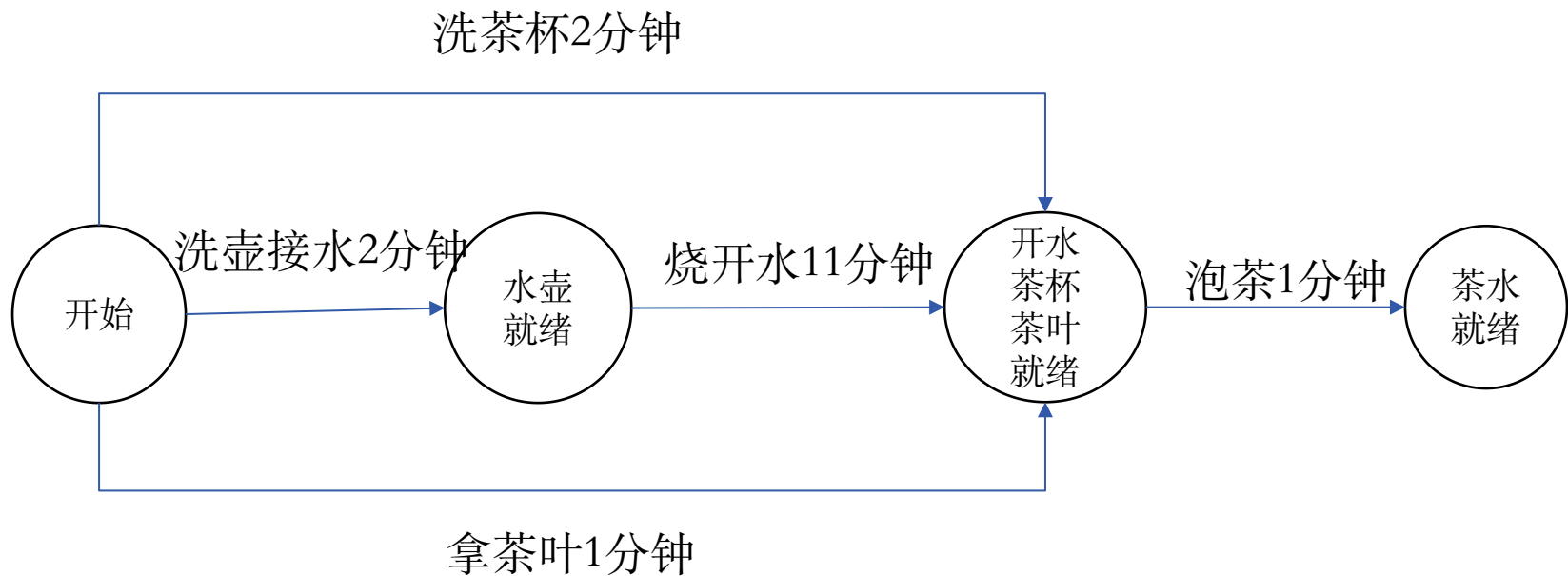
各个事件的最早发生时间:

开始: 0

水壶就绪: 2分钟

开水茶杯茶叶就绪: 13分钟

茶水就绪: 14分钟



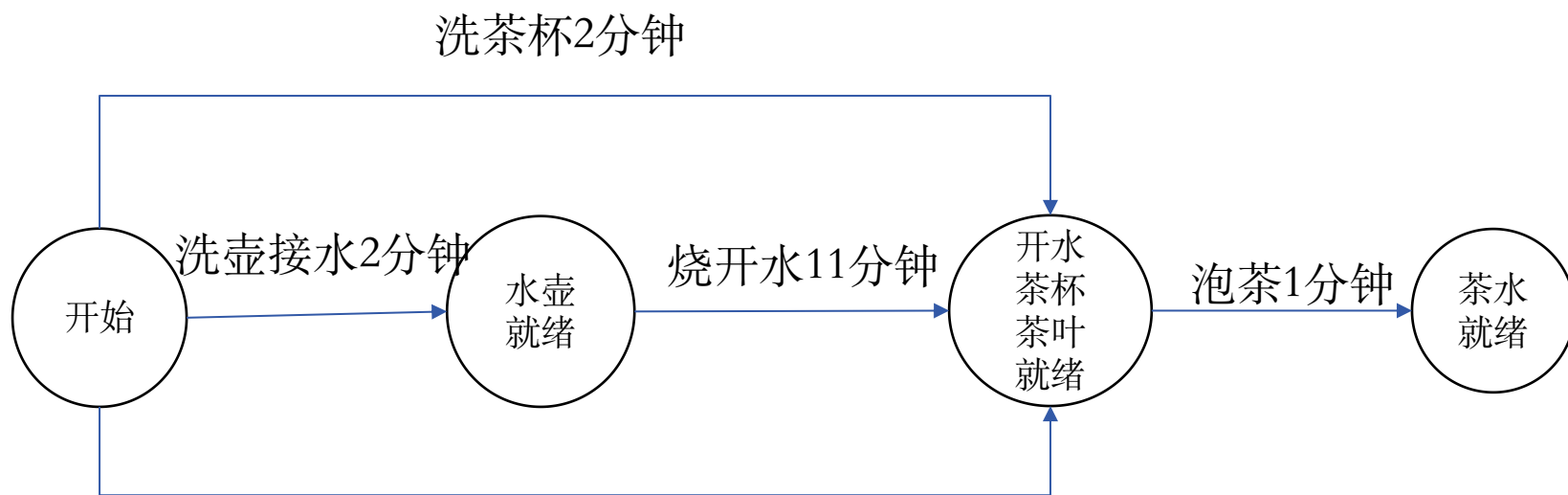
各个事件的最晚发生时间:

茶水就绪: 14分钟

开水茶杯茶叶就绪: 13分钟

水壶就绪: 2分钟

开始: 0分钟



拿茶叶1分钟

各个活动的最早开始时间

最晚开始时间

泡茶: 13分钟

13分钟

烧开水: 2分钟

2分钟

拿茶叶: 0分钟

12分钟

洗壶接水: 0分钟

0分钟

洗茶杯: 0分钟

11分钟

最早开始时间=最晚开始时间的活动就是关键活动

哪些是关键活动？

根据每个活动的最早开始时间 $e[i]$ 和最晚开始时间 $l[i]$ 就可判定该活动是否为关键活动，也就是那些 $l[i]=e[i]$ 的活动就是关键活动，而那些 $l[i]>e[i]$ 的活动则不是关键活动， $l[i]-e[i]$ 的值为活动的时间余量。关键活动确定之后，关键活动所在的路径就是关键路径。

求关键路径的过程

(1) 按照式5-1求事件的最早发生时间 $ve[k]$

$$ve[1]=0$$

$$ve[2]=3$$

$$ve[3]=4$$

$$ve[4]=ve[2]+2=5$$

$$ve[5]=\max\{ve[2]+1, ve[3]+3\}=7$$

$$ve[6]=ve[3]+5=9$$

$$ve[7]=\max\{ve[4]+6, ve[5]+8\}=15$$

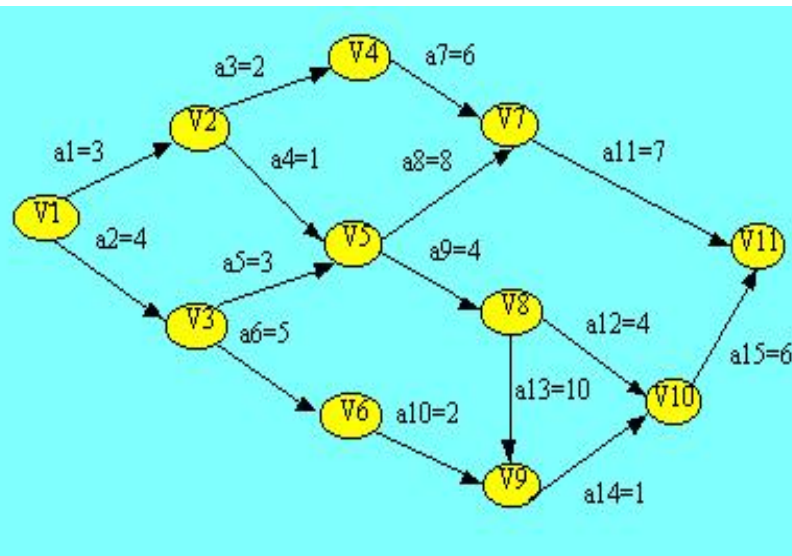
$$ve[8]=ve[5]+4=11$$

$$ve[9]=\max\{ve[8]+10, ve[6]+2\}=21$$

$$ve[10]=\max\{ve[8]+4, ve[9]+1\}=22$$

$$ve[11]=\max\{ve[7]+7, ve[10]+6\}=28$$

$$\begin{cases} ve(0)=0 \\ ve(j)=\max\{ve(i)+dut(<i,j>)\} \\ (i \in j \text{的前驱}) \end{cases}$$



正推

(2) 按照式5-2求事件的最迟发生时间 $vl[k]$ 。

$$vl[11] = ve[11] = 28$$

$$vl[10] = vl[11] - 6 = 22$$

$$vl[9] = vl[10] - 1 = 21$$

$$vl[8] = \min\{vl[10] - 4, vl[9] - 10\} = 11$$

$$vl[7] = vl[11] - 7 = 21$$

$$vl[6] = vl[9] - 2 = 19$$

$$vl[5] = \min\{vl[7] - 8, vl[8] - 4\} = 7$$

$$vl[4] = vl[7] - 6 = 15$$

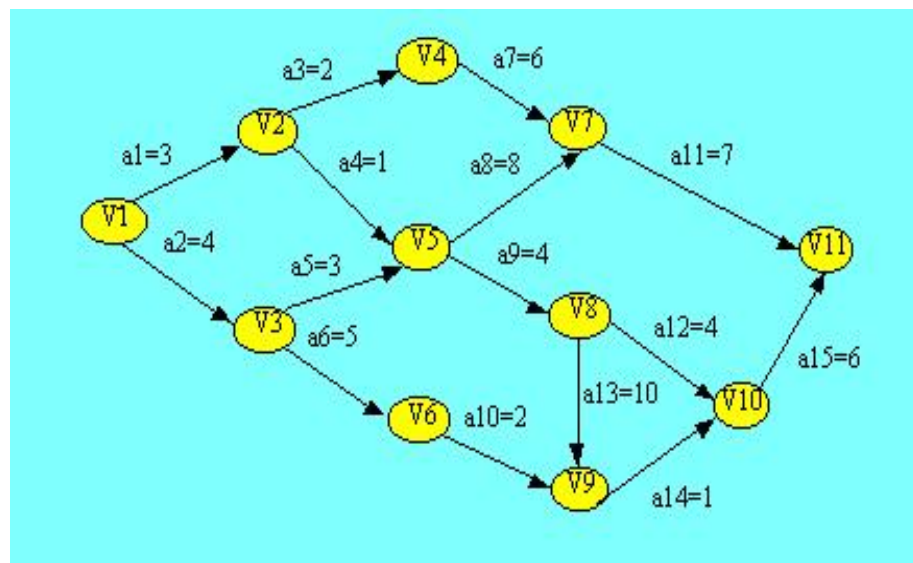
$$vl[3] = \min\{vl[5] - 3, vl[6] - 5\} = 4$$

$$vl[2] = \min\{vl[4] - 2, vl[5] - 1\} = 6$$

$$vl[1] = \min\{vl[2] - 3, vl[3] - 4\} = 0$$

$$\begin{cases} vl(n-1) = ve(n-1) & \{\text{保证 } ve(n-1) \text{ 的前提下}\} \\ vl(j) = \min\{vl(k) - \text{dut}(\langle j, k \rangle)\} \end{cases}$$

($k \in j$ 的后继)



倒推

(3) 求活动 a_i 的最早开始时间 $e[i]$ 和 (4) 最晚开始时间 $l[i]$

活动a1	$e[1]=ve[1]=0$	$l[1]=vl[2]-3=3$
活动a2	$e[2]=ve[1]=0$	$l[2]=vl[3]-4=0$
活动a3	$e[3]=ve[2]=3$	$l[3]=vl[4]-2=13$
活动a4	$e[4]=ve[2]=3$	$l[4]=vl[5]-1=6$
活动a5	$e[5]=ve[3]=4$	$l[5]=vl[5]-3=4$
活动a6	$e[6]=ve[3]=4$	$l[6]=vl[6]-5=14$
活动a7	$e[7]=ve[4]=5$	$l[7]=vl[7]-6=15$
活动a8	$e[8]=ve[5]=7$	$l[8]=vl[7]-8=13$
活动a9	$e[9]=ve[5]=7$	$l[9]=vl[8]-4=7$
活动a10	$e[10]=ve[6]=9$	$l[10]=vl[9]-2=19$
活动a11	$e[11]=ve[7]=15$	$l[11]=vl[11]-7=21$
活动a12	$e[12]=ve[8]=11$	$l[12]=vl[10]-4=18$
活动a13	$e[13]=ve[8]=11$	$l[13]=vl[9]-10=11$
活动a14	$e[14]=ve[9]=21$	$l[14]=vl[10]-1=21$
活动a15	$e[15]=ve[10]=22$	$l[15]=vl[11]-6=22$

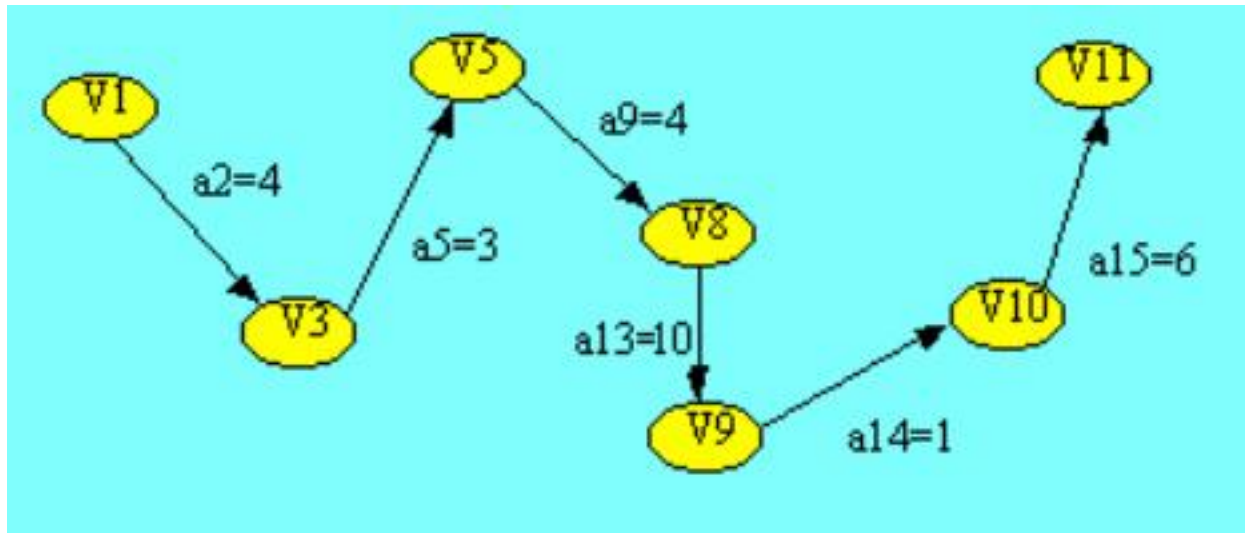
$$e(i)=ve(j)$$

$$l(i)=vl(k)-dut(<j,k>)$$

关键活动 $e(i)=l(i)$

(5) 求关键路径:

比较 $e[i]$ 和 $l[i]$ 的值可判断出 $a_2, a_5, a_9, a_{13}, a_{14}, a_{15}$ 是关键活动，关键路径如下图所示。



[求关键路径算法步骤]

主数据结构：图—邻接矩阵或邻接表

辅助： $ve(0..n-1)$, $vl(0..n-1)$, $e(0..m-1)$, $l(0..m-1)$

{ n 为图的顶点数目； m 为图的弧数目}

T—栈，存储拓扑排序的顶点序列

(1)输入所有弧的信息，建立AOE网的存储结构

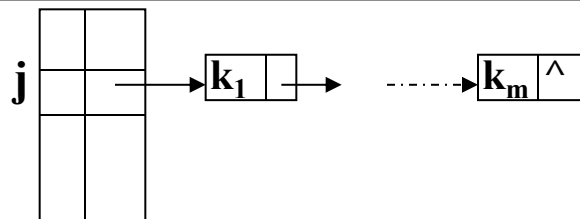
(2)求拓扑排序的顶点序列

每求得一个顶点 j ，则使 j 入栈T，并调整 j 的所有后继的 ve （最早发生）值

(3)按栈T的从栈顶到栈底的顺序求顶点的 vl （最晚发生）值

(4)求每个活动 i 的 $e(i)$ 和 $l(i)$ 值，确定关键路径

[算法描述-Part1]



Status TopologicalOrder(ALGraph G, Stack &T)

//计算各顶点的**最早发生**时间ve(全局变量), 并用栈T返回G的一个拓扑序列

{ **FindInDegree**(G, indegree); //求各顶点入度indegree[G.vexnum]

InitStack(S); count=0; ve[0..G.vexnum-1]=0; //初始化为0, 栈S用来辅助求拓扑序列

for (i=0; i<G.vexnum; ++i) if (! indegree[i]) push(S, i); //入度为0的入栈

InitStack(T);

while (! StackEmpty(S)) {

pop(S, j); **push**(T, j); ++count;

for (p=G.vertices[j].firstarc; p; p=p->nextarc){

k=p->adjvex;

if (! (--indegree[k])) push(S,k); //入度-1后若为0则入栈

if (ve[j]+(p->weight) > ve[k]) **ve[k]= ve[j]+(p->weight)**; //更新ve[k], 求max

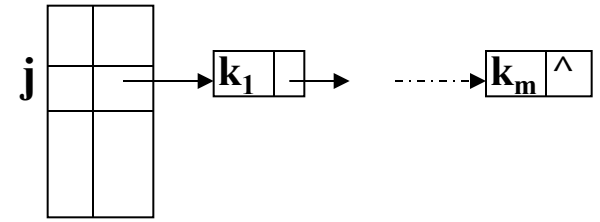
}

}

if (count<G.vexnum) return ERROR; else return OK;

}//TopologicalOrder

[算法描述-Part2]



Status CriticalPath(ALGraph G)

//G为有向网，输出G的各项关键活动

```
{ if (! TopologicalOrder(G,T) ) return ERROR;
  vl[0.. G.vexnum-1] = ve[G.vexnum-1] ;
  while (! StackEmpty(T)) //按拓扑逆序求各顶点的vl值
    for ( pop(T, j), p=G.vertices[j].firstarc; p; p=p->nextarc) {
      k=p->adjvex;
      if (vl[k]-(p->weight) < vl[j]) vl[j]= vl[k]-(p->weight); //更新vl, 求min
    }
  for (j=0; j<G.vexnum; ++j) //求各活动的ee、el和确定关键活动
    for (p=G.vertices[j].firstarc; p; p=p->nextarc) {
      k=p->adjvex; dut=p->weight;
      ee=ve[j]; el=vl[k]-dut;
      tag=(ee==el) ? '*' : '.';
      printf(j, k, dut, ee, el, tag);
    }
}
```

}//CriticalPath

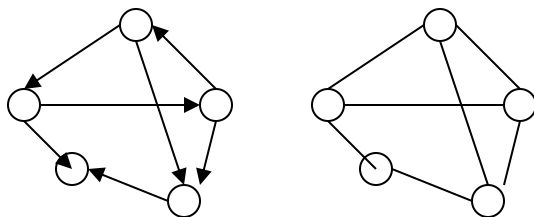
7.6 最短路径

7.6.1 概述

交通网、通信网中要解决的应用问题：两地之间是否有路相通？在多条通路的情况下，哪一条最短？

[问题对象]

有向网络（可扩展到无向网络）



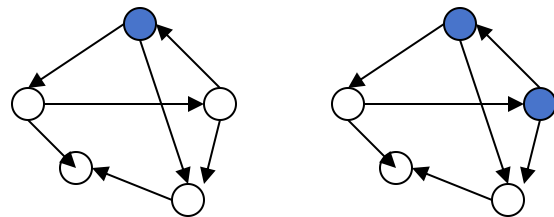
[两个典型算法]

单源最短路径（可扩展到多源问题）

迪杰斯特拉([Dijkstra](#))算法

每一对顶点间的最短路径

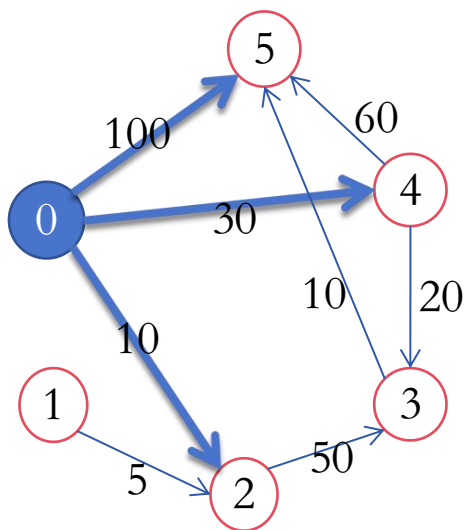
弗洛伊德([Floyd](#))算法



7.6.2 从某个源点到其余各顶点的最短路径

[Dijkstra算法原理]

按路径长度递增次序产生各顶点的最短路径。



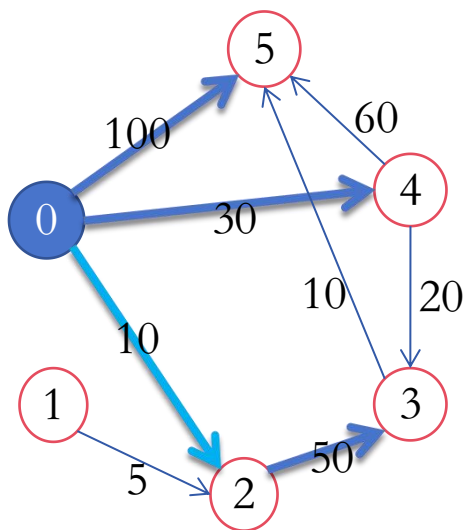
设源点为 V_0 ，求到其余各顶点的最短路径：

- 先找到和源点之间最短路径最短的顶点。
 - 最短的肯定属于直接邻接自源点的顶点之一，即 V_2 、 V_4 、 V_5 之一，不可能是间接的，如 V_3 。
- 在上述从源点出发的弧中找权值最小的 $\langle V_0, V_2 \rangle$ ， V_2 即为最短路径最短的顶点。

7.6.2 从某个源点到其余各顶点的最短路径

[Dijkstra算法原理]

按**路径长度递增**次序产生各顶点的最短路径。



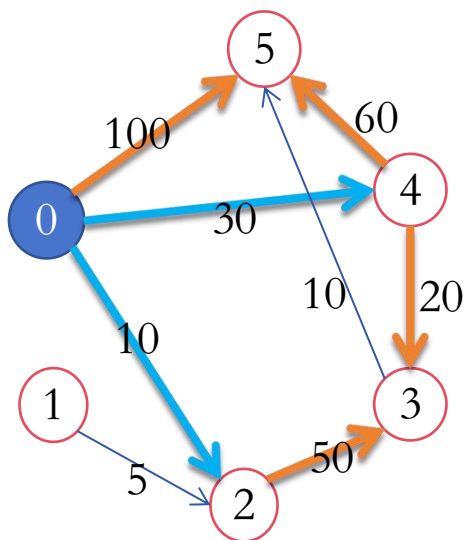
设源点为 V_0 ，求到**其余各顶点**的最短路径：

- 再找到和源点之间最短路径**长度次短**的顶点
 - 可能性1：该顶点是直接邻接自源点的顶点，如 V_4 、 V_5 。
 - 可能性2：经过 v_2 后再到达的某个顶点。
 - 在以上可能性中找最小

7.6.2 从某个源点到其余各顶点的最短路径

[Dijkstra算法原理]

按路径长度递增次序产生各顶点的最短路径。



设源点为 V_0 ，求到其余各顶点的最短路径：

- 再依次求其它...
 - 可能性1：该顶点是直接邻接自源点的顶点(已经确定的顶点除外)。
 - 可能性2：从源点经过已求得最短路径的顶点，再到达该顶点。
 - 在以上可能性中找最小

设置辅助数组Dist，其中每个分量Dist[k] 表示当前所求得的从源点 v_0 到其余各顶点 k 的最短路径。

1) 初值: $\text{Dist}[k] = \langle \text{源点到顶点 } k \text{ 的弧上的权值} \rangle$

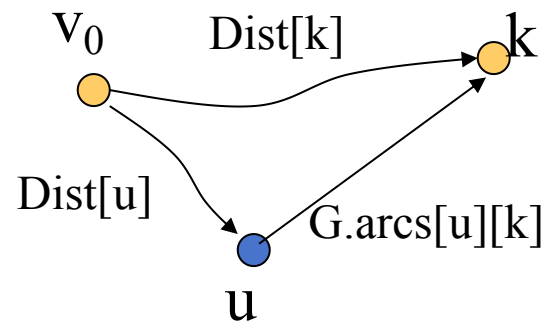
$$G.\text{arcs}[v_0][k]$$

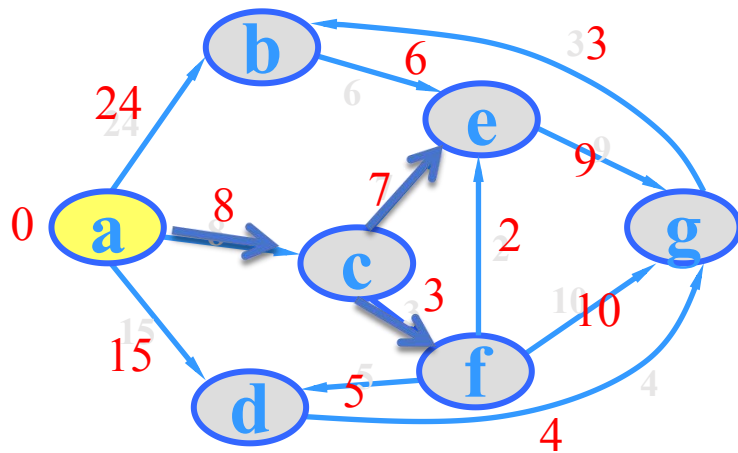
2) 在所有从源点出发的弧中选取一条权值最小的弧，即为第一条最短路径。假设求得最短路径的顶点为u。

3) 修改其它各顶点的Dist[k]值。若 $\text{Dist}[u] + G.\text{arcs}[u][k] < \text{Dist}[k]$

则将 $\text{Dist}[k]$ 改为 $\text{Dist}[u] + G.\text{arcs}[u][k]$

4) 在没有确定最短路径的顶点中选择一条路径最短的，被选中的顶点再设为u，转3)，直到所有顶点被选完。





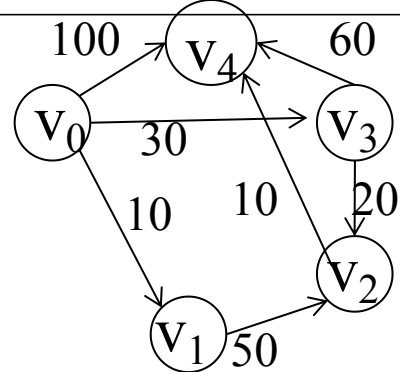
	a	b	c	d	e	f	g
a	0	24	8	15	∞	∞	∞
b	∞	0	∞	∞	6	∞	∞
c	∞	∞	0	∞	7	3	∞
d	∞	∞	∞	0	∞	∞	4
e	∞	∞	∞	∞	0	∞	9
f	∞	∞	∞	5	2	0	10
g	∞	3	∞	∞	∞	∞	0

~~选择 dist 值最小的未处理顶点 u 并标记为已处理~~

~~更新所有与 u 相邻的未处理顶点 v 的 dist 值~~

	a	b	c	d	e	f	g	
i	0	1	2	3	4	5	6	S
dist[i]		22	8	15	13	11	19	a c f e
path[i]		adgb	ac	ad	acfe	acf	adg	d g b

[数据结构] 设图中的顶点个数为 n



- 主：邻接矩阵 $G.arcs[n][n]$ (或者邻接表)
- 辅：
 - 一维数组 $S[0..n-1]$ ：记录相应顶点**是否已被确定**最短距离
 - 一维数组 $D[0..n-1]$ ：记录源点到相应顶点**路径长度**
 - 一维数组 $P[0..n-1]$ ：记录相应顶点的**双亲顶点**

dist

*Path*变异

例中各辅助数据结构初值如下：

S:	0	FALSE
		...
	n-1	FALSE

D:	0	0
	1	10
	2	∞
	3	30
	n-1	100

P:	0	-1
	1	0
	2	-1
	3	0
	n-1	0

[算法步骤]

设确定 v 为源点

1)初始化辅助数据结构: ($i=0..n-1$) $O(n)$

1.1) $S[i]=FALSE$;

1.2) $D[i]=G.arcs[v][i]$;

1.3) 若 $D[i]$ 不为 ∞ , 则 $P[i]=v$, 否则 $P[i]=-1$

2)确定源点自身的最短距离: $S[v]=TRUE, D[v]=0$ $O(1)$

3)循环确定其余 $n-1$ 个顶点到 v 的最短距离: $O(n^2)$

3.1)找出 D 中未被确定最短距离的顶点中距离最小的顶点,

设该顶点序号为 k , 且 $D[k]=\min$;

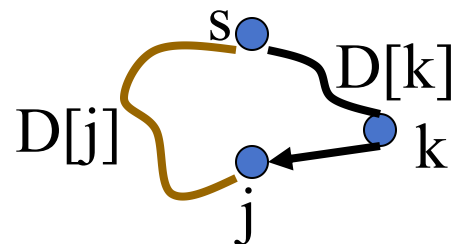
3.2)若 $\min$ 为 ∞ , 退出本算法;

3.3)赋值 $S[k]=TRUE$;

3.4)调整尚未被确定最短距离的顶点的估算距离: ($j=0..n-1$)

若 $S[j]=\text{FALSE}$ 且 $D[j]>D[k]+G.\text{arcs}[k][j]$ //可以优化

则 $D[j]=D[k]+G.\text{arcs}[k][j]$, $P[j]=k$ //通过k更佳



4)依次打印各顶点的最短路径及其距离:

$O(n^2)$

($i=0..n-1$)

4.1)printf(i, D[i]);

4.2)确定终点i的前趋: $\text{pre}=P[i]$

4.3)若 $\text{pre}=-1$, 则转4)处理下一顶点

否则print(pre), $\text{pre}=P[\text{pre}]$, 转4.3)

$O(n^2)$

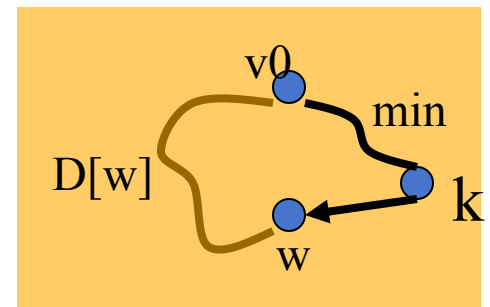
说明: 本算法与教材189页算法7.15使用的P结构不同, 算法7.15不能得到最短路径上顶点的顺序。

```

void ShortestPath_DIJ( Mgraph G,int v0,
                      PathMatrix &P, ShortPathTable &D)
{
    for (v = 0; v < vexnum; v++) {
        final[v] = FALSE; D[v] = G.arcs[v0][v];
        for (w = 0; w < vexnum; w++) P[v][w] = FALSE;
        if (D[v] < INFINITY) P[v][v0] = p[v][v] = TRUE;
    }
    D[v0] = 0; final[v0] = TRUE;
    for (i = 1; i < vexnum; i++) {
        min = INFINITY;
        for (w = 0; w < vexnum; w++)
            if (!final[w] && D[w] < min) { k = w; min = D[w]; }
        final[k] = TRUE;
        /*考虑新加入的节点k */
        for (w = 0; w < vexnum; w++)
            if (!final[w] && (min + G.arcs[k][w] < D[w])) {
                D[w] = min + G.arcs[k][w];
                P[w] = P[k]; P[w][w] = TRUE;
            }
    }
}

```

P	0	1	2		w		n-1
0							
1							
⋮							
⋮							
w	F	F	F		F		F
⋮							
⋮							
n-1							



7.6.3 每一对顶点之间的最短路径

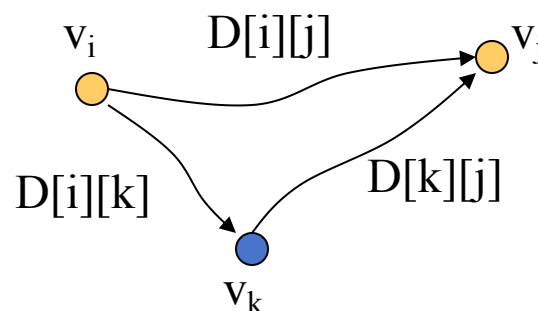
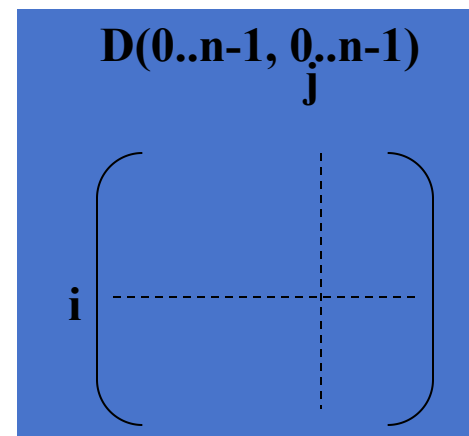
- 每次选一个顶点为源点，反复利用Dijkstra算法
- Floyd算法 $O(n^3)$ 但算法相对简单

[Floyd算法思想]

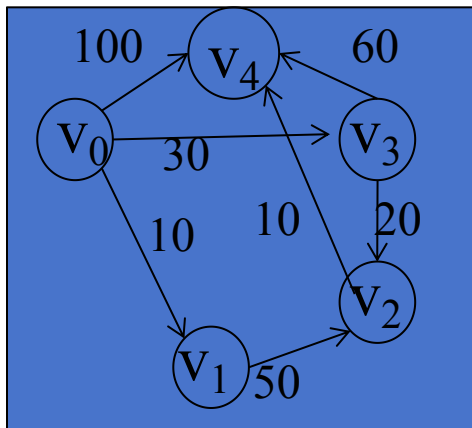
- 利用二维数组 $D(0..n-1, 0..n-1)$, $D[i][j]$ 记录当前 v_i 到 v_j 的最短路径长度, 初值 $D^{(-1)}[i][j]=G.arcs[i][j]$
- 集合 S 记录当前允许的中间顶点, 初值 $S=\phi$
- 依次向 S 中加入 $v_0, v_1, \dots, v_{n-1}$, 每加入一个顶点, 对 $D[i][j]$ 进行一次修正: 设 $S=\{v_0, v_1, \dots, v_{k-1}\}$, 加入 v_k , 则 $D^{(k)}[i][j]=\min\{D^{(k-1)}[i][j], D^{(k-1)}[i][k]+D^{(k-1)}[k][j]\}$

$D^{(k)}[i][j]$ 的含义: 允许的中间顶点的序号最大为 k 时的从 v_i 到 v_j 的最短路径长度

$$(0 \leq k \leq n-1)$$



[例]



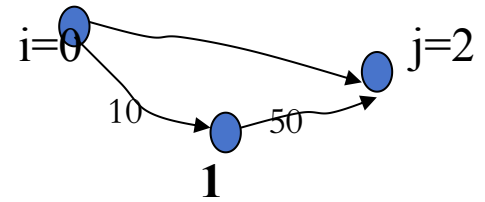
G.vexs

0	V0
1	V1
2	V2
3	V3
4	V4

G.arcs

	0	1	2	3	4
0	0	10	∞	30	100
1	∞	0	50	∞	∞
2	∞	∞	0	∞	10
3	∞	∞	20	0	60
4	∞	∞	∞	∞	0

其它顶点不能到
顶点0, 所以加
入 V_0 无改善



$D^{(-1)} = G.arcs$

	0	1	2	3	4
0	0	10	∞	30	100
1	∞	0	50	∞	∞
2	∞	∞	0	∞	10
3	∞	∞	20	0	60
4	∞	∞	∞	∞	0

$D^{(0)}$

	0	1	2	3	4
0	0	10	∞	30	100
1	∞	0	50	∞	∞
2	∞	∞	0	∞	10
3	∞	∞	20	0	60
4	∞	∞	∞	∞	0

$D^{(1)}$

	0	1	2	3	4
0	0	10	60	30	100
1	∞	0	50	∞	∞
2	∞	∞	0	∞	10
3	∞	∞	20	0	60
4	∞	∞	∞	∞	0

$path^{(-1)}$ 直接路径

(0,0) (0,1) () ...

非 ∞ 处以 (i,j)
赋初值, ∞ 处以
()赋值

$path^{(0)}$

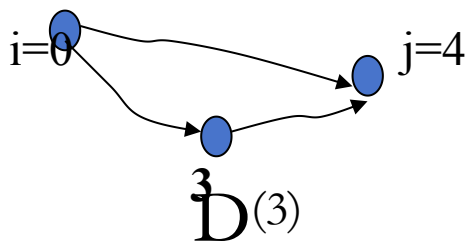
(0,1)
(1,2)

$path^{(1)}$

(0,1,2)

(2,4)

[例续]



$D^{(2)}$

0	0	10	60	30	70
1	∞	0	50	∞	60
2	∞	∞	0	∞	10
3	∞	∞	20	0	30
4	∞	∞	∞	∞	0

$D^{(3)}$

0	0	10	50	30	60
1	∞	0	50	∞	60
2	∞	∞	0	∞	10
3	∞	∞	20	0	30
4	∞	∞	∞	∞	0

$D^{(4)}$

0	0	10	50	30	60
1	∞	0	50	∞	60
2	∞	∞	0	∞	10
3	∞	∞	20	0	30
4	∞	∞	∞	∞	0

$path^{(2)}$

(0,3)

	(0,1,2)	$\uparrow$	(0,1,2,4)
			(1,2,4)
(i,j)			(3,2,4)

$path^{(3)}$

(0,3,2) (0,3,2,4)
(1,2,4)

(i,j) (3,2,4)

$path^{(4)}$

(0,3,2) (0,3,2,4)
(1,2,4)

(i,j) (3,2,4)

[数据结构] 设图中的顶点个数为 n

- 主: 邻接矩阵 $G.arcs[0..n-1][0..n-1]$
- 辅: $-D[0..n-1][0..n-1]$: 记录顶点之间的最短路径长度
 $-path[0..n-1][0..n-1]$: 记录最短路径, 元素为数组类型(线性表)

```
void ShortestPath_FLOYD1(Mgraph G, PathMatrix &Path, DistanceMatrix &D)
{ for (v = 0; v < n; v++)
    for (w = 0; w < n; w++) {
        D[v][w] = G.arcs[v][w]; //初始化 $D^{(-1)}$ 
        path[v][w] =  $\phi$ ;
        if (D[v][w] < INFINITY)
            path[v][w] = [v]+[w]; //初始化直接路径
    }
    for (k = 0; k < n; k++) //依次引入中间顶点
        for (v = 0; v < n; v++)
            for (w = 0; w < n; w++)
                if (D[v][k] + D[k][w] < D[v][w]) { //如果经过中间点k能优化长度
                    D[v][w] = D[v][k] + D[k][w];
                    Path[v][w] = Path[v][k]+Path[k][w];
                }
    }
} // ShortestPath_FLOYD1
```

```

void ShortestPath_FLOYD2(Mgraph G, PathMatrix &P, DistanceMatrix &D)
// P[v][w][k]为TRUE, 则从v到w的最短路径中含有k节点
// D[v][w]从v到w的最短路径的长度
{ for (v = 0; v < n; v++)
    for (w = 0; w < n; w++) {
        D[v][w] = G.arcs[v][w];
        for (k = 0; k < n; k++) p[v][w][k] = FALSE;
        if (D[v][w] < INFINITY) //存在直接路径
            P[v][w][v] = P[v][w][w] = TRUE; //从v到w的路径中包括v和w
    }
for (k = 0; k < n; k++) //依次加入中间顶点
    for (v = 0; v < n; v++)
        for (w = 0; w < n; w++)
            if (D[v][k] + D[k][w] < D[v][w]) { //如果经过中间点k能优化长度
                D[v][w] = D[v][k] + D[k][w];
                for (i = 0; i < n; i++)
                    P[v][w][i] = P[v][k][i] || P[k][w][i];
                //v经k到w的路径包含的节点为v到k的路径节点 || k到w的路径节点
            }
} // ShortestPath_FLOYD2

```